

ARM PrimeCell

SDRAM Controller (PL170)

Design Manual

ARM PrimeCell SDRAM Controller (PL170) Design Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell SDRAM Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	SDRAM CONTROLLER DESIGN OVERVIEW	3
4.1	Features	3
4.2	Functional Description	4
4.3	Signal Description	5
4.4	Programmer's Model	7
4.4.1	Functional Mode Registers	7
4.5	Block Partitioning	7
4.5.1	BigEndian	8
4.5.2	AHB Interface	8
4.5.3	Register Block	12
4.5.4	SDRAM Control Engine	13
4.5.5	Pad Interface	14
5	SDRAM CONTROLLER IMPLEMENTATION DETAILS	16
5.1	Design Hierarchy	16
5.2	SDRAM Controller Top Level Block (Sdram)	16
5.3	SDRAM BigEndian module (SdramBigEndian)	16
5.4	SDRAM AHB Interface (SdramAhbif)	17
5.4.1	Request control state machine	20
5.4.2	Data transfer state machine	21
5.4.3	Write control state machine	21
5.4.4	Read control state machine	24
5.5	SDRAM Read Buffer (SdramAhbRdBuf0)	27
5.5.1	Read data valid state machine	28
5.6	SDRAM Write Buffer (SdramAhbWrBuf0 / SdramAhbWrBuf1)	29

5.7	SDRAM Register Block (SdramAhbRegBlk)	31
5.7.1	Refresh control state machine	33
5.8	Power-On Initialization sequence	34
5.9	SDRAM Control Engine (SdramEngine)	35
5.9.1	Command Flow	35
5.10	Arbiter (SdramArbiter)	36
5.10.1	Top Level Arbiter	38
5.10.2	Address, Data, Control Multiplexers.	39
5.11	Arbitration FSM (SdramArbFSM)	39
5.12	DRAM Bank Controller (SdramDramCntl)	40
5.12.1	Generation of Response signals to the Arbiter (SplitEn, SplitWait and Assigned)	41
5.12.2	DRAM FSM selection logic	41
5.12.3	Selection of Command to be posted to the Command Sequencer from the DRAM Bank controller	42
5.13	DRAM control state machine (SdramDramFSM)	42
5.14	Timing Controller (SdramTimCntl)	44
5.14.1	Usage of the ActCntx counters	45
5.14.2	Usage of the RWPRCntx counters	45
5.15	Command Sequencer (SdramCmdSeq)	45
5.15.1	Command state machine	45
5.15.2	Access state machine	47
5.16	Pad Interface (SdramPins)	49
6	SDRAM CONTROLLER AHB TEST DETAILS	50
6.1	SDRAM Test benches	50
6.1.1	VHDL Bus Compliance Test bench	51
6.1.2	Verilog Bus Compliance Test benches	55
6.2	SDRAM Controller Trickbox	61
6.2.1	Trickbox Signal Description	61
6.2.2	Trickbox functional description	62
6.2.3	Trickbox Register Description	63
6.3	Functional Tests	64
6.3.1	CAS 3 Tests	64
6.3.2	CAS 2 Tests, CAS 1 Tests, CAS 2 256M tests and CAS 3 256M tests	68
APPENDIX A	ADDRESS MAPPING	71
A.1	Address Mapping turned ON	71
A.1.1	Address mapping.	71
A.2	Address Mapping turned OFF	73

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	6 December, 1999	First release
B	25 January, 2000	Updated for 256Mbit device support Additions made to section 5.3 Functional Test section 6.3 updated Appendix A updated
C	10 April 2000	Updated for BigEndian support

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM DDI 0159	ARM PrimeCell SDRAM (PL170) Technical Reference Manual
2	PL170 INTM 0000	ARM PrimeCell SDRAM (PL170) Integration Manual
3	ARM IHI 0011	AMBA Specification (Rev 2.0)
4	ARM DUI 0006	AMBA Compliance Test Suite User Guide
5	ARM DDI 0138	Example AMBA System Technical Reference Manual
6	ARM DUI 0092	Example AMBA System User Guide

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AMBA	Advanced Micro-controller Bus Architecture
AHB	Advanced High-performance Bus
APB	Advanced Peripheral Bus
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
RTL	Register Transfer Level
STA	Static Timing Analysis
SDRAM	Synchronous Dynamic Random Access Memory

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell SDRAM Controller. The design description is split between two sections – Section 4 and Section 5. The testbenches and the test programs for functional verification are described in Section 6.

Section 4 presents an overview of the SDRAM Controller design, describing the functional partitioning of the SDRAM Controller and the signal interconnections. Section 5 presents detailed descriptions on the SDRAM Controller design. Section 6 describes the testbench and the test programs used to generate test vectors for functional verification of the SDRAM Controller.

3 INTRODUCTION

The SDRAM Controller is a part of a library of reusable IP blocks, which can be more easily integrated into a typical ASIC design flow.

The SDRAM controller is a macrocell that can track up to four open SDRAM banks simultaneously. For further information on this block refer to the ARM PrimeCell SDRAM (PL170) Technical Reference Manual [Ref. 1]. The ARM PrimeCell SDRAM (PL170) Integration Manual [Ref. 2] describes how the block may be integrated into a typical industry standard ASIC design flow.

The AMBA Compliance Test Suite User Guide [Ref.4] is a source for further information about AMBA bus compliance testbenches.

The Example AMBA System Technical Reference Manual [Ref.5] and User Guide [Ref.6] describe an example microcontroller based on an ARM processor and the low-power, generic design methodology of AMBA.

4 SDRAM CONTROLLER DESIGN OVERVIEW

4.1 Features

- Clock Speed Single Clock Domain core logic - 90 MHz operation
- Bus Interface 32bit AHB Slave
- DMA Interface 32bit AHB Slave – 3 Ports
- External Memory Interface 32bit/16bit SDRAM bus
- SDRAM Configuration 4 Chip Selects, each controls up to 4 devices (x8/x16/x32)
 Each device may be 16Mbit, 64Mbit, 128Mbit or 256Mbit.
 All devices controlled by a Chip select should be of the same type.
- Memory Support Min : 2 MBytes (16-bit external data bus)
 One 16Mbit (2Mbyte) x16 device controlled by Chip Select 0
 Max : 512 MBytes (32-bit external data bus)
 Four 256Mbit (32 Mbytes) x8 devices controlled by Chip Select 0
 Four 256Mbit (32 Mbytes) x8 devices controlled by Chip Select 1
 Four 256Mbit (32 Mbytes) x8 devices controlled by Chip Select 2
 Four 256Mbit (32 Mbytes) x8 devices controlled by Chip Select 3
- Programmable CAS Latency Cas latencies of 1, 2 and 3 supported.
- Programmable Refresh Timer 16-bit timer
- Can actively track accesses to 4 open banks/pages at a time.
- Power Saving Modes Each SDRAM set may be put in to a low power mode by deactivating
 their CKE lines when inactive.
 The off-chip SDRAM bus is driven with the last write data value when
 the bus is idle to prevent the external bus from remaining in tri-state.
 The clocks to the SDRAMs may be dynamically shut down.
- Self Refresh Mode The external SDRAMs may be placed in SELF REFRESH mode under
 Control from a Power Management Unit.
- BigEndian Mode In BigEndian mode all AHB transfers use BigEndian byte order
- Soft macro-cell available in Verilog and VHDL.
- Fully scan insertable design.
- ARM's BusTalk test environment used for functional verification.
- Compatible with ARM7xx, ARM9xx and ARM10xx processors.
- Compatible with AMBA AHB on chip bus systems.

- Data is transferred between the controller and the SDRAM in quad word bursts.
- 3 clock cycle latency from HSEL to SDRAM command issue
- Supports multiplexing of external pins (e.g. address and data) with other memory controllers (SRAM, ROM etc).
- Dynamic shutdown option for external clock.

4.2 Functional Description

The SDRAM controller interfaces SDRAM chips to “system on a chip” ASIC peripherals. The SDRAM controller appears as a standard bus slave on the on-chip bus, and controls the SDRAM on the external bus. There are up to four on chip bus interface ports, which can request access at the same time allowing the controller to hide slow DRAM bank operations while data is being transferred in the foreground. On receiving an access request, the SDRAM controller decides on the appropriate commands to send to the SDRAM memory. The command sequence depends on the current state of the DRAM state machines (DRAM Bank Controller), and the state of other currently active access requests (arbiter and command sequencer).

The DRAM bank controller sequences all of the commands required to complete a read or write request to an SDRAM memory location with timing controlled by the CASDelay[1:0], and RASDelay[1:0] values.

The bus interface is a slave on the main processor bus, it contains the control register block. The bus interface produces read, write, refresh and mode register write requests to the SDRAM control engine, and software supplied configuration information. Data will normally be transferred to and from the SDRAM as unbroken quad words. This is a convenient data packet size for ARM cache line fills, and buffered writes. For accesses smaller than a quad word, extra read data will be ignored by the SDRAM controller, for writes, DQM pins are used to force the SDRAM's to ignore invalid data. For access sizes larger than a quad word, multiple quad word accesses are issued to the SDRAM control engine. For ARM processors which do not contain a write buffer (typically ARM7xx, and some ARM9xx's) a merging write buffer may optionally be included in the Bus Interface. A merging write buffer compacts writes of all widths into quad-word busts, which can be efficiently transferred to the SDRAM.

The SDRAM controller has 4 AHB ports which allow multiple peripherals to have direct access to the SDRAM without data having to pass over a single system bus. The control engine prioritizes accesses and hides DRAM bank activates, restore and pre-fetch commands.

A block diagram of the device is shown in **Figure 4.2-1**.

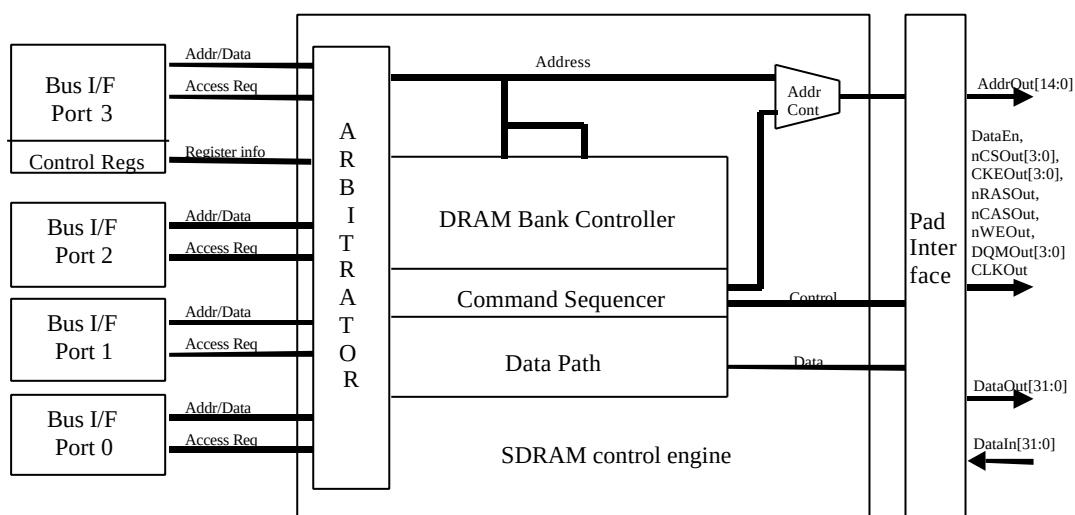


Figure 4.2-1 ARM PrimeCell SDRAM Controller Block diagram

4.3 Signal Description

The following tables summarize the SDRAM Controller signal list. **Table 4.3-1** lists the AHB interface signals while **Table 4.3-2** lists the block specific non-AHB signals.

Signal Name	Type	Source / Destination	Description
HCLK	IN	Clock Generator	AHB Bus Clock
HRESETn	IN	Reset Controller	Bus reset (active low).
HSELreg3	IN	Decoder	Slave select for register accesses
For each of the 4 ports ($n = 0-3$), the following signals are repeated.			
HSIZE $n[1:0]$	IN	Master	Indicates the current transfer size.
HWRITE n	IN	Master	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
HTRANS $n[1:0]$	IN	Master	AHB Transfer type
HBURST $n[2:0]$	IN	Master	AHB Burst type
HREADY inn	IN	AHB Bus	AHB transfer done
HSELram n	IN	Decoder	Slave select for memory accesses
HADDR $n[28:0]$	IN	Master	This is part of the AHB address bus, which is used for decoding the internal address space.
HWDATAN $n[31:0]$	IN	Master	AHB Write Data bus.
HRDATAN $n[31:0]$	OUT	AHB Bus	Read Data to AHB
HRESP $n[1:0]$	OUT	Master	AHB transfer response
HREADYout n	OUT	Master	AHB transfer done

Table 4.3-1: AHB signal descriptions

Signal Name	Type	Source / Destination	Description
CLKIn	IN	System	Fed back version of the clock to the external SDRAMs.
CLKOut	OUT	SDRAM	Clock output to the SDRAMs.
DataIn[31:0]	IN	SDRAM	Data input from the SDRAMs
DataOut[31:0]	OUT	SDRAM	Data output to the SDRAMs
DQMOut[3:0]	OUT	SDRAM	Data Mask output to the SDRAMs
DataEn	OUT	PAD	SDRAM pad enables.
AddrOut[14:0]	OUT	SDRAM	Address output to the SDRAMs
CKEOut[3:0]	OUT	SDRAM	Clock Enable outputs to the SDRAMs
nCSOut[3:0]	OUT	SDRAM	Chip Selects to the SDRAMs
nRASOut	OUT	SDRAM	Row Address Strobe to the SDRAMs
nCASOut	OUT	SDRAM	Column Address Strobe to the SDRAMs
nWEOut	OUT	SDRAM	Write Enable to the SDRAMs.

nPOR	IN	Reset Controller	Power on Reset.
SREFReq	IN	PMU	Self Refresh Request
SREFAck	OUT	PMU	Self Refresh Acknowledge
ExtBusReq	OUT	External Bus Arbiter	Bus Request for External Memory address and data bus
ExtBusGnt	IN	External Bus Arbiter	Bus Grant for External Memory address and data bus
ExtCtlReq	OUT	External Bus Arbiter	Bus Request for External Memory control bus
ExtCtlGnt	IN	External Bus Arbiter	Bus Grant for External Memory control bus
SCANENABLE	IN	Test Controller	This signal is used by the Test Controller during Scan testing to shift in and shift out Scan data.
SCANIN	IN	Test Controller	Scan Data Input
SCANOUT	OUT	Test Controller	Scan Data Output
BIGENDIAN	IN	System	Sets all AHB ports to BigEndian byte lane order

Table 4.3-2 Block specific signal description

4.4 Programmer's Model

4.4.1 Functional Mode Registers

Address	Type	Width	Reset Value	Name	Description
SDRAM register base address + 0x00	R/W	32	0x01F00000	Configuration Register 0	Configuration register that contains memory related parameters.
SDRAM register base address + 0x04	R/W	5 4	0x0000000C	Configuration Register 1	Configuration register that contains non-memory parameters. If the design is configured with both read and write buffers, then 4 bits are writeable.
	R/W	4 3	0x00000008		If the design is configured with two write buffers, then 3 bits are writeable.
	R/W	4 3	0x00000004		If the design is configured with one read buffer, then 3 bits are writeable.
	R/W	3 2	0x00000000		If the design is not configured with read and write buffers, then 2 bits are writeable.
SDRAM register base address + 0x08	R /W	16	0x00000080	Refresh Register	Refresh count (number of clocks between two successive refresh requests) is stored in this register
SDRAM register base address + 0x0C	R/W	16	0x00000000	Write Timeout Register	Contains the write timeout value. This register is writeable only if the design is configured with two write buffers.
		0	0x00000000		If the design is not configured with the write buffers, this register is not writeable and returns zero when read.

Please see the ARM PrimeCell SDRAM Controller (PL170) Technical Reference Manual [Ref. 1] for a detailed description of each register.

4.5 Block Partitioning

The sub-blocks in the design are listed below.

- BigEndian blocks. These invert the two least significant address bits of each AHB port when BigEndian byte lane order is selected (BIGENDIAN signal asserted).
- AHB Interface blocks (Includes the Write buffers, the Read buffer and the core Bus interface logic. The Register block is included in the AHB interface block for Port 3)
- SDRAM Control Engine (Includes the Arbiter, the DRAM Bank Controller and the Command Sequencer)
- Pad Interface (Includes synchronizers for data and control signals between the internal device clock and the fed-back external clock)

The design of the above-mentioned sub-blocks is detailed below.

4.5.1 BigEndian

The two least significant bits of each AHB address bus connect to a BigEndian module. A BigEndian module is instantiation for each AHB port. Each BigEndian module output connects to the two least significant address bits of the corresponding AHB port. When asserted the common BIGENDIAN signal causes each BigEndian module to invert the two least significant address lines of each AHB address bus, which selects BigEndian byte lane order. Internal operation of the SDRAM controller is not affected.

4.5.2 AHB Interface

The AHB interface consists of two write buffers, a read buffer, registers and the control logic. The registers are instantiated in the top level of the SDRAM controller (Sdram.v). The AHB interface deals with all the bus transfers to both the memory and registers. The main characteristics of the AHB interface are:

- Supports both the Non-sequential and Sequential transfers of the AHB.
- Does not put any restriction on the number of transfers in a burst of word transfers
- OKAY response for all data transfers (not a SPLIT or RETRY capable slave).
- Multiple quad word transfers for INCR4 / INCR8 / INCR16 accesses.
- Supports 32 bit AHB and 32bit or 16 bit off chip data buses.

The read and write buffers can be optionally included in the AHB interface. The parameters NO_OF_READ_BUFFERS and NO_OF_WRITE_BUFFERS control their inference in the design. These parameters are defined in SdramDefs.v file. There can be only one read buffer, which can store up to one quad word of data, in the AHB interface. Hence the parameter NO_OF_READ_BUFFERS should be set to 1 to infer the read buffer. For other values of the parameter, the read buffer will not be inferred at all.

There are two write buffers, each can store up to one quad word of data, performing the merging of write data. When one buffer writes the data to the memory the other write buffer merges the write data from the AHB bus. Both these buffers have to be included in the AHB interface. The parameter NO_OF_WRITE_BUFFERS should be set to 2 to infer both the write buffers in the AHB interface. For other values of the parameter, write buffers will not be inferred at all.

Functional description of the AHB interface

As mentioned earlier, the AHB interface handles all the transactions initiated on the AHB bus that targets the memory and configuration registers. The following is a description of the functionality of the AHB interface in detail.

Register read access

All register read accesses will return an OKAY response without any delay. i.e., there are no WAIT responses for a register read access.

Register write access

All register write accesses will return an OKAY response without any delay. i.e., there are no WAIT responses for a register write access.

Note that configuration register 0 (Sdram register base + 0x00) contains memory related parameters such as device type, bank size, RAS and CAS delays etc. The contents of this register are directly connected to the memory control engine. Hence this register should not be modified when the memory is being accessed by the control engine. An AHB master that wants to modify the contents of configuration register 0 should sample memory control engine status bit (Bit 5 in configuration register 1) before doing so. The initiator of the register write access should ensure

that there are no read or write access to any of the four ports of the SDRAM controller. Then it should read the configuration register 1 (Sdram register base + 0x04) to monitor the memory control engine status bit. Any write to configuration register 0 should be initiated only after ensuring that the memory control engine is idle.

Memory write access

For an AHB write access the behavior of the AHB interface depends on the inference of the write buffers. When the AHB interface is not configured with write buffers, any write access from the AHB interface will initiate a write request to the memory. The Write3 signal is asserted to initiate a memory write request. The address from the AHB bus is driven on AddrIn3 bus to the control engine along with Write3. The first write data has to be provided to the engine along with the request. Hence the first write data gets a default OKAY response, from the AHB interface, on the bus. The assertion of XferOut3 by the memory control engine indicates that it takes the first write data on the same rising edge of HCLK. Subsequent data should be driven on DataIn3 on the same rising edge of HCLK. The DQM3 signals are generated from the address and bus size information from the AHB master. The control engine can take one quad word of data for every write request. The second data transfer takes place on the same clock XferOut3 is next sampled asserted. This assertion of XferOut3 is based on the memory bus grant signal and hence the XferOut3 may not be continuous for a write request. There can be dead clocks between the first and second XferOut3. The signal EndOfXfer3 is asserted along with the third data transfer. It indicates that the engine can take the fourth data of the request at the next rising edge of HCLK. The start address of the request is the same word address with which the AHB master initiated the write request. Hence if the AHB transaction type is WRAP4 or quad word aligned INCR4, successive data provided by the AHB master can be written to the memory in the same request. For other transaction types the data is taken from the AHB bus only if it matches with the address to which the data is being written. If the AHB master terminates the transaction before writing all the four words of a quad word, DQM3 will be de-asserted ("1111") to indicate the memory that the data is invalid.

Two write buffers

The AHB interface can be configured with two write buffers by setting the parameter NO_OF_WRITE_BUFFERS to 2 in the SdramDefs.v file. The valid values for this parameter are 0 or 2 only. If this parameter is set to anything other than 2, then the write buffers are not inferred in the design.

The write buffers are useful when the processor does not have a write cache. Write buffers merge data from the AHB master before writing the data to memory, to improve the performance.

An AHB write access is called a write hit when the write buffer is empty, or when the write buffer contains data and the AHB write access is to the same quad word address as data already in the write buffer.

An AHB write access is called a write miss when the write buffer contains valid data and the AHB write access is to an address different from the one in the write buffer.

The two write buffers in the AHB interface are called "write buffer 0" and "write buffer 1" respectively. Initially both write buffers are empty (after reset). Any AHB access is evaluated against the address and data valid bit of the write buffer being pointed to by the write buffer select signal. The write buffer select signal points to write buffer 0 after reset. Hence the first write access is a write hit to write buffer 0, by default. This write buffer 0 data will be written to the memory only when:

- An AHB write access is initiated on the bus to a different quad word address
- An AHB read access is initiated on the bus to the same quad word address in write buffer
- Write buffer time out has occurred
- A long burst read or write access in which the quad word address in write buffer becomes part of the targeted address range. (Please refer to the sub-section on long burst read / write accesses later on).

In the first case, the AHB write access is a write miss access to write buffer 0. Therefore the write buffer 0 will initiate a write request to the memory and write all the quad word of data available with it. The AHB write access will be taken by the other write buffer, write buffer 1. This happens without any WAIT response. The write buffer select signal also switches to write buffer 1 now. Any new AHB write access will be evaluated against the quad word

address in write buffer 1. If it is a write hit to write buffer 1, then the data will be merged within the write buffer. If it is to another quad word address, then write buffer 0 will take this data after the memory write is over. The write buffer 1 will initiate a write request to the memory. The write buffer select signal switches back to write buffer 0 now. In this way, the two write buffers switch their functions after every write miss access from the AHB master.

In the second case, the AHB read access is a read hit to write buffer 0. Therefore the write buffer has to update the data in the memory before reading the whole quad word from it. Hence a write request will be initiated first to write the data to the memory. After the write is over, the read access to the memory will take place.

In the third case, the write buffer time out limit has been reached. The time limit is set by the write time out value in the register block. The write time out counter loads the write time out register value when the write buffer takes in the latest write data. It counts down on every rising clock edge and issues the write time out signal when the counter reaches zero. The write timeout signal initiates a write access to the memory and writes the data from the buffer to the memory.

The write buffers can be disabled using the write buffer enable bit (Bit 3 in the configuration register 1). Software can use this write buffer enable bit to control the write buffers in the design. By default this bit will be set if the design is configured with write buffers. If the design is not configured with write buffers, then this write buffer enable bit will be reset and it becomes a read-only bit. If the software decides not to use the write buffers, then it has to reset the write buffer enable bit. Any valid data in the write buffers will be written to the memory before the write buffers are disabled and the state machine in the top level of the AHB interface will then handle any new write access from the AHB bus.

Long burst write access:

An AHB write access with HBURST as SINGLE / INCR / WRAP4 / INCR4 is considered a normal access. As the amount of data written is less than or equal to a quad word of data, they are considered a normal accesses that can transfer a quad word of data to the memory for every write request.

Note : An INCR4 access should start with the first word of the quad word (i.e. HADDR[3:2] = "00").

An AHB write access with HBURST as INCR4 starting with miss-aligned quad word address / WRAP8 / INCR8 / WRAP16 / INCR16 is considered as a long burst access. Here it is assumed that the master intends to access more than a quad word of data, hence this called a long burst access. The engine input ReqNumCmd3[2:0], part of the request control signals. It indicates the number of quad-words to be transferred in a single request. The following table gives the relationship between ReqNumCmd3 and the total number of data transfers for a read / write request.

ReqNumCmd3[2:0]	Amount of data transferred
000	One quad word
001	Two quad words
010	Three quad words
011	Four quad words
100	Five quad words
101	Six quad words
110	Seven quad words
111	Eight quad words

The following are the requirements for a write request to be considered as a long burst write access.

- The data transfer size should be word.
- The master is not expected to insert BUSY cycles throughout the write access.

The first requirement is mandatory. If the data transfer size is not a word, it is not possible to write data to the memory continuously during every rising edge of HCLK. The second requirement ensure maximum performance from the AHB interface. If it is not met, then successive AHB accesses may suffer due to the large number of data transfers to be completed in a long burst access. The long burst write access is handled by the write control state machine in the top level of the AHB interface. It does not go through the write buffers as the write buffers can handle a maximum of only a quad-word of data. If the write buffer contains any data that falls in the same address range as the long burst access, then a write request is initiated from the write buffer first and then the long burst write request is initiated to the memory.

Memory read access

The behavior of the AHB interface depends on the inference of the read buffers, for an AHB read access. When the design is not configured with the read buffer, the AHB interface has to access the memory for each read access. When there is a read access, the AHB interface initiates a read request by asserting the Read3 signal. The assertion of XferOut3 from the control engine indicates that the read data will be provided from the next rising edge of HCLK. A quad word of data will be provided for every read request. On sampling the assertion of XferOut3, the AHB interface returns an OKAY response to the AHB bus in the next rising edge of HCLK along with the first data from the memory. If the AHB master also needs one quad word of data, then the same can be provided in the next four clocks, without any WAIT response. For this to happen, the transaction has to be a WRAP4 access with the HSIZE as word or it has to be a quad word aligned INCR4 / INCR access with HSIZE as word. For other transactions types, the data can be returned only if the address is the same word for which the data is being fetched from the memory. If the AHB master terminates the read access early, the rest of the data from the memory is discarded by the AHB interface. Every new read access will initiate a read request to the memory.

If the design is configured with write buffers and if the read access is to the same quad word of data in the write buffer, the write buffer will initiate a write request to the memory. The read request can be initiated only after the write access is over.

When the design is configured with write buffers, it is possible that there is already a write request proceeding, due to a write time out, at the same time the read access is initiated on the AHB bus. In that case the read request will be initiated only after the write data transfer has started.

The memory control engine ignores DQM3 for a read request. Therefore memory provides the whole quad word of data for any read request. If the AHB master has not requested a word access, the byte steering logic in the top level of AHB interface (SdramAhbif) takes care of replicating the data on the correct byte lane.

Read buffer

The AHB interface can be configured with the read buffer using the parameter NO_OF_READ_BUFFERS in SdramDefs.v file. This should be set to 1 to infer the read buffer in the design. For any other value the AHB interface will not contain the read buffer.

If the AHB master initiates a read access to the same quad word of data that is available in the read buffer, then it is called a read hit to the read buffer.

If the AHB master initiates a read access to a different quad word of data from the one that is available in the read buffer, then it is called a read miss to the read buffer. If the read buffer does not contain valid data, any read access will fetch the data from memory. Hence it is also a read miss to the read buffer.

When the AHB interface is configured with the read buffer, any AHB read access is evaluated for a hit to the read buffer. If the AHB access is a hit, data will be returned from the read buffer without any WAIT states. If the AHB read access is a read miss, data must be fetched from memory. When XferOut3 is sampled asserted, the OKAY response will be returned to the AHB master and the data is returned directly from the memory, as in the case when the design has no read buffer. Read data is also latched into the read buffer and the read data valid bit is set in the read buffer. Any future read hit access will return data directly from the read buffer.

The read buffer can also be disabled using the read buffer enable bit (Bit 2 in the configuration register 1). Software can use the read buffer enable bit to control the read buffer in the design. By default this bit will be set if the design is configured with the read buffer. If the design is not configured with the read buffer, the read buffer enable bit will be reset and is read-only. If the software decides not to use the read buffer, then it should reset the read buffer enable bit. The state machine in the top level of the AHB interface will then handle any read access from the AHB bus.

If there is a write transaction initiated on the AHB bus to the same quad word of data in the read buffer, then the data in the read buffer is invalidated by setting the read data valid bit to logic zero. This ensures that any future access has to fetch the data from memory. A read access hit to the write buffer will initiate a write request to the memory before the read request.

Long burst read access:

An AHB read access with HBURST as SINGLE / INCR / WRAP4 / INCR4 is considered a normal access. As the amount of data read during the access is less than or equal to a quad word of data, A normal access transfers a quad word of data from memory for every read request.

Note : An INCR4 access should start with the first word of the quad word (i.e. HADDR[3:2] = "00"). An AHB read access with HBURST as INCR4 starting with miss-aligned quad word address / WRAP8 / INCR8 / WRAP16 / INCR16 is considered a long burst access. The long burst request accesses more than a quad word of data in the memory. The signal ReqNumCmd3 is used to control the number of quad word data transfers for a long burst read request.

The following are the requirements for a read request to be considered a long burst access.

- The data transfer size should be a word.
- The master is not expected to insert BUSY cycles during the read access.

The first requirement is mandatory. If the data transfer size is not a word, it is considered as a normal access. The second requirement ensures maximum performance from the AHB interface. If it is not met, then successive AHB accesses may suffer due to the large number of data transfers needed to complete a long burst access.

The long burst read access is handled by the read control state machine in the top level of the AHB interface. The read buffer does not store the long burst read data, as it can handle a maximum of only a quad-word of data. If the write buffer contains any data that falls in the address range of the long burst access, a write request is initiated from the write buffer before the long burst read request is initiated to the memory.

4.5.3 Register Block

The register block contains the configuration registers, refresh count register, write time out register and optional lock registers for all four ports. It also contains the refresh counter, refresh control state machine and control logic to update the registers and decode logic to handle register access.

Whenever there is an AHB write access to a register, the AHB interface returns the OKAY response and asserts a write enable to the register block. This write enable signal is used along with address and bus size information to update the corresponding byte of data in the selected register.

A register read returns all the bits in the register (each register is 32 bits wide) to the AHB interface. The AHB interface performs the necessary byte steering and passes on the data to the AHB bus.

The refresh counter is a 16 bit free running counter in the register block. It loads the default value (0x0080) on reset. It counts down on every rising edge of HCLK. It loads the refresh count from the refresh count register and raises the refresh request when:

- The count reaches zero
- There is a new refresh value written into the refresh count register.

The refresh request state machine controls the refresh request to the memory engine.

This module contains some of the parameterized bits and registers.

- The write buffer enable bit is inferred only if the parameter NO_OF_WRITE_BUFFERS is set to 2.
- The read buffer enable bit is inferred only if the parameter NO_OF_READ_BUFFERS is set to 1.
- The write time out register is inferred only if the parameter NO_OF_WRITE_BUFFERS is set to 2.

If the parameters are set to anything other than those specified above these bits are not inferred in the design and they are read only bits.

4.5.4 SDRAM Control Engine

The SDRAM Control Engine instantiates the Arbiter, the DRAM Bank Controller and the Command Sequencer. A simplified block diagram of the SDRAM Control Engine is shown in **Figure 4.5-1**.

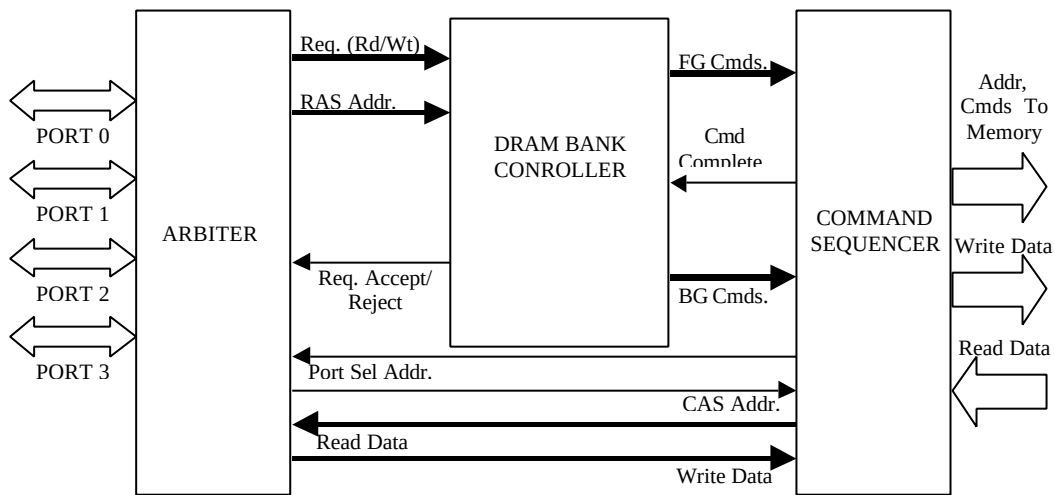


Figure 4.5-1 Block Diagram of the SDRAM Control Engine

Requests from the Arbiter are handled by the DRAM Bank Controller. The DRAM Bank Controller generates the sequence of device-level commands such as PRE, ACT, RDA, WTA etc. to service the request and issues these commands to the Command Sequencer. The Command Sequencer translates commands from the DRAM Bank Controller into the appropriate binary values on the memory pins.

Arbiter

The Arbiter prioritizes requests from the Bus interface ports to select one request to be issued to the rest of the control engine. Selection of access requests is done on a priority basis with Port 0 having highest priority and Port 3 having the lowest. The fixed priority scheme is further modified by a two-state re-arbitration state machine which attempts to route requests with the highest chance of being accepted to the DRAM FSMs.

DRAM Bank Controller

The DRAM Bank Controller responds to requests from the Arbiter. The DRAM Controller can track accesses to up to four different banks simultaneously. Each bank is tracked by one of four DRAM FSM blocks instantiated in the DRAM Bank Controller. The DRAM FSMs convert the access requests from the Arbiter to the appropriate sequence of device-level commands to be issued to the SDRAMs. The DRAM FSMs also ensure compliance to any timing parameters that govern the separation between these commands. Commands from the DRAM FSMs are serialized

by the DRAM Bank Controller and then issued to the Command Sequencer. The DRAM FSMs also handle Mode Register Set requests from Port 3.

The DRAM Bank controller instantiates a Timing Controller. The Timing Controller tracks inter-bank timing parameters and other timing parameters that govern the issue of AREF, SREF and MRS commands.

Figure 4.5-2 shows a simplified block diagram for the DRAM Bank Controller.

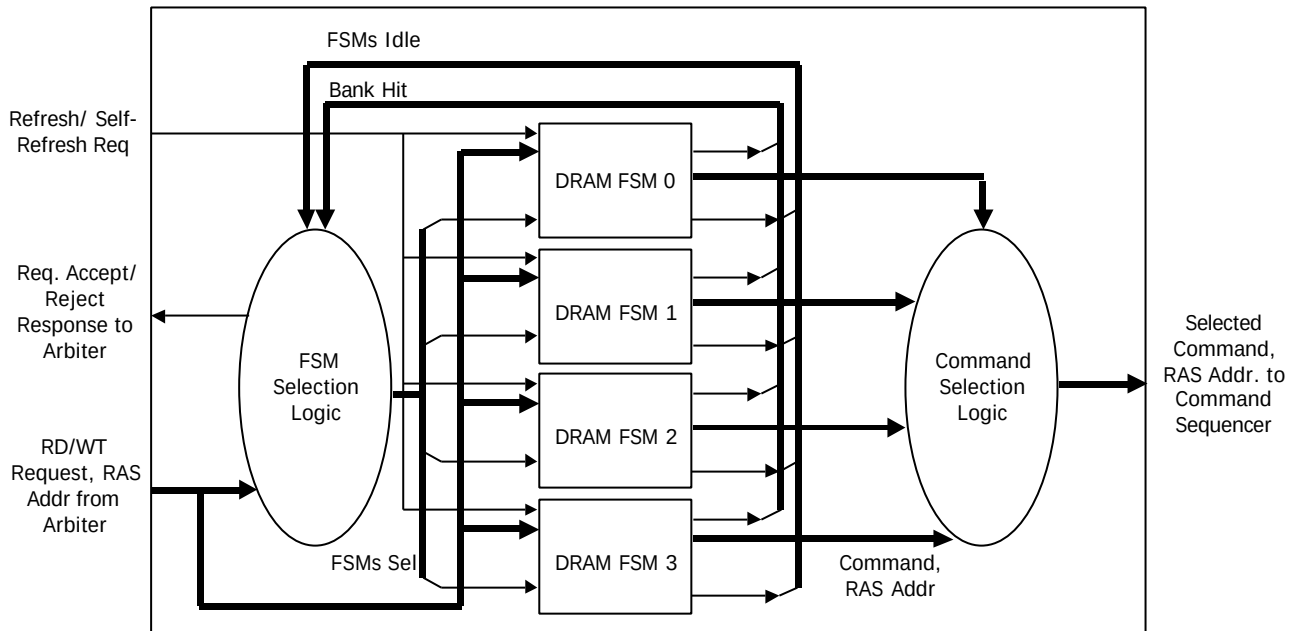


Figure 4.5-2 Block diagram of the DRAM Bank Controller

Command Sequencer

The Command Sequencer converts commands from the DRAM Bank controller to corresponding binary values on the memory pins. The Command Sequencer has a Command State machine and an Access State machine. The Command State machine handles background commands (non-data access commands such as ACT, PRE, AREF, SREF etc.) The Access State machine handles foreground commands (data access commands such as RD, RDA, WT and WTA). The Command Sequencer can buffer up to 4 foreground commands, two of these are buffered within the Access State machine. By buffering foreground commands within the Access State machine, successive data access commands are issued with the optimal separation, thereby facilitating the efficient use of the memory databus.

4.5.5 Pad Interface

This block synchronizes memory-related output signals to the fed-back external clock, CLKIn, on which the memory devices operate. Similarly Read data from the memory devices is clocked in on CLKIn and fed to the SDRAM Control Engine. CLKOut, the clock on which the memory devices operate, is driven from this block. Optionally, CLKOut can be dynamically shutdown when the SDRAMs are not being accessed.

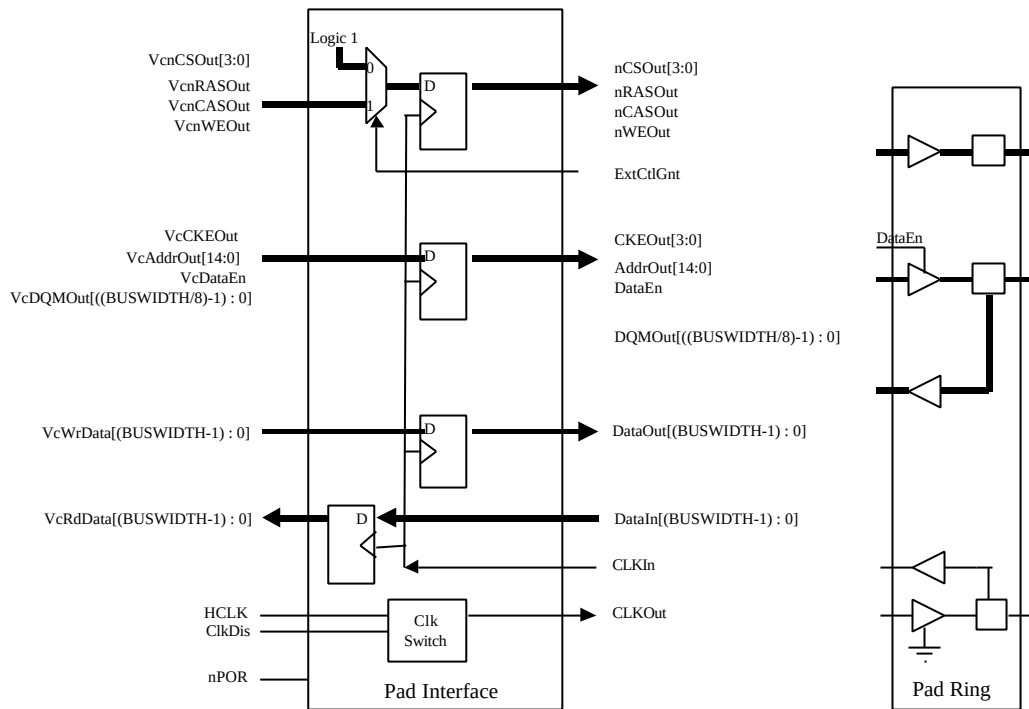


Figure 4.5-3 Block diagram for the Pad interface

5 SDRAM CONTROLLER IMPLEMENTATION DETAILS

5.1 Design Hierarchy

The design hierarchy in the SDRAM Controller is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-blocks is also given within brackets.

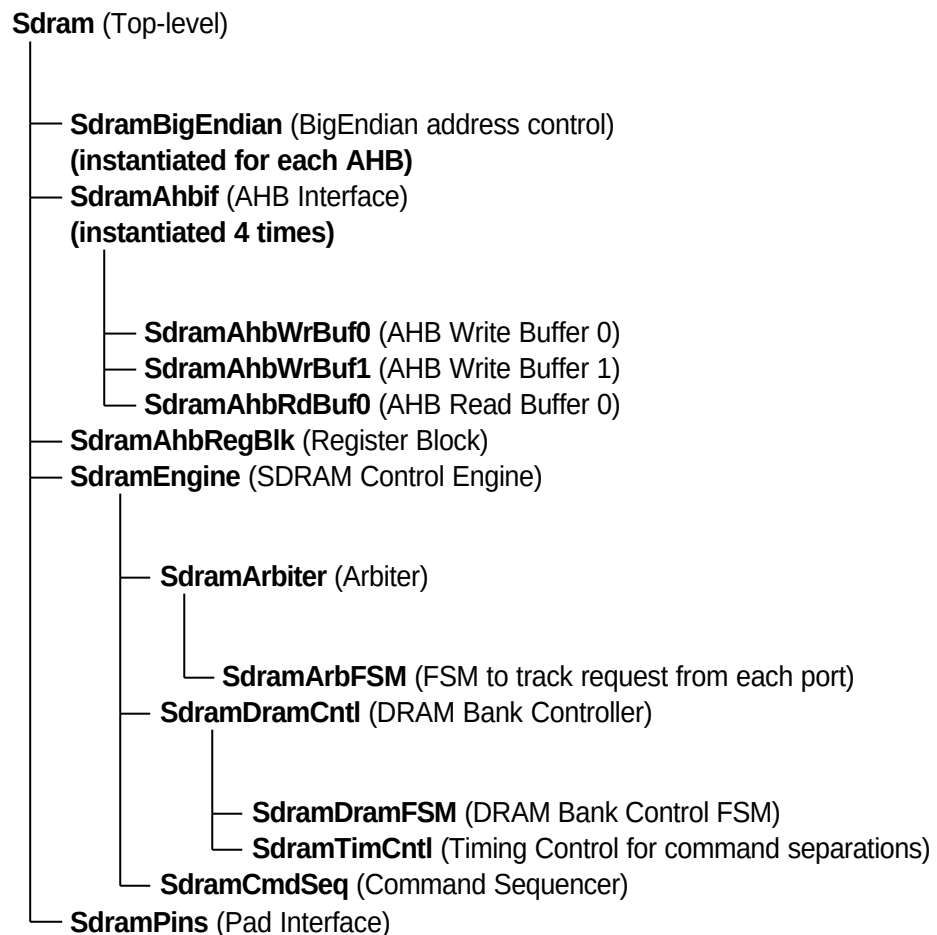


Figure 5.1-1 SDRAM Design Hierarchy

5.2 SDRAM Controller Top Level Block (Sdram)

The top-level block is a structural block which instantiates and interconnects the main functional blocks in the SDRAM Controller.

5.3 SDRAM BigEndian module (SdramBigEndian)

This module instantiates two XOR gates which are used to invert the two least significant address lines when the shared control input signal (BIGENDIAN) is asserted.

5.4 SDRAM AHB Interface (SdramAhbif)

The top level of the AHB interface has two write buffers and a read buffer instantiated. These buffers are optional. The AHB interface can be configured with or without these buffers. The write and read buffers can be separately controlled. The inference of write and read buffers depends on the parameters WR_BUF_COUNT and RD_BUF_COUNT respectively. The constants NO_OF_WRITE_BUFFERS and NO_OF_READ_BUFFERS are assigned to the parameters respectively. The parameters are passed to the instances from this top-level block.

There are two active low reset inputs to the AHB interface. The bus reset HRESETn and power-on-reset nPOR. All the state machines and the registers are reset on assertion of either of them, by an internally generated combined reset signal called nReset.

There are two signals, SplitProceed3 and SplitWait3, related to the AHB split response. The memory control engine asserts SplitWait3 to indicate when it wants to initiate a request to the memory and waits for the assertion of SplitProceed3 before proceeding. The AHB interface is expected to assert SplitProceed3 when the AHB master continues with a bus transaction that was split. Currently there is no split response returned by the AHB interface. Hence SplitWait3 is directly connected to SplitProceed3.

As the AHB is a pipe-lined bus, the address (AddrD1[28:0]) and control signals are latched on the rising edge of HCLK qualified with HREADYin. The assertion of HREADYin indicates that there is a response returned on the AHB bus by a slave and the address and control signals on this clock edge correspond to the next access. Moreover the AHB interface returns a default OKAY response for the first data transfer. The address corresponding to this data transfer should be latched to be driven on AddrIn3. At the time of transferring the data there can be another access on the AHB bus to the memory and this address and control information are valid only for one HCLK cycle. Therefore a second address latch (AddrD2[28:0]) is also used. For a read request to the memory engine, the address is driven from AddrD1 and for a write access it is driven from AddrD2.

The ReqNumCmd3 is controlled by the transfer size and burst information. Any INCR4 started with miss-aligned quad word address / WRAP8 / INCR8 / WRAP16 / INCR16 access, with the data transfer size as word, is considered as a long burst access. An INCR4 started with miss-aligned quad word address / WRAP8 / INCR8 access will be initiated with "010" on the ReqNumCmd3 lines, requesting two quad word data transfers for a read / write request. A WRAP16 / INCR16 access will be initiated with "011" on the ReqNumCmd3 lines, requesting four quad word data transfers for a read / write request. For this purpose the HSIZE and HBURST signals are separately latched into Lsize and LBurst on a rising edge of HCLK. The latches Lsize and LBurst may take their input either from the bus or from the clocked control signals such as SizeD or BurstD. If a bus request is immediately raised as a read / write request to the control engine, then they latch the bus signals directly. Otherwise the clocked control signals are latched.

It was explained in the functional description that the ReqNumCmd3 signal is used to indicate the total number of data transfers requested by the AHB interface for the corresponding read / write request. This ReqNumCmd3 should be as follows:

- For all Non-Long burst transactions, it should always be "000" i.e., One quad word for every request
- For a long burst access if the number of quad words indicated by the burst size is in the same page then ReqNumCmd3 is same as the burst size requested by the master.
- For a long burst access if all the quad words indicated by the burst size are not in the same page for an SDRAM access, then ReqNumCmd3 is generated such that it will request data till the end of the current page. At the end of the page a new request is initiated to transfer the rest of the data.

Signals SDQWCount[9:4] indicate the number of quad words available in the same page for an SDRAM access. The ReqNumCmd3 is generated by comparing it with the requested burst size. This range of address is based on the AHB specification that the master cannot cross the 1KB boundary during any access.

The following table indicates the transaction type of AHB transfers and the expected number of memory requests initiated to the memory with the corresponding ReqNumCmd3[2:0] values.

Note : It is assumed that the master does not insert any busy cycle in-between and the requested number of data does not cross a page boundary.

X – indicates that the address is a don't care bit.

“-“ - indicates that the transaction can be completed with one read / write request to the memory engine.

Transaction type	AHB Address HADDR[5:2]				ReqNumCmd3 Value	
	HADDR[5]	HADDR[4]	HADDR[3]	HADDR[2]	1 st Request	2 nd Request
SINGLE	X	X	X	X	0	-
INCR	X	X	X	X	0	-
WRAP4	X	X	0	0	0	-
WRAP4	X	X	0	1	0	-
WRAP4	X	X	1	0	0	-
WRAP4	X	X	1	1	0	-
INCR4	X	X	0	0	0	-
INCR4	X	X	0	1	0	-
INCR4	X	X	1	0	0	-
INCR4	X	X	1	1	0	-
WRAP8	X	0	0	0	1	-
WRAP8	X	0	0	1	1	0
WRAP8	X	0	1	0	1	0
WRAP8	X	0	1	1	1	0
WRAP8	X	1	0	0	0	0
WRAP8	X	1	0	1	0	1
WRAP8	X	1	1	0	0	1
WRAP8	X	1	1	1	0	1
INCR8	X	0	0	0	1	-
INCR8	X	0	0	1	2	-
INCR8	X	0	1	0	2	-
INCR8	X	0	1	1	2	-
INCR8	X	1	0	0	1	-
INCR8	X	1	0	1	2	-

INCR8	X	1	1	0	2	-
INCR8	X	1	1	1	2	-
WRAP16	0	0	0	0	3	-
WRAP16	0	0	0	1	3	0
WRAP16	0	0	1	0	3	0
WRAP16	0	0	1	1	3	0
WRAP16	0	1	0	0	2	0
WRAP16	0	1	0	1	2	1
WRAP16	0	1	1	0	2	1
WRAP16	0	1	1	1	2	1
WRAP16	1	0	0	0	1	1
WRAP16	1	0	0	1	1	2
WRAP16	1	0	1	0	1	2
WRAP16	1	0	1	1	1	2
WRAP16	1	1	0	0	0	2
WRAP16	1	1	0	1	0	3
WRAP16	1	1	1	0	0	3
WRAP16	1	1	1	1	0	3
INCR16	0	0	0	0	3	-
INCR16	0	0	0	1	4	-
INCR16	0	0	1	0	4	-
INCR16	0	0	1	1	4	-
INCR16	0	1	0	0	3	-
INCR16	0	1	0	1	4	-
INCR16	0	1	1	0	4	-
INCR16	0	1	1	1	4	-
INCR16	1	0	0	0	3	-
INCR16	1	0	0	1	4	-
INCR16	1	0	1	0	4	-
INCR16	1	0	1	1	4	-
INCR16	1	1	0	0	3	-

INCR16	1	1	0	1	4	-
INCR16	1	1	1	0	4	-
INCR16	1	1	1	1	4	-

It may be noted that there are not more than two requests to the memory engine for any type of the AHB transfer. Moreover the following transactions can be completed with one request to the memory.

- All SINGLE / INCR / WRAP4 accesses – Non long burst requests
- All INCR4 / INCR8 / INCR16 accesses – Long burst requests
- INCR4 access starting with quad word aligned address (HADDR[3:2] = “00”). – Long burst requests
- WRAP8 access starting with double-quad word aligned address (HADDR[4:2] = “000”) – Long burst requests.
- WRAP16 access starting with quad-quad word aligned address (HADDR[5:2] = “0000”) – Long burst requests.

There are two 4-bit counters, one each for read and write data transfers. They indicate the word data that is being transferred by the AHB interface to the memory. The counters load the address of the second data word to be transferred for every request. Then increment for every data transfer. This way the address of the next word of data to be transferred is directly generated. They are used to generate MemWrAddr and MemRdAddr that indicate the address of the data being transferred for a write / read request respectively. The latched bus address AddrD1 is always compared with this address. The data on the bus is taken only if it happens to be the next data to be transferred.

5.4.1 Request control state machine

The AHB interface has different sources from which a read or write request is initiated. The request can be initiated from the write control state machine, either of the write buffers, the read control state machine or the read buffer. It is also possible that there are two accesses initiated on the AHB bus to the memory. As the AHB interface returns a default OKAY response for the first write data for any write request, there can be another new AHB access initiated immediately to the memory, on the AHB bus. To avoid every source from monitoring the other requests this state machine is introduced.

This routes the read or write request from the corresponding source to the Read3 or Write3 signals respectively. Moreover it ensures that either Read3 or Write3 request is initiated to the memory at a time. On reset it goes to Idle state. Any request can be passed at this time. Once a request is sampled asserted it switches to the corresponding state and passes only the request from that particular source. The following table gives the source and the corresponding state.

Source of request	State defined
Write control state machine	ST_REQ_WRITE
Read control state machine	ST_REQ_READ
Write buffer 0	ST_REQ_WRBUF0
Write buffer 1	ST_REQ_WRBUF1
Read buffer	ST_REQ_RDBUF
No Request	ST_REQ_IDLE

The order in which the requests are processed is important in this state machine. When the write buffers are enabled, there can be an AHB access that hits the write buffer. In this case the write buffer has to write the data to the memory before the AHB access proceeds. The write buffer and the top level state machines raise the requests at the same. However the order of the requests in this state machine passes the write buffer requests first before allowing others proceed.

The NewXfer signal is valid during the first data transfer signal for a request. This is used to detect the first data transfer signal for a request. This state machine switches from one state to the other during the sampled assertion of the first data transfer signal. If there is another request asserted at that time, then it switches to the corresponding source state. If there are no requests found asserted, it goes to idle state. It is ensured that there cannot be more than two sources initiating a request at the same time. Moreover if it is a mode read access the engine will assert the data transfer signal only for a clock. Therefore it goes to idle state on sampling the data transfer signal for a mode read request.

5.4.2 Data transfer state machine

The data transfer state machine is introduced to relieve the state machines or the buffers from sampling the common data transfer signal (XferOut3) and to route the same to the source that initiated the request. This state machine finds the request source before the first data transfer signal for every request and it switches to the corresponding state. It stays in that state till the last data transfer for that request. The following table indicates the source of data transfer and the corresponding state.

Source of request	State defined
Write control state machine	ST_XFER_WRITE
Read control state machine	ST_XFER_READ
Write buffer 0	ST_XFER_WRBUF0
Write buffer 1	ST_XFER_WRBUF1
Read buffer	ST_XFER_RDBUF
No Request	ST_XFER_IDLE

It may be noted that the last data transfer of a read request and the first data transfer of a successive write request can take place on the same clock edge. When the external memory buswidth is 16 bits, the first half word of data is sampled even before the assertion of the data transfer signal. The data transfer state machine could not take care of this condition as it can switch only at the end of the data transfer for a particular request. Therefore a similar control state machine, called write data transfer state machine (CurrentWrXfer), is used to route the write data to the memory. It switches only between the sources of write data transfer. I.e., write buffers or the write control state machine at the top level of the AHB interface.

5.4.3 Write control state machine

The write control state machine handles the memory write requests. It handles the following write transactions.

- All the write transactions from the AHB bus when the design is not configured with write buffers.
- All the write transactions from the AHB bus when the AHB interface is configured with write buffers, but the write buffer enable bit is set to logic zero.
- All the long burst write accesses

In the first case, every new write access initiated on the AHB bus will be initiated to the memory as a new request. On sampling the write access indication from the AHB bus it asserts the write request to the arbiter. It goes to the Request (REQ1) state at this point. On sampling XferOut3 it goes to the Data (DATA) state. If the transaction is

terminated by either AHB master or memory control engine the state machine goes to Idle (IDLE) state and waits for the next write access to the AHB interface.

In the second case, the AHB interface is configured with write buffers. But the buffers are disabled by setting write buffer enable bit, in configuration register 1, to logic zero. In this case all AHB write transactions are handled by this state machine as if the write buffers are not available. But if the write buffers contain a valid data before being disabled, the data in the write buffers are written to the memory before any AHB write transaction is handled.

In the third case, all the INCR4 started with miss-aligned quad word address / WRAP8 / INCR8 / WRAP16 / INCR16 accesses are handled by this state machine. They do not go through the write buffers even if they are enabled. Because the data transferred in a long burst request is more than a quad word. However it is possible that the range of address targeted in a long burst access includes the quad word address range in either of the write buffers.

If the write buffers contain a valid data in that long burst address range, then it will be written to the memory before initiating the long burst write access to the memory. In that case this state machine has to wait till the write request from the write buffer is over and then it initiates a write request. Moreover if the starting address is not a quad word aligned address, then the default OKAY is not returned. The state machine goes to a miss-aligned long burst request state (LREQ1). It reaches the data (DATA) state only if the data on the bus is the same as the next word to be transferred to the memory. Till that time the AHB interface returns WAIT response on the bus.

Implementation details :

On reset, it goes to idle (IDLE) state. It waits in this state till there is a write access initiated on the AHB bus targeting the AHB interface. From this state it can go to the request (REQ1) state or long burst request (LREQ1) state, if there is a write request to be initiated to the memory. The REQ1 state is reached if the write request is a quad word aligned long burst request or a non long burst access when the write buffers are disabled. The LREQ1 state is reached if there is a long burst write access with a miss-aligned quad word address. The difference is that if it reaches REQ1 state, then the first write data is returned with a default OKAY response. If it is a miss-aligned quad word address for a long burst write access, then the data is taken only when it has to be written to the memory. The AHB interface forces the starting address (AddrIn3) of the long burst access to be quad word aligned to the memory engine.

The request (REQ1) state of the state machine, indicates a quad-word aligned long burst access or a buffer disabled write access. It waits in this state till the data transfer (XferOut3) signal is sampled asserted for the write request. On sampling the data transfer (XferOut3) signal asserted, it goes to data (DATA2) state or "No more data transfer" (NOXFER) state or second request (REQ2) state based on the next access from the AHB master. If the next access on the AHB bus is addressed to the next word, then it goes to data transfer (DATA) state. If there is an AHB write access, but not to the next word, or if the transaction was terminated by the master, then it goes to the second request (REQ2) state and keeps the write request active. If there are no AHB write access to the memory, then it reaches "No more data transfer" (NOXFER) state. The state machine changes from this state only when the data transfer signal (XferOut3) is sampled asserted.

The long burst (LREQ1) state indicates that a miss-aligned quad word access is initiated on the AHB bus. However the AHB interface forces the address input to the memory engine to be a quad-word aligned address. Therefore the data on the bus is taken only when it is the next word of data to be transferred to the memory. On sampling the data transfer signal it can go to the long burst data (LDATA) or data transfer (DATA) state. If the data on the bus is not the next word of data to be written to the memory then it goes to LDATA state. If the data on the bus is same as the next word of data to be written to the memory then it goes to DATA state and the data on the AHB bus is taken into the data latch (BData).

The long burst data (LDATA) state indicates that the data on the AHB bus is not the same as the next word of data to be written to the memory for a long burst access. The data on the bus is taken only when it is the next word of data to be written to the memory. Once this condition is met the state machine goes to the (DATA) state.

The DATA state is the data transfer state for any write request. The data from the AHB bus is latched in this state and passed on to the memory. The state machine stays in this state till the data transfer goes on without any interruption. I.e., the AHB master provides a continuous burst of data. If the AHB master initiates a new AHB access before the end of data transfer (EndOfXfer3) signal is sampled asserted, then it should initiate a new write request to the engine. If the data transfer signal (XferOut3) is sampled asserted on the same clock edge, then it

goes to the second request (REQ2) state. Otherwise it goes to next request (NXTREQ) state. It is because the REQ2 state is used to mask off the DQM3 input to the memory engine. But the DQM3 can be masked only after the current data transfer is over. Until that time, it stays in NXTREQ state, in which state the request is asserted but the DQMs are not masked. If the AHB master initiates a new access when the EndOfXfer3 is sampled asserted it goes to the request state (NXTREQ) to continue with the next request. If there are no write access from the AHB master then it goes to "No data transfer" (NOXFER) state and stays there.

The "No data transfer" (NOXFER) state is reached because the AHB master has not initiated any write access on the AHB bus. The state machine waits in this state till a new AHB write access is initiated or the delayed end of data transfer signal (EndOfWrXferD) is sampled asserted. If a new write access is initiated on the AHB bus before the current write request is over, then it goes to the second request (REQ2) state. Also if there is a new write access on the AHB bus, when the delayed end of data transfer (EndOfWrXferD) signal is sampled asserted, then it directly goes to the request state (REQ1 or LREQ1) depending on whether the new access is a long burst access started with the miss-aligned quad word or not. If there are no write access on the AHB bus till the end of data transfer for the current memory write request, then it goes to the idle (IDLE) state.

The second request (REQ2) is reached when there needs to be a new write request to be initiated to the memory while there is already a write request going on. The state machine waits in this state till the delayed end of data transfer (EndOfXfer3D) is sampled asserted. On sampling the delayed end of data transfer signal it goes to the request state (REQ1 or LREQ1) and continue with the new write request.

The next request (NXTREQ) is reached because there is a data transfer going on the memory bus. On sampling the data transfer signal (XferOut3) for the current data transfer, if the data transfer phase of the current request is not yet over, it goes to the second request (REQ2) state. If the data transfer phase of the current request is over, i.e., if the delayed end of data transfer (EndOfWrXferD) signal is sampled asserted, then it goes to the request (REQ1 or LREQ1) state.

The following signals are generated from this state machine:

NoBufWrite3: This is the write request from this state machine. It is generated when the state machine is in idle (IDLE) state, no data transfer (NOXFER) state, request (REQ1) state, long burst request (LREQ1) state, next request (NXTREQ) state or second request (REQ2) state. When the state machine is in IDLE or NOXFER state, the write request is generated combinatorially with the bus access. During other states the state machine itself will keep the Write3 signal asserted.

DataWrEn1 : This signal is asserted to return a default OKAY response on the AHB bus for the first data of a write request.

DataWrEn : This signal is asserted to return a OKAY response on the AHB bus for subsequent data transfers.

The following are the control signals generated in the AHB interface for write accesses:

RegWrDone: This indicates the OKAY response for a register write access. Any register write access is returned with a zero wait state OKAY response.

AhbWrDone: This indicates that OKAY response is returned on the AHB bus for a memory write transaction. This is a combination of OKAY response signals (DataWrEn1 and DataWrEn) from the write state machine and write buffers. The signals from the write buffers (WrBuf0Done and WrBuf1Done) are effective only when they are enabled. It is also used as a write enable signal to latch the data on the AHB bus. This data will be written to either the write buffers or memory.

IntWrite3: This is the combined write request signal from the write state machine and write buffers. The requests from the write buffers (WrBuf0Write3 and WrBuf1Write3) are effective only when they are enabled. It is used by the read buffer to delay the read request when the write request is already on.

LBIncr4: Long burst INCR4 access. When the INCR4 access is not started with a quad word aligned address, two quad words need to be accessed in the memory. For the subsequent sequential accesses, this signal stays asserted indicating that it is a long burst access. It is used in the write state machine and in the write buffers.

QWCount: It indicates the number of quad words to be accessed in the memory for any read / write request. This gets set to the total number of quad words required for an AHB access on sampling a non-sequential access. Then the value gets decremented on completion of every quad word transfer. It is used to generate a corresponding ReqNumCmd3 signal.

5.4.4 Read control state machine

The read controls state machine handles all memory read transactions when

- The read access is a mode access
- The AHB interface is not configured with read buffer.
- The AHB interface is configured with read buffer, but the read buffer enable bit is set to logic zero.
- The read is a long burst access

The first case is not a memory data read access. Hence the data transferred during that access should not be stored in the read buffer as a valid data. Hence it is always passed through this state machine irrespective of the read buffer's availability.

In the next two cases they are memory data read access with the read buffer disabled. Therefore all the read accesses are handled by this state machine. When there is an AHB read access, it goes to the request state to initiate a read request to the memory. On sampling the read data transfer signal, for the read request, it goes to data state and transfers a quad word of data to the AHB interface. If the AHB master has terminated the access then it goes back to idle state. This should also take care of the condition that when the AHB interface is configured with write buffers, the AHB read access may be a read access hit to the write buffers. In that case the write buffer's data is first written to the memory before initiating the read request.

In the last case, all the long burst accesses are handled by this state machine as they are not passed through the read buffer. However if the write buffers are enabled and the long burst read access hits the write buffer, then the write buffer has to write the data before initiating the read request. Hence the state machine will wait for the write request to be initiated on the bus and then it will initiate a read request.

Implementation Details:

On reset, it goes to IDLE state. It waits in this state till there is a read access initiated on the AHB bus targeting the AHB interface. From this state it can only go to the request (REQ) state if there is a read request to be initiated to the memory. It will go to the request state if it is a mode access or when the read buffer is not active or it is a long burst read access and the write buffers do not contain the data in the same address range. It is possible that when the write buffers are enabled, the read access is to the same quad word of data that is in the write buffer. Then the write buffer initiates a write request to the memory. A read request is initiated only after the data transfer of the write request starts.

The request (REQ) state of the state machine keeps the read request asserted. It waits in this state till the data transfer (XferOut3) signal is sampled asserted for the read request. On sampling the data transfer (XferOut3) signal asserted, it goes to data (DATA) state or long burst data (LDATA) state. It goes to the long burst data state if the read access is a long burst read with miss-aligned quad word address.

The DATA state is the data transfer state for any read request. The data from the memory is directly passed to the AHB bus in this state. The state machine stays in this state till the request is over on the memory bus or the AHB master terminates the access. On finding that either the AHB master has terminated the access before it is over on the memory side it goes to the "No data transfer" (NOXFER) state. If the request is over on the memory bus, and if there is a read access on the AHB bus then it may go to the request state again. Otherwise it goes to idle (IDLE) state.

The long burst data (LDATA) is reached due to a long burst access with a miss-aligned quad word access. From this state it goes to data (DATA) state when the data from the memory is the same as that requested by the AHB master.

The “No data transfer” (NOXFER) state is reached if the read access is terminated by the AHB master. It stays in this state till the data transfer is completed by the memory engine. From this state it may go to the request state again depending upon a new read access initiated by the AHB master. Otherwise it goes to the idle (IDLE) state at the end of the data transfer.

The following signals are generated from this state machine:

NoBufRead3: This is the read request from this state machine. It is generated when the state machine is in idle (IDLE) or request (REQ) state. When the state machine is in idle state, the read request is generated combinatorially with the bus access. However the request state of the state machine directly asserts the read request.

RdDone: This is the OKAY response on the AHB interface. It is returned one clock after the data transfer (XferOut3) signal is sampled asserted on the memory bus. The OKAY response is returned only when the XferOut3 is sampled asserted for further data transfers. For successive data transfers the AHB bus address is compared with the word address for which the data transfer is going on the memory bus for every clock. The address for which the data transfer is going on is calculated by using the starting address of the request and the counter RdWordNum.

The following are the control signals generated in the AHB interface for read accesses:

RegRdDone: This indicates the OKAY response for a register read access. Any register read access is returned with a zero wait state OKAY response.

AhbRdDone: This indicates that OKAY response is returned on the AHB bus for a memory read transaction. This is a combination of OKAY response signals from the read state machine and read buffer. The signal from the read buffer (RdBuf0Done) is effective only when it is enabled.

The **WrBufSel** is the signal that selects the write buffer to take the AHB write data for any memory write access. This signal is logic zero by default, indicating that the write buffer 0 should take the first write hit access. After that it switches to write buffer 1 on finding that there is a write miss access to write buffer 0 on the AHB bus and it is a write hit to write buffer 0. Similarly it switches back to write buffer 0 on finding that there is a write miss access to write buffer 1 and it is a write hit access to write buffer 0. This signal is passed on to the write buffers. The write data can be latched only to the buffer that is pointed to by the **WrBufSel** signal.

The **AddrIn3**, **DataIn3** and **DQM3** are the address, write data and data mask signals, to be driven to the memory engine for a write access.

The **AddrIn3** can be from any of the following sources.

1. From AHB bus directly when:
 - There is a memory read access to be initiated to the memory.
 - There is a memory write access when the write buffers are disabled.
2. From the address latch (**AddrD1**) when there is a read3 request initiated. As the AHB is a pipelined bus, the address and control information is not available when the response is to be returned for that transactions. Hence the address is latched and driven to the memory engine.
3. From the second address latch (**AddrD2**) when there is a write3 request initiated.
4. From the write buffers when they are active.

The address is then mapped as described in Appendix A.

Similarly the **DataIn3** and **DQM3** are selected either from the write buffers or from the data latch. When the write buffers are active the respective write buffer drives the data and mask for a write request. It may be noted that the DQMs are ignored by the memory control engine for a read request. It always provides a word data of all the locations. The routing of **DataIn3** and **DQM3** are controlled by the output of the write data transfer state machine (**CurrentWrXfer**) that indicates the source from which these signals have to be driven.

Table 5.4-1 indicates the generation of data mask signals for a write data transfer. It is generated from the latched address and bus size information. Note that the data mask are active low signals.

Transfer type (latched HSIZE)	Address (Latched HADDR[1:0])	DQM[3]	DQM[2]	DQM[1]	DQM[0]
00 (Byte)	00	1	1	1	0
00 (Byte)	01	1	1	0	1
00 (Byte)	10	1	0	1	1
00 (Byte)	11	0	1	1	1
01 (Half word)	0x	1	1	0	0
01 (Half word)	1x	0	0	1	1
10 (Word)	Xx	0	0	0	0
11 (DWord)	Xx	1	1	1	1

Table 5.4-1 DQM generation

For any read access the read data provided by the sources, such as the register block for the register read access and the memory control engine or the read buffer for memory read access, is always of word-wide. This data is passed through the byte-steering logic to drive the AHB data bus. **Table 5.4-2** indicates the data driven on the AHB data bus for various transfer types. The ReadData[31:0] is the word-wide data.

Transfer type (latched HSIZE)	Address (Latched HADDR[1:0])	HRDATA[31:24]	HRDATA[23:16]	HRDATA[15:8]	HRDATA[7:0]
00 (Byte)	00	ReadData[7:0]	ReadData[7:0]	ReadData[7:0]	ReadData[7:0]
00 (Byte)	01	ReadData[15:8]	ReadData[15:8]	ReadData[15:8]	ReadData[15:8]
00 (Byte)	10	ReadData[23:16]	ReadData[23:16]	ReadData[23:16]	ReadData[23:16]
00 (Byte)	11	ReadData[31:24]	ReadData[31:24]	ReadData[31:24]	ReadData[31:24]
01 (Half word)	0x	ReadData[15:8]	ReadData[7:0]	ReadData[15:8]	ReadData[7:0]
01 (Half word)	1x	ReadData[31:24]	ReadData[23:16]	ReadData[31:24]	ReadData[23:16]
10 (Word)	Xx	ReadData[31:24]	ReadData[23:16]	ReadData[15:8]	ReadData[7:0]
11 (DWord)	Xx	InValid	InValid	InValid	InValid

Table 5.4-2 Byte-steering logic

The AHB interface contains a write time-out counter. This is a 16 bit down counter. It is inferred only if the AHB interface is configured with two write buffers. Hence the parameter `WR_BUF_COUNT` is used to infer the write time-out counter. It loads the write time-out value from the register (WTORegister) when the load enable is high, and counts down only when the count enable is high. The load enable and count enable are generated in the AHB interface. It is expected that the load enable is generated whenever the write buffer is updated with a new data and the count enable is active whenever the write buffer contains a valid data.

The read and write buffers are instantiated in the AHB interface with the parameters being passed to them. The design details of the read and write buffers are as follows:

5.5 SDRAM Read Buffer (SdramAhbRdBuf0)

This module is a parameterized one. It will be there in the design only if the AHB interface is configured with one read buffer. I.e., only if the constant `NO_OF_READ_BUFFERS` in the `SdramDefs.v` file is set to 1. This constant is assigned to the parameter `RD_BUF_COUNT` in the design. This parameter is used throughout the design to infer the logic only when it is enabled.

Note : The read buffer does not handle the mode and long burst read accesses.

This read buffer is used to store the latest quad word of data accessed from the memory by the AHB interface. Whenever there is an AHB read access, the read buffer compares the address of the transaction with the address stored in it. For a transaction to be a read hit access to the read buffer, all the following conditions have to be satisfied by the AHB read access.

- The address of the incoming AHB read access should be in the same quad word address range available in the read buffer
- The read buffer should contain a valid data (The data valid bit in the read buffer should be set to 1).

If all the above conditions are satisfied, then the read data can be returned from the read buffer directly to the AHB master. There are no WAIT responses returned to the AHB master. The data is provided in the next rising edge of HCLK, without any delay. The address on the AHB bus is latched on a rising edge of HCLK. This is used to return the word data from the read buffer to the AHB interface. This read data is passed through the byte-steering logic in the AHB interface before being driven on the AHB bus.

For a transaction to be a read miss access to the read buffer at least one of the following conditions have to be satisfied by the AHB read access.

- The address of the incoming AHB read access is not in the same quad word address range available in the read buffer
- The read buffer does not contain a valid data (The data valid bit in the read buffer is set to 0).

If one of the above conditions are satisfied, then the AHB read access is a read miss access and hence the read buffer control state machine should initiate a read access to the memory to fetch the data.

Implementation Details:

On reset, it goes to the idle (IDLE) state. It stays in the idle state till there is an AHB read access that targets the AHB interface. The address information of the AHB read access is then compared with the address stored in the read buffer along with the data valid bit. The address comparison takes place on a quad word boundary. If the transaction is found to be a read hit access, then the state machine does not come into the picture at all. The data will be immediately returned from the read data buffer. If the AHB access is a read miss access, then the state machine goes to the request (REQ) state only if there is no write transaction going on already on the AHB bus. This write transaction to the memory can happen if the AHB interface is configured with the write buffers. A write miss access or a write time-out could have started the write transaction to the memory while the AHB master has initiated a read access on the bus. Moreover, the read access could have hit the write buffer, hence the write buffer's data

should be first written to the memory before reading it. This needs to be taken care of, as the read and write request to the memory control engine cannot be initiated simultaneously from the same port.

The request (REQ) state of the state machine keeps the read request active. It stays in this state till the data transfer (XferOut3) signal is asserted by the control engine. The assertion of data transfer signal indicates that the data transfer will take place for the next four rising edges of the clock, one word of data per clock edge. Hence the state machine goes to the DATA state on sampling XferOut3.

Note: The state machine going from the request to the idle state is an exception. It happens only if a register write access to set the mode bit is followed by an immediate mode read access in the next rising edge of HCLK, while providing the data for the register write access. In this case the mode is sampled asserted only in the second rising edge of HCLK. Hence the request from this state machine should be reset immediately as it is handled by the read control state machine at the top level of the AHB interface.

The DATA state of the state machine indicates that the read access is going on the AHB as well as the memory bus. The memory bus is logically connected to the AHB bus in this state (i.e., the data from memory is directly driven on the AHB bus). The state machine stays in this state as long as both the AHB and memory accesses are going on. It stays in this state till the end of the data transfer, i.e., the assertion of EndOfXfer3D signal. If there is a new AHB read access from the AHB master to a different quad word, then it goes to the "next request" (NXTREQ) state to keep the read request active. If the new access is initiated on the AHB bus on the same clock the delayed end of data transfer signal (EndOfXfer3D) is sampled asserted, then it goes to request state (REQ) again. If there are no other AHB read request at the end of the data transfer it goes to idle (IDLE) state.

The next request (NXTREQ) state is reached if there is already a memory read request going and the AHB master has terminated that read access before reading all the four data and initiated a new read request. On sampling the delayed end of data transfer signal it goes to request (REQ) state. If the data transfer is started immediately, then it goes to data state.

The above state machine is used to generate the following signals.

RdBuf0Read3: This signal is a read request from the read buffer. It is generated when the state machine is in idle, request or next request state. When it is in idle state, it is generated combinatorially with the read access on the AHB bus.

RdBuf0Done : This signal is used by the AHB interface to return the OKAY response for any AHB read access. The OKAY response will be returned when:

- The AHB access is a read hit to the read buffer
- The AHB read access is a read miss access and the data transfer signal is asserted for the same read request to the memory.

There is a word by word comparison to ensure that the correct word data is returned from the memory to the AHB interface. For a read miss access, the memory read data is logically connected to the AHB read data bus. For a read hit access, it is returned from the buffer.

The address on the memory bus, is latched when the data transfer signal is sampled asserted for a read request. This is used to evaluate the AHB read access to find the hit or miss conditions.

The DATA and NXTREQ states of the state machine are used to latch the word data from the memory to the read buffer. So that if there is an AHB access to the same quad word address, then the corresponding data can be returned from the read buffer itself.

5.5.1 Read data valid state machine

The Read data valid state machine is used to set the data valid bit in the read buffer. The read data valid bit is set, whenever the read buffer is updated with the new quad word data for a read miss access. The data valid bit is reset when there is a write transaction on the AHB bus to the same quad word address in the read buffer. In this case, the data should be fetched from the memory after updating it with the new write data. Hence the data valid bit will be

reset. Moreover, if the read buffer is disabled by read buffer enable bit, then also the data valid bit is reset. The following state diagram explains the read data valid state machine.

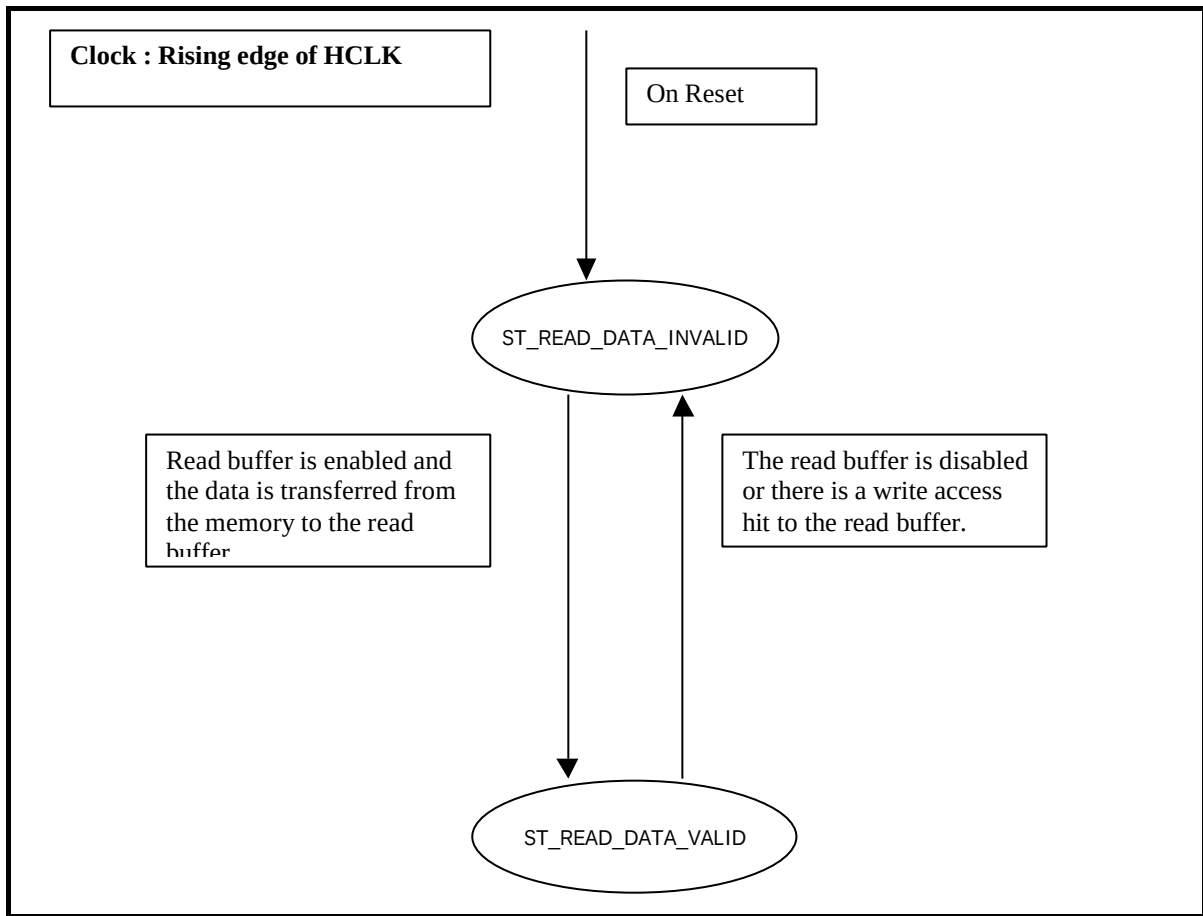


Figure 5.5-1 State diagram, of the read data valid state machine, in the read buffer(SdramAhbRdBuf0)

The ST_READ_DATA_VALID state of this state machine sets the read data valid bit.

5.6 SDRAM Write Buffer (SdramAhbWrBuf0 / SdramAhbWrBuf1)

This is a parameterized module. This will contain any logic inside only if the AHB interface is configured with two write buffers. The constant NO_OF_WRITE_BUFFERS is defined in SdramDefs.v file. It should be set to 2 to have the two write buffers in place. This constant is assigned to the parameter WR_BUF_COUNT in the design. This parameter is used throughout the design to ensure that it is inferred only when this parameter is set to 2.

The write buffers are used to merge the AHB writes and hence improve the performance of the SDRAM controller. Whenever there is an AHB write access, it is evaluated to find the hit or miss conditions. The quad word address on the AHB bus is compared against the address stored in the write buffer. For a transaction to be a write hit access it should satisfy at least one of the following conditions.

- The write buffer has to be empty (i.e., the data valid bit should be reset)
- The quad word address of the incoming AHB write access should be the same as that of the write buffer.

If the transaction is a write hit access, then it will be returned a OKAY response immediately by the write buffer and the data is taken into the write buffer.

For a transaction to be a write miss access it should satisfy all the following conditions.

- The write buffer should contain valid data (i.e., the data valid bit should be set)
- The quad word address of the incoming AHB write access should not be the same as that of the write buffer.

If there is a write miss access to the write buffer, then the state machine initiates a write request to the memory control engine and writes the data in the write buffer to the memory.

If there is a read hit access to the write buffer, then also the state machine should initiate a write request to the memory and write the data to the memory. For a transaction to be a read hit access to the write buffer, the following conditions are to be satisfied.

- The write buffer should contain valid data (i.e., the data valid bit should be set)
- The quad word address of the AHB read access should be the same as that of the write buffer.

If there is a long burst read / write access that hits the write buffer, then also the state machine should initiate a write request to the memory and write the data to the memory. As the long burst read / write access is handled by the respective state machines at the top level of the AHB interface, the stale data in the write buffer should not be written to the memory later on. To avoid this the write buffers are cleared by writing the data to the memory before letting the long burst access to proceed. For a transaction to be a long burst hit to the write buffer, the following conditions are to be satisfied.

- The write buffer should contain valid data (i.e., the data valid bit should be set)
- For an INCR4 started with miss-aligned quad word address or a WRAP8 access it should be in the double quad-word address range or for a WRAP16 access it should be in the quad-quad-word range (4 quad words) and for an INCR8 / INCR16 access it should be in the kilo-word range.

Note: The kilo-word range is taken as the boundary as an AHB master is not expected to cross that boundary by a burst of transactions. It has to initiate a new access if it crosses the boundary.

Implementation Details:

On reset, the state machine comes to the IDLE state. The write request is to be initiated from this state machine under the following conditions.

1. There is a write miss access from the AHB master.
2. There is a read access from the AHB master that has hit the write buffer.
3. The write buffers are disabled by write buffer enable bit.
4. There is a long burst read / write hit access from the AHB master.
5. The write time-out has occurred.

In the first four cases the state machine will go the request (REQ) state and keeps the write request active.

In the last case, the write request is initiated internally due to the write time-out. In this case, if there is a read access from the AHB interface, then it will bypass the write time-out condition. The state machine will allow the other request to proceed. This will go to the NXTREQ (next request) state and waits for the start of data transfer phase for the previous request.

The NXTREQ (next request) state indicates that the write buffer is about to initiate a write request, but there is a read request already going on. It will wait in this state till the read request is active. On finding that the read request has started the data transfer phase, it goes to the request (REQ) state.

The REQ (request) state keeps the write request active. It waits in this state till the data transfer signal is asserted for this transaction. On sampling the data transfer signal for the current request, the state machine goes to the data transfer phase.

The DATA state indicates that the data transfer phase is going on for the write request from the write buffer. The state machine stays in this state till all the four word data are written to the memory with their corresponding data mask signals asserted. From the DATA state it can go to the idle (IDLE) state on sampling the delayed end of data transfer signal.

The following signals are generated from the state machine output:

Note: Change “WrBuf0” to WrBuf1” in the following signals to get the corresponding signals in write buffer 1.

WrBuf0Write3: This is the write request from the write buffer 0. It is generated only when the state machine is in IDLE or REQ state.

WrBuf0Commit: It is generated only when the state machine is in the request state.

WrBuf0Done: This signal is used by the AHB interface to return OKAY response on the AHB bus. It is a combination of OKAY responses for a write hit and miss accesses. The write hit access will be given the response immediately. The only time this write hit access is not taken is when the write buffer is writing data to the memory. At this point the incoming access will be taken as a new access.

WTOLoadEn0: This signal is used as data latch enable as well as the load enable signal to write time-out counter in the top level of the AHB interface. The write time-out counter has a higher priority for the load enable signal which means that whenever there is at least a byte of data updated in write buffer 0, this signal will make the write time-out counter to restart counting again. I.e., the write time-out starts from the last write to the write buffers.

WrDqmEn: This is used as a data mask write enable signal. It will be active whenever the data is being written to the write buffer as well as the data is written to the memory from the write buffer. In the first case, this will update the data mask enable for every word of data. In the second case, it resets all the data mask enables to logic one when the data is written to the memory.

WrHitAddrEn: This signal is used as an address write enable. The quad word address is updated whenever there is a write hit access to write buffer 0.

There is a decode logic to drive the correct word of data during data transfer phase. The data selection is based on the counter WordNum. It loads the first word of data to be transferred for every request and increments for every data transfer.

There is a write data valid bit set by a write data valid state machine (VldSmState). It sets the valid bit with the first write hit access to the write buffer. The bit is reset when the first data transfer takes place for a memory write request from this buffer.

5.7 SDRAM Register Block (SdramAhbRegBlk)

The register block contains the registers and the related control logic. In this block, there are some configuration bits that are inferred based on the parameters. In the configuration register 1, the read buffer enable bit and write buffer enable bit are the ones controlled by the parameters.

The constant NO_OF_WRITE_BUFFERS defined in the SdramDefs.v file is assigned to the parameter WR_BUF_COUNT in the module. This parameter is used to control the inference of the write buffer enable bit (Bit 3) in configuration register 1.

The constant NO_OF_READ_BUFFERS defined in the SdramDefs.v file is assigned to the parameter RD_BUF_COUNT in the module. This parameter is used to control the inference of the read buffer enable bit (Bit 2) in configuration register 1.

The address, write enable and the select signals for every byte of data are registered on a clock edge. These are used to update the data from the AHB directly into the registers. **Table 5.7-1** shows the condition under which every byte of the register is updated. A '1' in “Byte x (where x = 0,1,2 and 3)” column indicates that byte is updated.

HSIZE (Transfer type)	HADDR[1:0]	Byte 3	Byte 2	Byte 1	Byte 0
00 (Byte)	00	0	0	0	1
00 (Byte)	01	0	0	1	0
00 (Byte)	10	0	1	0	0
00 (Byte)	11	1	0	0	0
01 (Half word)	0x	0	0	1	1
01 (Half word)	1x	1	1	0	0
10 (Word)	xx	1	1	1	1
11 (DWord)	xx	0	0	0	0

Table 5.7-1 Generation of Register Write Enables

All the registers update their data on a rising edge of HCLK directly from the AHB bus.

Register 0 contains the memory related parameters. It contains Device type, Bank size, 256 Mbit device, Clock shut down bit, Clock enable, External memory bus width, RAS and CAS delays and Auto pre-charge bit. For a detailed explanation of them, please refer to the technical reference manual.

Register 1 contains the parameters that affect the behavior of the control engine and AHB interface. It contains mode bit, initialize bit, read buffer enable bit, write buffer enable bit and engine status bit. The read and write buffer enable bits are controlled by the parameters RD_BUF_COUNT and WR_BUF_COUNT respectively. These two bits can be read and written only if the design is configured with the read and write buffers respectively. Otherwise they can only be read and they return zero always. The engine status bit is a read only bit. The AHB master should sample this before attempting to write to configuration register 0. As the configuration register 0 contains memory related parameters it should not be changed when the memory is being accessed. The AHB master should sample this engine status bit to be zero which indicates that the memory control engine is idle, before attempting to write to the configuration register 0.

Register 2 is the refresh count register. It contains the number of clock cycles at which the memory should be refreshed periodically. It loads the default value (0x0080) on reset. It can be modified by the AHB master at any time.

Register 3 is the write time-out register. This contains the number of clock cycles after which the write buffers should write the data to memory if they are not updated by the AHB master. As this register is required only when the design is configured with write buffers, the inference of this register has been parameterized. If the design is not configured with the write buffers, then this register is a read only register and returns zero when read.

Registers 2 and 3 are 16 bit registers. The least significant 16 bits are only valid. Only these bits can be read and written. The most significant 16 bits are reserved and they return zero when read.

The register block contains a refresh counter. It is a free running, 16 bit down counter. It loads the default value (0x0080) on reset and starts counting down on every rising edge of HCLK. When it reaches zero, it loads the refresh value from the refresh register (Register 2) and at the same time initiates a refresh request to the refresh control state machine. It is the refresh control state machine that finally passes on this refresh request to the memory control engine. It also loads the refresh value and raises a refresh request simultaneously when the refresh register is updated with a new value.

5.7.1 Refresh control state machine

The refresh control state machine issues the refresh request to the memory control engine. It takes its input from the free running refresh counter in the register block. The refresh request is issued to the memory only after the memory is initialized. There can be a refresh request from the refresh counter periodically. The period is same as the refresh count programmed in the refresh register. This refresh request from the refresh counter is initiated to the memory control engine. However if there is another refresh request from the register block before getting the refresh completion signal from the control engine, the refresh request is kept asserted continuously. If there are more than one refresh requests from the register block, there can be still only one request to the memory.

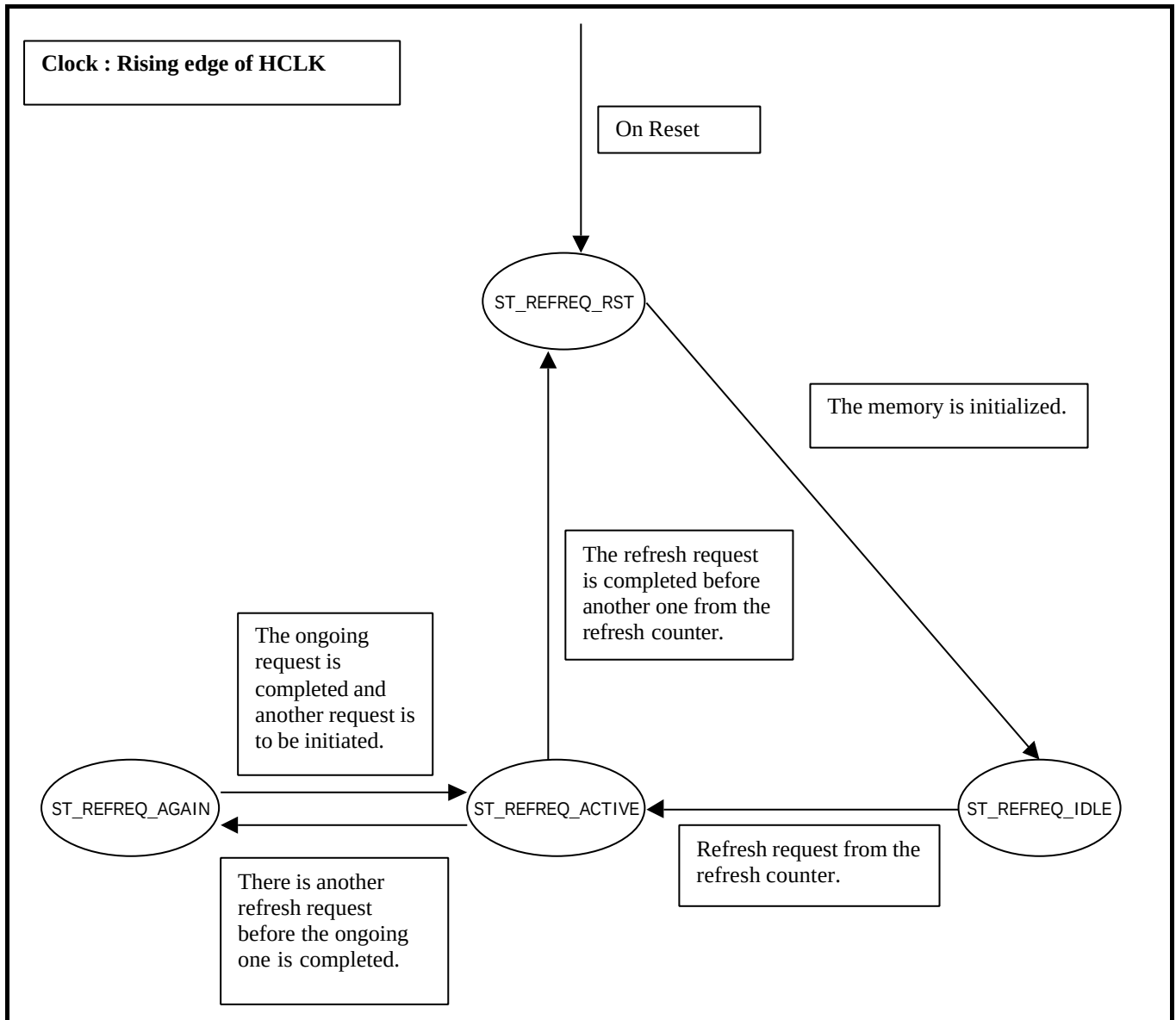


Figure 5.7-1 State diagram of the refresh state machine, in the AHB interface (SdramAhbif)

This state machine can be reset only by nPOR signal. On Power-on-reset it goes to the reset (RST) state. From this state it goes to idle (IDLE) state after the memory is initialized. The state machine is insensitive to any refresh request from the refresh counter in this state. This state is introduced to prevent it from issuing any refresh request to the memory control engine till the device is initialized. On sampling that the initialize bit is set and mode bit is reset it goes to the idle state.

The idle state indicates that there are no request from the refresh counter as well as there is no refresh going on the memory. It stays in this state till the refresh request from the refresh counter is asserted. On sampling the assertion of refresh request from the refresh counter, it goes to the active (ACTIVE) state.

The active state initiates a refresh request to the memory control engine. The active state continues until the refresh completion signal (Done) is asserted by the control engine. If there is another refresh request the refresh state machine goes to next request state. If there are no refresh request from the counter till the refresh completion signal is sampled asserted, it goes back to the idle state on sampling the Done signal from the memory control engine.

The next refresh request state indicates that there is another request to be initiated to the control engine after sampling the refresh completion signal from the same. It may be noted that this state machine can store at the most only one pending request before the completion of the previous refresh request. If there are multiple refresh requests from the refresh counter they are still considered as a single refresh request. From this state it goes to active state on sampling the refresh completion signal (Done) from the memory control engine.

The Refresh signal is asserted when this state machine is in active (ACTIVE) or next refresh request (AGAIN) state.

The register block also contains a decode logic based on the address input to return a register data for register read access. It returns the register data of size equal to the AHB bus width. This data will be passed through the byte-steering logic in the AHB interface to be finally returned on the AHB bus.

5.8 Power-On Initialization sequence

The initialization sequence follows the sequence described in the INTEL PC SDRAM specification. The initialization of the SDRAMs requires specific interaction with software to provide the long delays required and to program the Configuration register in the correct sequence.

The SDRAM initialization as described in the INTEL PC SDRAM specification is as follows:

Power Up Sequence:

The SDRAM is initialized by the following sequence of operations:

- Clock will be applied at power up along with power (clock frequency will be unknown).
- The clock will be stabilized within 100usec after power stabilizes.
- All the control inputs, RAS#, CAS#, WE#, CS# will be held in an undefined state (either valid high or low) during reset. After reset is complete CS# will be held inactive before the first access to SDRAM is attempted. All other address and control signals will be driven to a valid state.
- The levels on all the address inputs should be ignored. (All the address inputs can be indeterminate.)

Initialization Sequence:

Following the initialization sequence, the device must be ready for full functionality. SDRAM devices are initialized by the following sequence:

- At least one NOP cycle will be issued after the 1msec device deselect.
- A minimum pause of 200usec will be provided after the NOP.
- A pre-charge all (PALL) will be issued to the SDRAM.
- 8 Auto refresh (CBR) refresh cycles will be provided.
- A mode register set (MRS) cycle will be issued to program the SDRAM parameters (e.g. Burst length, CAS# latency etc.).
- After MRS the device should be ready for full functionality within 3 clocks after T_{mrd} is met.

The above power-up and initialization sequence is implemented in the SDRAM Control Engine under software control.

- Reset causes all devices to be deselected, and disables all refresh/pre-charge commands to the SDRAMs. The bits which are used for the Initialization sequence, power up with the default values of I (Initialize) = 0 and M (Mode) = 0.
- After a 100 us delay ensured by software, the I bit and the M bit in Configuration register 1 should be set to 1. This causes a NOP command to be issued to the external devices.
- After the NOP is issued, a minimum pause of 200 us needs to be provided before the next command can be issued to the devices. With I = 1 and M = 1, software should remain in a delay loop for 200 us.
- A PALL command needs to be issued to the SDRAM devices. Software should set I = 1 and M = 0 in the Configuration register 1. This causes a PALL to be issued to the SDRAMs.
- 8 AUTO REFRESH cycles should be issued to the SDRAM devices. Software should write a small value of 10 into the refresh timer register. This provides a refresh cycle every 10 clock cycles. Software should then wait for a time period equivalent to 80 clock cycles.
- Program the operational refresh value into the refresh register.
- A Mode Register Set access needs to be issued to the external devices. Software has to set I = 0 and M = 1 during Mode accesses. For a Mode Register Set request, a read needs to be performed from a carefully calculated address location such that the bit pattern on AHB address A[26:12] map to the SDRAM device address A[14:0]. Refer to the Appendix A for the exact mapping of bits on the AHB address bus to the bits on the SDRAM address bus.
- Program I = 0 and M = 0 to commence normal operation.

5.9 SDRAM Control Engine (SdramEngine)

The SDRAM Control Engine instantiates the Arbiter, the DRAM Bank Controller and the Command Sequencer.

From each of the four ports, the Command (Readx/Writex), Address (AddrInx), Write Data (DataInx), Write Data Mask (DQMInx) and the Burst size indicator(ReqNumCmdx) signals feed the Arbiter. Within the Arbiter, for each port, a separate FSM tracks the progress of the request raised by that port. The Arbiter serialises requests from the ports and presents them to the rest of the control engine. The signals from the Arbiter that constitute this request are RasAddr[28:12] (RAS Address), Read, Write, ReqSource[3:0](Port address) and ReqNumCmd[2:0].

The DRAM Bank Controller handles requests from the Arbiter. For each request from the Arbiter, the DRAM Bank Controller asserts the SplitEn signal if that request cannot be accepted. With every Split, the Arbiter re-assigns Port priorities such that a Port whose request is Split, takes the least priority allowing other pending requests from other ports to proceed. If a request has been assigned to a DRAM FSM, the Assigned signal to the Arbiter is asserted. When the data phase for a request is about to start, the SplitWait signal to the Arbiter from the DRAM Bank Controller is asserted. When the Bus master that requested this transfer, is ready for the data phase, the SplitProceed input signal for that Port is asserted. On detecting this signal, the data access command corresponding to this request is issued.

5.9.1 Command Flow

The command presented by the Arbiter is first seen by the DRAM Bank Controller. If a command cannot be assigned to a DRAM FSM, the SplitEn signal is asserted as mentioned before. If a command can be accepted, the DRAM Bank Controller allocates the request to a DRAM FSM and asserts the Assigned signal to the Arbiter. The DRAM FSMs generate the required sequence of commands (such as ACT, RD, WT, RDA, WTA etc) to be issued to the memory devices. The DRAM Bank Controller prioritizes these commands and issues the selected command to the Command Sequencer through the DRasAddr[26:12] (RAS Address), DevAddr[3:0] (Device address), Cmd[6:0] (Command), Source[3:0] (Port address), FSMNum[3:0] (DRAM FSM number) and NumCmd[2:0] (Burst size indicator) signals. When the Command Sequencer is about to issue a foreground command, it asserts the

AddrSource[3:0] output to the Arbiter to get the CAS Address (CasAddr[15:2]) for the foreground command. The Command Sequencer then issues the foreground command and asserts the DataXfer (data phase indicator) and DataXferSource[3:0] (Port address corresponding to the current data phase) signals to the Arbiter, which, in turn routes the DataXfer signal to the appropriate Port. After the Command Sequencer has issued the commands, it responds with a one-HCLK-wide pulse on the FSMxxxIssued[3:0] (FSMRDIssued, FSMWTIssued, FSMRDAIssued, FSMWTAIssued, FSMPREIssued and FSMACTIssued) signals. Each bit in these vectors corresponds to one DRAM FSM. In the case of Refresh and Mode commands, a pulse on the CmdIssued signal along with the command encoding on the BGCmdIssued[4:0] signal is used to indicate the issue of these commands.

5.10 Arbiter (SdramArbiter)

The Arbiter implements the following functions:

- Independent FSMs to register requests from each port.
- Priority arbitration logic.
- Re-arbitration FSM to re-prioritize accesses based on the busy status of the engines.

Figure 5.10-1 shows the block diagram of the Arbiter.

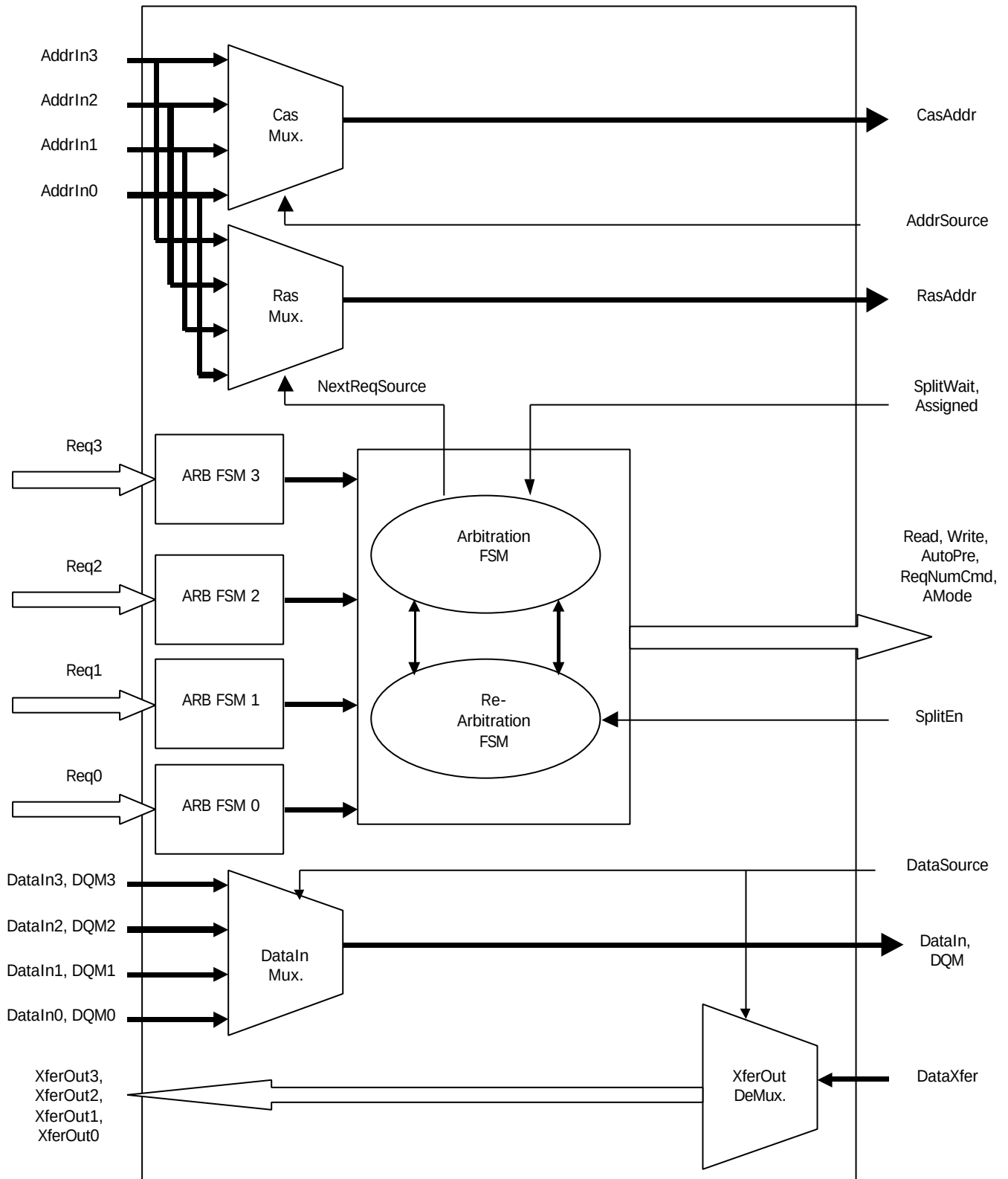


Figure 5.10-1 Block diagram of the Arbiter

5.10.1 Top Level Arbiter

The top level of the Arbiter instantiates 4 Arb FSM modules. The requests generated by each of the Arb FSMs are priority encoded to generate a single Request bus. The final request bus consists of the following control signals to the engine – Read, Write, AutoPre, RasAddr[28:12]. The single request is qualified by a 4-bit one-hot encoded ReqSource bus that indicates the port from which the access originated. The priority encoding is such that Port0 is the highest priority and Port3 is the least. In order to optimize data access times, the priority encoding scheme is modified based on a 2-bit state machine called the Re-arbitration FSM. **Figure 5.10-2** shows the state transitions of the Re-arbitration FSM.

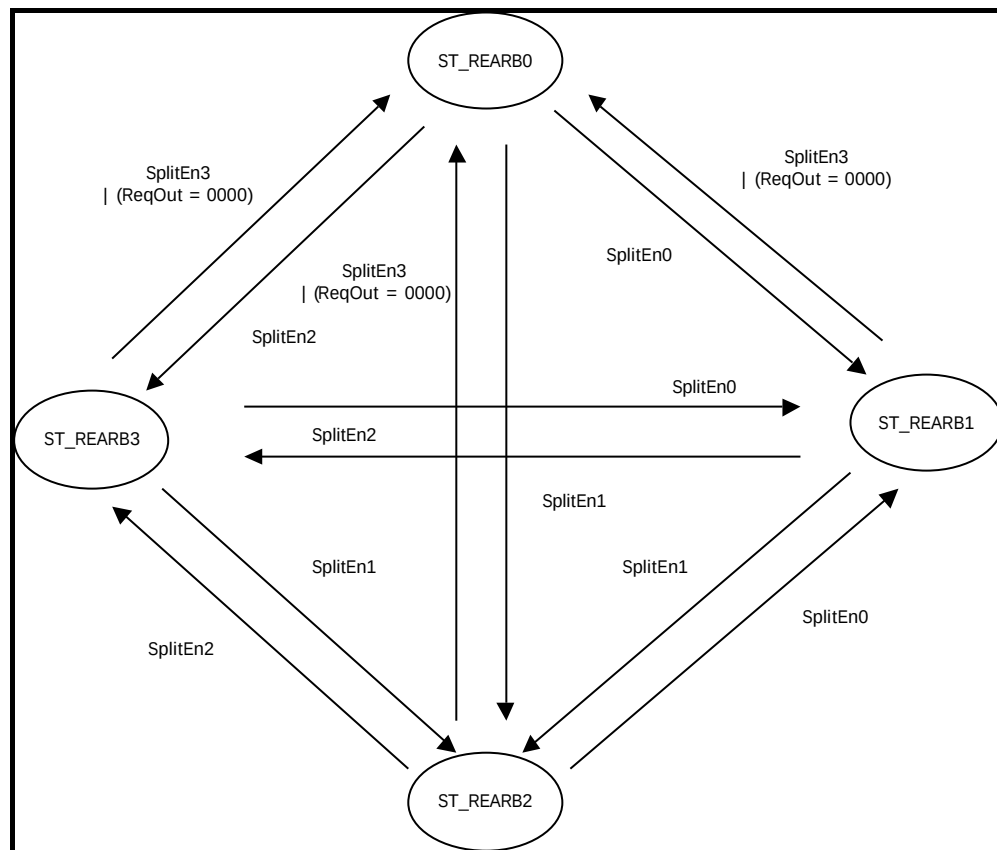


Figure 5.10-2 State diagram for the Re-arbitration FSM

When SplitEn is active, the Re-arbitration FSM advances such that the port access which caused the SplitEn becomes the lowest priority requestor and the port whose priority is one less than that causing the SplitEn becomes the highest priority requestor. This process continues until all the accesses have been serviced. The FSM always enters to the ST_REARB0 state when there are no outstanding requests. This restores the default priority of 0 -> 1 -> 2 -> 3. In the state diagram, SplitEnx indicates that ReqOutx has been Split by the Engine. ReqOut = 0000 indicates that there are no more requests left to be serviced.

Consider the following Re-arbitration sequence example.

At reset the FSM enters the ST_REARB0 state. In this state the priority order is 0 -> 1 -> 2 -> 3 (Highest to Lowest). Ports 1 and 2 are requesting simultaneously. Since Port 1 is at a higher priority, its request is propagated to the engine by the Arbiter. Since Port 1 request was to a Busy bank, the control engine splits the request by asserting SplitEn1. The Re-arbitration FSM transitions to the ST_REARB2 state. The priority order has been changed to 2 -> 3 -> 0 -> 1. Note that the port that was Split (Port 1) is now the lowest priority requestor while the port whose priority was one less (Port 2) is now the highest priority. The Arbiter routes the Port 2 request to the engine. If this were to a non-busy bank, the access would proceed to completion.

The Re-arbitration mechanism provides a mechanism to change priorities on the fly based on the ease with which a particular request can be serviced by the engine. This will improve access efficiency when multiple ports are requesting simultaneously and there is a mix of matches and mismatches to pages.

5.10.2 Address, Data, Control Multiplexers.

In addition to performing the functions listed in the previous sections, the Arbiter block also handles the routing of address, data and control signals to and from the Ports.

The $\text{AddrInn}[28:2]$ buses from all 4 Ports are split into a $\text{RasAddr}[28:12]$ component and a $\text{CasAddr}[15:2]$ component. The RasAddr multiplexing is done using the source bits corresponding to the current request. The CasAddr multiplexing is done using the AddrSource vector received from the Command Sequencer.

The DataInn and the DQMn buses contain the write data and masks from the Ports. They are multiplexed onto a single DataIn and DQM bus using the DataSource vector driven by the Command Sequencer.

The Arbiter also creates individual XferOutn signals for each port to indicate the start of a data transfer operation.

The XferOut to a Port will be asserted under the following conditions:

- The Command Sequencer indicates that a Data transfer to/from this port can proceed. This is generated by qualifying the DataXfer signal with the DataXferSource vector from the command sequencer. The XferOutn signal will be kept asserted for the entire burst duration.
- The Command Sequencer has just issued a MODE register command to the devices. In this case, there will not be an actual data transfer but the Bus/DMA engines need to be notified that the current access can terminate. The XferOutn signal will be asserted for a single clock pulse.

5.11 Arbitration FSM (SdramArbFSM)

Requests originating from each of the 4 ports are handled by a separate Arbitration FSM (AFSM). Each AFSM registers an access request – Read or Write – and then propagates it to the top level of the Arbiter for priority encoding. The generation of the request to the top level is controlled by the state transitions of a 5-state one-hot encoded machine. **Figure 5.11-1** depicts the AFSM state machine.

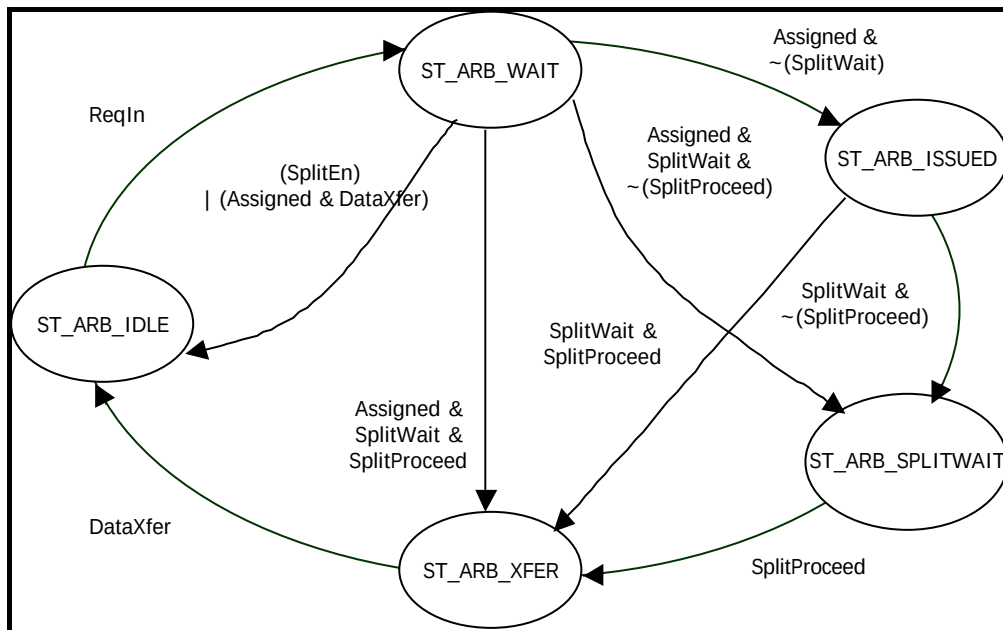


Figure 5.11-1 State diagram for the Arbitration FSM (AFSM)

On a Power-On-Reset the AFSM enters the ST_ARB_IDLE state.

In the ST_ARB_IDLE state, the AFSM samples requests on the ReqIn line. ReqIn is the logical OR of Read and Write. On sampling ReqIn high due to any of the above two sources, the machine transitions to the ST_ARB_WAIT state. The ReqOut signal is combinationally asserted to the top level of the Arbiter.

In the ST_ARB_WAIT state, the AFSM primarily determines whether the initiated access should be aborted or continued with. If an access is to proceed further, the FSM needs to receive a single clock wide pulse on the Assigned input. The following state transitions are possible for accesses that are assigned by the Dram Control Engines.

- If SplitWait is not sampled high on the clock where Assigned is sampled, the machine enters the ST_ARB_ISSUED state. This is an indication that the Control engine is not ready to issue a command yet.
- If SplitWait is sampled high on the clock where Assigned is sampled, but SplitProceed is low, the machine enters the ST_ARB_SPLITWAIT state. This is an indication that even though the Control engine is ready to issue a command, the Bus interface is not. In the AHB implementation, SplitProceed is simply a reflection of the SplitWait pulse and hence both these signals will be sampled on the same clock.
- If the SplitWait, SplitProceed and the Assigned signals are sampled high on the same clock, the machine enters the ST_ARB_XFER state. This is an indication that both the engine and the bus interfaces are ready to issue a command.

The following state transitions are possible for accesses that are not assigned by the engine.

- If the SplitEn signal is sampled, the machine enters the ST_ARB_IDLE state. This is an indication by the control engine that the request presented to it cannot be accepted on that particular clock. This could be because the current access may be a match to a busy bank or channel. Once the machine returns to ST_ARB_IDLE, it will again return to the ST_ARB_WAIT state since on a Split, the bus interface will not deactivate the original request. If the engine continues to be busy, the request could be repeatedly Split before it is finally assigned to a DRAM FSM. In this scenario, the AFSM will continue to transition back and forth between the ST_ARB_WAIT and the ST_ARB_IDLE states.

The AFSM remains in the ST_ARB_WAIT state if none of the above conditions are met. The ReqOut line to the top level of the Arbiter will continue to be asserted in this case.

In the ST_ARB_ISSUED state, the AFSM samples both the SplitWait and the SplitProceed signals. If SplitWait is asserted but SplitProceed is not, the machine enters the ST_ARB_SPLITWAIT state. This would be the case when the Arbiter is used with a bus interface that supports Split transactions. In this case the SplitWait signal from the engine would be used by the port to allow the originally Bus Master to return with the previously Split transaction. The delay between the SplitWait assertion and the SplitProceed will be several clocks. For Bus / DMA implementations which do not support Split transactions (e.g.: AHB), the SplitWait signal would simply be tied to the SplitProceed signal within the Bus / DMA interface module. Consequently, both the SplitWait assertion by the engine and its combinational reflection, SplitProceed would be sampled by the AFSM in the same clock cycle. The AFSM transitions directly to the ST_ARB_XFER state in this case.

In the ST_ARB_SPLITWAIT state, the AFSM samples the SplitProceed signal. On sampling it active the machine enters the ST_ARB_XFER state.

In the ST_ARB_XFER state, the AFSM waits for the DataXfer signal from the Command Sequencer, which indicates the first beat of data is valid. The machine returns to ST_ARB_IDLE where it can accept a new request.

5.12 DRAM Bank Controller (SdramDramCntl)

The DRAM Bank Controller instantiates four DRAM FSM blocks, and a Timing Controller.

In addition, the DRAM Bank Controller has logic to implement the following functions:

- Generation of the SplitEn, SplitWait and Assigned response signals to the Arbiter for each request posted by the Arbiter.

- Selection of one DRAM FSM instance from four, to which a request is to be assigned.
- Selection of one command to issue to the Command Sequencer from among the commands being posted by the four DRAM FSM instances.
- Generation of a combined DSetCKE[3:0] from the individual SetCKEn[3:0] signals from the four DRAM FSM instances. Each bit in the individual SetCKEn[3:0] signals refers to one slot (bit 0 – Slot 0, bit 1 – Slot 1, bit 2 – Slot 2, bit 3 – Slot 3). The combined DSetCKE[3:0] signal is generated by bit-wise ORing the individual SetCKEn signals.
- Decoding the BGCmdIssued[4:0] and FGCmdIssued[1:0] vectors to detect the issue of specific background and foreground commands. The decoded signals such as ACTIssued, RDIssued etc. are used by the Timing control block (SdramTimCntl) to track device timing parameters governing command separations.
- Generation of the EngineBusy signal. Software samples this signal through Config register 0. If this bit is sampled 0, it indicates that the SDRAM controller has completed any pending transfers and that software can modify device parameters through Mode Register Set commands. This signal is generated by using status information from the DRAM FSMs (AllIdle) and the Command Sequencer (CmdSeqIdle).
- Generation of the DiffTxxx signals for each of the timing parameters being tracked. Parameters related to the DRAM mode are checked against a value of 2 and adjusted to subtract 2 if the value is greater than 2. Else, the value of the DiffTxxx signal is set to 0. The check is made against a value of 2 because commands from the DRAM FSMs in the DRAM mode are clocked out and are issued to the memory devices after one more clock in the Command Sequencer.

5.12.1 Generation of Response signals to the Arbiter (SplitEn, SplitWait and Assigned)

Generation of the SplitEn[3:0] signal

The SplitEn signal indicates to the Arbiter that the current request that has been posted by the Arbiter cannot be accepted by the DRAM Bank controller.

A request may have to be Split if the current request that has been posted by the Arbiter is to a Bank that is Busy (i.e. a DRAM FSM indicates a Bank Match and if that DRAM FSM is busy).

The DSplitEn signal indicates the occurrence of a Split condition. Each bit in the SplitEn[3:0] output from the DRAM Bank Controller refers to a SDRAM Controller Port (bit 0 – Port 0, bit 1 – Port 1, bit 2 – Port 2, bit 3 – Port 3). The DSplitEn signal from the DRAM Bank Controller is combined with the Port address (ReqSource[3:0]) for the current request that has been posted by the Arbiter, to generate the SplitEn[3:0] output from the DRAM Bank controller.

Generation of the SplitWait[3:0] signal

The SplitWait signal indicates that the data phase for a particular request is about to begin. Each bit in the SplitWait[3:0] output from the DRAM Bank Controller refers to a Port of the SDRAM Controller (bit 0 – Port 0, bit 1 – Port 1, bit 2 – Port 2, bit 3 – Port 3). SplitWait[3:0] is asserted one clock after the request posted by the Arbiter has been assigned to a DRAM FSM.

Generation of the Assigned[3:0] signal

The Assigned signal indicates to the Arbiter that the request posted by the Arbiter has been allocated to a DRAM FSM / Mode controller. Each bit in the Assigned[3:0] output from the DRAM Bank Controller refers to a Port of the SDRAM Controller (bit 0 – Port 0, bit 1 – Port 1, bit 2 – Port 2, bit 3 – Port 3). The DAssigned signal from the DRAM Bank Controller is combined with the Port address (ReqSource[3:0]) for the current request that has been posted by the Arbiter, to generate the Assigned[3:0] output from the DRAM Bank controller.

5.12.2 DRAM FSM selection logic

Out of the four DRAM FSM instances, one FSM is assigned the current request. The selection rules are as follows:

- If there is a Bank match, the new command will be issued to that FSM provided it is not busy. If it is busy, the new request is Split (SplitEn asserted).

- If there is an FSM that is Idle and indicates a BankHit and if the SellInhibit output from that FSM is asserted, then assign the new request to that FSM. The SellInhibit signal indicates internal counters within that FSM are still tracking timing parameters for that Bank and that it should not be assigned another request that targets a different bank.
- If there is an FSM that is Idle, then the new access is issued to that FSM provided that there are no counters within that FSM, that are still tracking SDRAM timing parameters. This is indicated by the negation of the SellInhibit output from that FSM. If more than one FSM meets these conditions, the FSM is chosen according to the FSM number (FSM0 - highest priority, FSM3 - lowest priority).
- If there are no Idle FSMs, the new access is issued to an FSM that has left a bank open. That an FSM has left a Bank open is indicated by the assertion of the Act output from that FSM. The assigned FSM now has to pre-charge the bank that it has been handling and activate the assigned bank. If more than one FSM meets this condition, the FSM is chosen according to the FSM number (FSM0 - highest priority, FSM3 - lowest priority). As before, the SellInhibit output signals from the FSMs are sampled to ensure that the FSMs are free to be re-assigned a new request.

5.12.3 Selection of Command to be posted to the Command Sequencer from the DRAM Bank controller

The rules for selecting the command that is to be posted to the Command Sequencer are as follows:

- Here, data access commands (RD, RDA, WT, WTA) issued from the DRAM FSMs take higher priority over non-data-access commands (ACT, PRE etc.). The FGCmd output from each DRAM FSM instance indicates whether the command being issued is a foreground command. Then, commands associated with Port 0 requests take highest priority, while commands associated with Port 3 take the lowest priority. The SourceOut[3:0] output from each DRAM FSM indicates the Port address associated with the command being handled by that FSM. Further, command selection is done on the basis of FSM instance number. Commands from FSM0 take higher priority over those from FSM1, FSM1 takes higher priority over FSM2 and FSM2 takes higher priority over FSM3.
- When multiple FSMs assert their Issue outputs indicating an intention to issue commands to the Command Sequencer, one of them is selected, while the others have their IssueNotAcc inputs asserted by the command selection logic. A DRAM FSM instance can assume that its command has been posted to the Command Sequencer when its IssueNotAcc input is not asserted on the clock following an assertion of its Issue output.

5.13 DRAM control state machine (SdramDramFSM)

The state diagram for the command state machine is shown in **Figure 5.13-1**.

On reset, the state machine is in the ST_DCMD_IDLE state. When the Sel input is asserted, it indicates that a new request has been allotted to the DRAM FSM. If it is found that the timing parameters for the issue of the ACT command have been satisfied, an ACT command is posted to the Command Sequencer and the state machine transitions to the ST_DCMD_ACT state. The Command from the DRAM FSM is held by holding logic within the DRAM FSM block till the command has been issued to the Command Sequencer. If the timing parameters for the issue of the ACT command have not been satisfied, the state machine transitions to the ST_DCMD_ACTPND state. The state machine waits in this state till the relevant timing parameters are satisfied for the safe issue of the ACT command, after which, it posts the ACT command and transitions to the ST_DCMD_ACT state.

After RAS-to-CAS delay has elapsed, the state machine issues the RD/RDA/WT/WTA command to the Command Sequencer and transitions to the ST_DCMD_RD/ST_DCMD_RDA/ST_DCMD_WT/ST_DCMD_WTA state. After the RDA/WTA command has been issued by the Command Sequencer to the SDRAMs, the state machine transitions to the ST_DCMD_IDLE state. In the case of the issue of the RD/WT commands, the state machine transitions to the ST_DCMD_WAIT state to keep track of the fact that the bank has not been pre-charged.

In the ST_DCMD_WAIT state, the state machine waits for the next request to be allocated to this FSM. If the new request is to a different bank, then the current bank is pre-charged. If the relevant timing parameters for the issue of the PRE command are not satisfied, the state machine transitions to the ST_DCMD_ACT state, where it waits till

the relevant conditions are satisfied. The state machine then posts the PRE command and transitions to the ST_DCMD_PRE state. After the PRE command has been issued by the Command Sequencer to the SDRAMs, the state machine enters the ST_DCMD_PREWT state. In the DCMD_PREWT state, the FSM waits for the DActCnt counter to count down to zero to ensure that the PRE-ACT delay parameter is met.

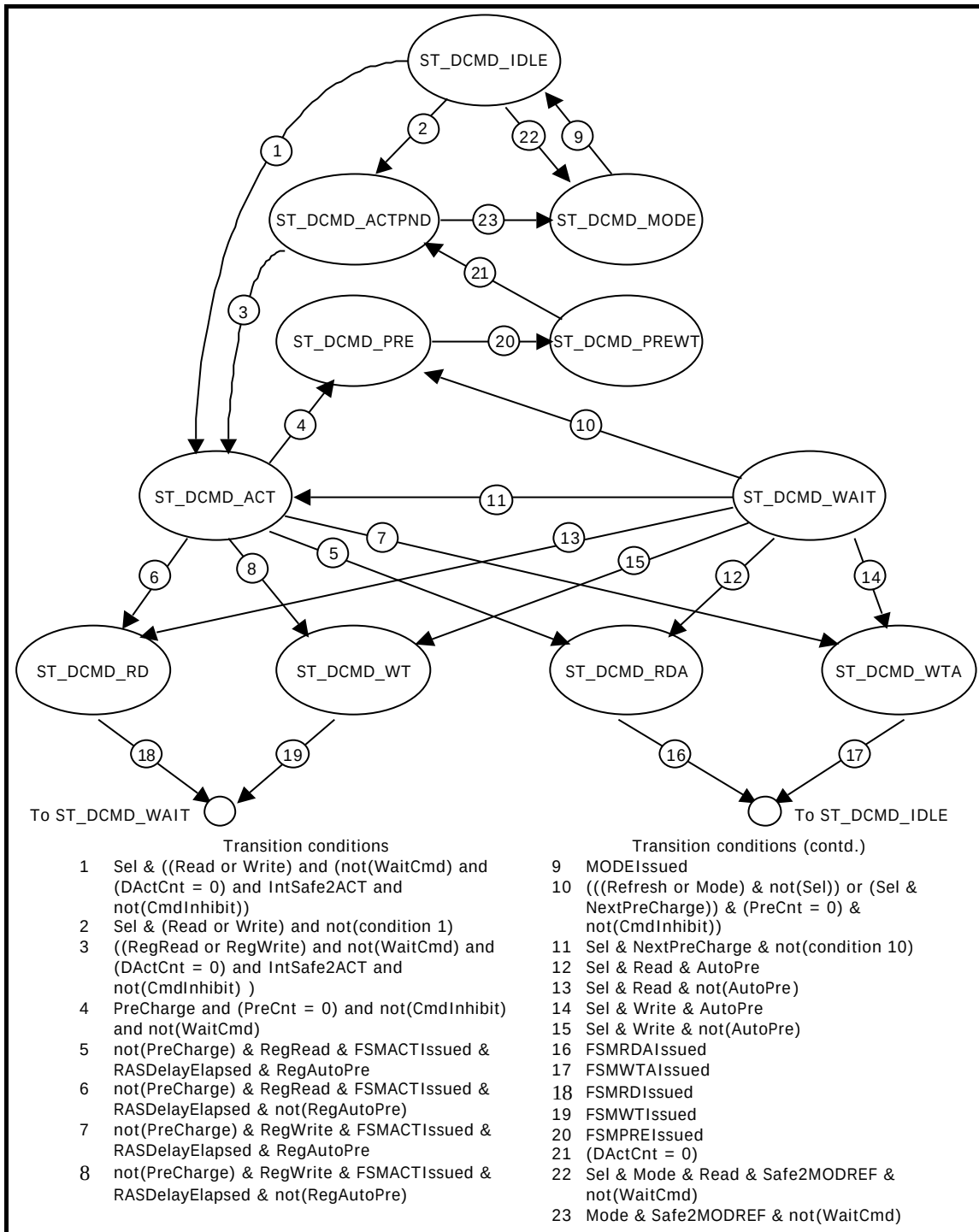


Figure 5.13-1 State diagram for the DRAM mode Command FSM in the DRAM Bank controller

If the state machine were to transition to Idle directly after loading the DActCnt counter, there is a possibility that the next request from the Arbiter targeting the same bank will get assigned to a different DRAM FSM. If that happens,

the PRE-ACT delay parameter could get violated since the other DRAM FSM has no mechanism of reading the counters in this FSM.

In the ST_DCMD_IDLE state, if a Mode request is assigned, to the DRAM FSM and if the Safe2MODREF signal is asserted, indicating that the relevant timing parameters for the safe issue of the Mode command have been satisfied, the state machine posts the Mode command to the Command Sequencer and transitions to the ST_DCMD_MODE state. If, due to some reason, it is found that the Mode command cannot be issued immediately, the state machine transitions to the ST_DCMD_ACTPND state and remains in this state till the Mode command can be safely issued. From the ST_DCMD_MODE state, the state machine transitions to the ST_DCMD_IDLE state after the Mode command has been detected to be issued.

If the Refresh signal, the IntSREFReq signal or the Mode signal is asserted and the DRAM FSM is in the ST_DCMD_WAIT state, indicating that a bank is open, the FSM pre-charges the bank and transitions to the ST_DCMD_IDLE state. After all the DRAM FSMs have returned to the Idle state, the Command Sequencer issues the Refresh/Self-Refresh command to the SDRAMs.

5.14 Timing Controller (SdramTimCntl)

The Timing Control block tracks timing parameters that govern the issue of certain background commands to the memory devices. **Table 5.14-1** lists the commands, the associated timing parameters being tracked and the signals used to convey information about whether these timing parameters have been satisfied. Each bit in the Safe2xxx[3:0] signals refers to one slot (Bit 0 – Slot 0, Bit 3 – Slot 3).

Sl. No.	Device Command	Timing parameters that govern the issue of the Command	Name of signal that indicates that the parameters have been satisfied
1	ACT (Different bank)	tRRD (ACT-ACT different bank)	Safe2ACT[3:0]
5	PALL	tRC (ACT – PALL) tRP (PRE – PALL) tRAS (RFSH – PALL) tRSA (MODE – PALL) tXSR (SREFExit – PALL) tRWL + BURSTLENGTH (WT – PALL) tAPW + BURSTLENGTH (WTA – PALL) BURSTLENGTH (RD – PALL) CASDelay + BURSTLENGTH (RDA – PALL)	Safe2PALL[3:0]
6	AREF, SREF, MRS	tRC (ACT – REF, REF – REF, MRS – REF) tXSR (SREFExit – REF) tRP (PALL – REF)	Safe2MODREF[3:0]

Table 5.14-1 List of timing parameters tracked by the Timing Controller

In addition, in the DRAM mode, after a RDA or WTA command has been issued, any command to the same device has to be inhibited till the timing parameter tAPR (for RDA) or the timing parameter tAPW (for WTA) have been satisfied. These parameters are also tracked by the SDRAMTimCntl block.

This module consists of the following counters to track these timing parameters:

- a) **ActCntx[1:0]**. (x = 0, 1, 2, 3) Four counters, one per slot, to track inter-bank parameter related to ACT.

- b) **RWPRCntx[3:0]**. ($x = 0, 1, 2, 3$) Four counters, one per slot, to track parameters related to RDA and WTA.
- c) **ActRefCnt[3:0]**. A counter to track parameters related to the issue of the REF, SREF and MRS commands. The Safe2MODREF signal is asserted when the ActRefCnt counter is zero.

5.14.1 Usage of the ActCntx counters

The tRRD parameter is tracked using the ActCntx counter (one per slot). The counter is loaded with DiffRRD (tRRD – 2 or 0, whichever is higher) whenever there is an ACT being issued to a device. The assertion of the Safe2ACT[x] signal depends on whether the ActCntx counter has reached zero.

5.14.2 Usage of the RWPRCntx counters

Any RDA or WTA access being issued to the command sequencer results in the WaitDev[x] signal for that device being asserted to inhibit any further commands to the same device before the relevant timing parameters are satisfied.

When a RDA / WTA to a device is detected to be issued to the Command sequencer, the RWPRCntx counter is loaded. The counter starts counting down when the RDA / WTA command is actually issued to the device. When the counter counts down to 0, the WaitDev[x] signal is negated indicating that further commands can be issued to the device.

In the RDA case, the value that is loaded into the counter is adjusted to account for the 2 clock read data synchronization delay.

5.15 Command Sequencer (SdramCmdSeq)

The Command Sequencer takes command requests from the DRAM Bank controller and expands them to the various logic levels on the control pins of the memory devices.

The Command Sequencer consists of a Command state machine and an Access state machine. The Command state machine deals with the issue of background commands such as ACT, PRE etc. The Access state machine deals with the issue of foreground commands such as RD, RDA, WT and WTA.

Commands from the Command Sequencer may be classified into three categories

- Background commands (non-data-access commands) that require the use of the Memory Control bus (nRAS, nCAS, nWE and nCSOut lines) only. The AREF, SREF and NOP commands fall under this category. To issue these commands the Command Sequencer asserts the ExtCtlReq and waits for the assertion of the ExtCtlGnt signal.
- Background commands that require the use of both the Memory Control bus and the Address bus. The commands that fall under this category are ACT, PRE, MODE and PALL. To issue these commands, the Command Sequencer asserts the ExtCtlReq output and the ExtBusReq output signal and the ExtBusGnt signal is waited for. It is assumed that the ExtCtlGnt signal, in these cases, would be asserted only along with ExtBusReq.
- Foreground commands such as RD, RDA, WT and WTA. These commands require the use of both the Control lines and the Address/Data bus. As with the previous case, to issue these commands, the ExtCtlReq output and the ExtBusReq output are asserted and the ExtCtlGnt input is waited for.

5.15.1 Command state machine

The Command state machine handles the initialization sequence and the issue of REF, SREF, NOP, PALL, MRS, PRE and ACT commands to the SDRAMs. The state diagram for this state machine is shown in **Figure 5.15-1**.

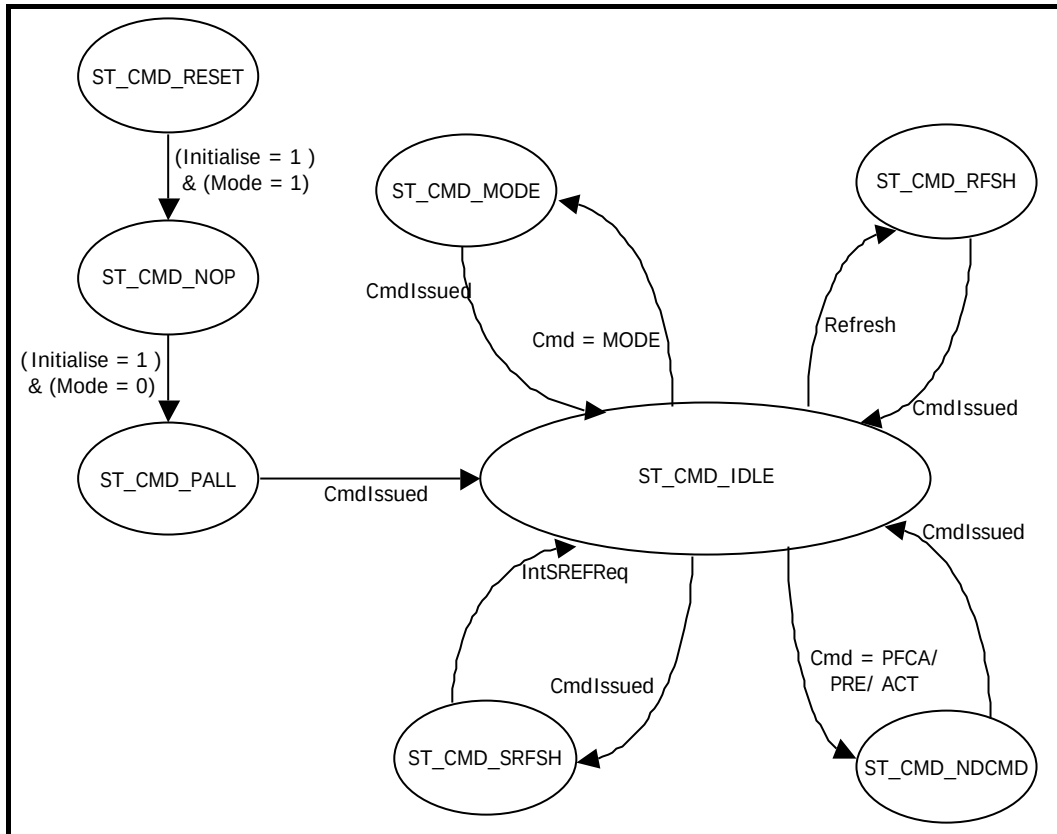


Figure 5.15-1 State diagram for the Command state machine in the Command Sequencer

On reset, the state machine is in the ST_CMD_RESET state. When the Initialize and Mode bits in Configuration register 1 are set, it marks the start of the Initialization sequence. A NOP is issued to the SDRAMs and the state machine transitions to the ST_CMD_NOP state. Subsequently, when the Initialize bit is set and the Mode bit is cleared, a PALL is issued to the SDRAMs and the state machine transitions to the ST_CMD_PALL state. After the PALL command has been issued to the SDRAMs, the state machine enters the ST_CMD_IDLE state.

The Cmd[6:0] input indicates the command being posted to the Command Sequencer from the DRAM Bank Controller. When the command being posted is a MRS, or if the Refresh / IntSREFReq input signal is asserted, the state machine issues the respective command and transitions to the ST_CMD_MODE, ST_CMD_RFSH or the ST_CMD_SRFSH state respectively. When these commands are clocked out, the ExtBusReq and the ExtCtlReq lines are asserted based on the command being issued. The command is clocked out to the SDRAMs by the external pins interface block when the ExtBusGnt/ExtCtlGnt inputs are asserted. The BGCmdIssued vector is updated to indicate the command being issued. The BGCtlHold signal indicates that the Command Sequencer has clocked out a command that requires the external control bus, but the ExtCtlGnt signal has not been asserted.

If the required command is not issued on the same clock due to the external bus not being granted or due to a higher priority foreground command also being requested on the same clock, the Command state machine keeps requesting the issue of the command till the command is actually issued.

In the case of Refresh requests, a Refresh counter is loaded with 3 to track refresh of all the 4 devices / slots. The slots are refreshed in a staggered manner to avoid a sudden surge in power consumption.

After the respective command has been issued, the state machine returns to the ST_CMD_IDLE state.

In the case of a PRE or ACT being posted to the Command Sequencer, the Command, Source address, RAS address and Device address are clocked in and used for command issue. This is because these input signals are valid only on the clock on which command is posted to the Command Sequencer. The state machine transitions to

the ST_CMD_NDCMD state for command issue and returns to the ST_CMD_IDLE state after the command has been issued.

The Next state vector of the state machine and the CmdIssue output from the state machine are used by the SDRAMCmd function to issue commands to the SDRAMs.

5.15.2 Access state machine

The Access state machine handles the issue of foreground commands such as RD, RDA, WT and WTA to the SDRAMs. The state diagram for this state machine is shown in **Figure 5.15-2**.

This state machine uses the following counters for its operation:

- a) **CasCnt[2:0]**. This counter is used to count the number of clocks to wait after a RD/RDA command has been issued to the SDRAMs, for read data to be available on the internal RdData[31:0] output signal from the Command Sequencer. When a RD/RDA is issued, this counter is loaded with (CASDelay + 2). The addition of 2 clocks is for read data re-synchronization.
- b) **DataCnt[DATACNTWIDTH-1 : 0]**. This counter tracks the number of data phases that have completed.
- c) **BurstCnt[1:0]**. This counter is used to detect whether the required number of data phase bursts have completed. The counter counts down by 1 with the start of every data phase burst.
- d) **CmdCnt[1:0]**. This counter is used to count the number of times a RD/RDA/WT/WTa command is issued.
- e) **BurstLenCnt[2:0]**. This counter is used to count the number of clocks between successive RD commands when a RD is immediately followed by a RD.

In the Access state machine, command buffering occurs in the ST_ACC_RDBUFF, ST_ACC_RD2WT, ST_ACC_WTBUFF and ST_ACC_WT2RD states. When a second command comes in and the previous command has not progressed enough to allow safe immediate issue of the second command, the second command is buffered and the state machine transitions to these buffering states. As soon as the relevant conditions regarding the data phase of the first command have been satisfied, the buffered command is issued.

The RD/RDA command to the SDRAMs is issued when the state machine transitions to the ST_ACC_RD state or the ST_ACC_RDMULT state. Similarly, a WT/WTa command is issued when the state machine transitions to the ST_ACC_WT state and the ST_ACC_WTMULT state.

The ST_ACC_WTXFER state is used to assert the DataXfer output signal to the Arbiter for write transfers.

The ST_ACC_DQM state is used to assert the DQMs for reads in the case of CAS Latency being set to 1. In this case, DQM latency remains 2. Thus, DQMs are asserted first followed by the issue of the RD/RDA command to the SDRAMs.

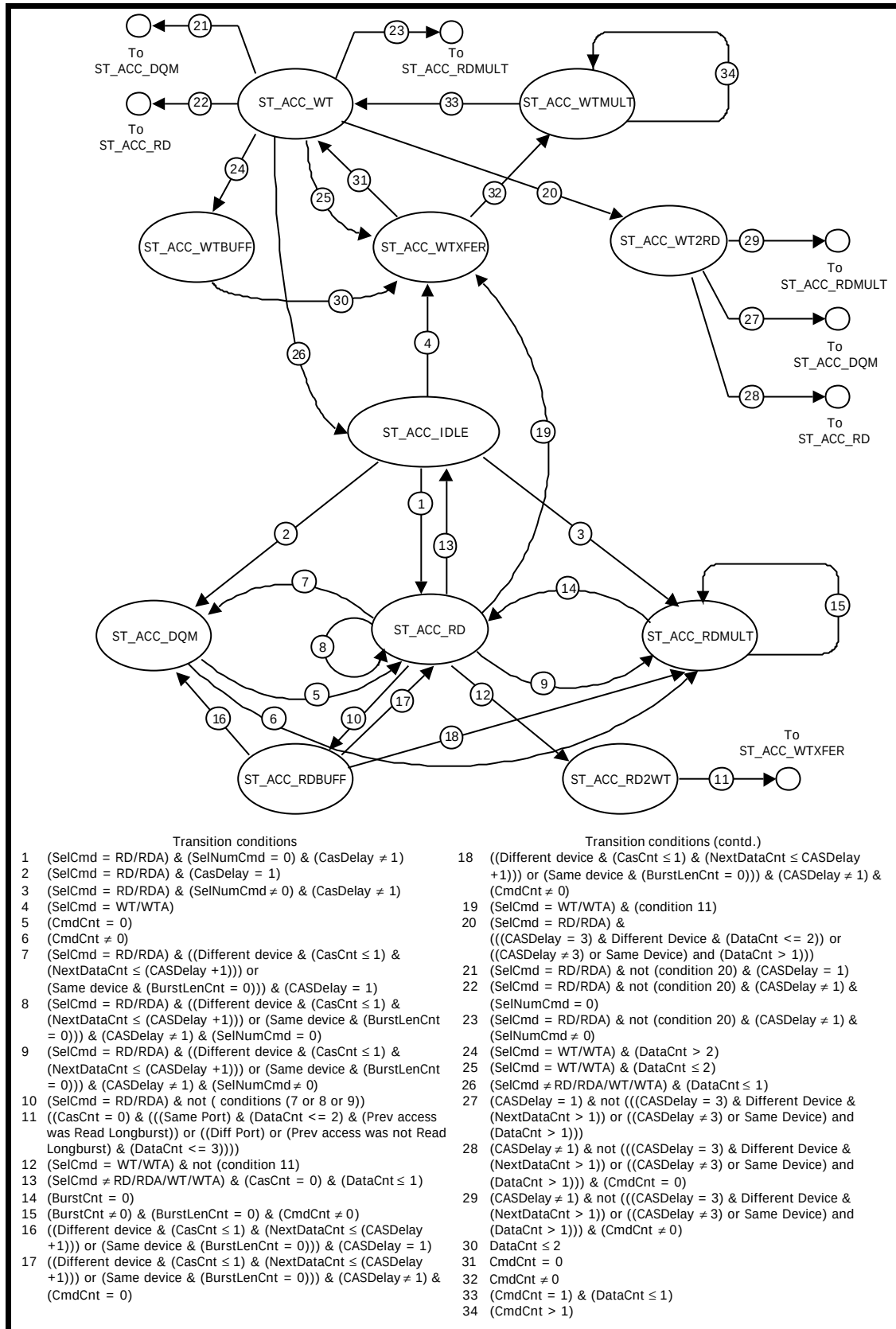


Figure 5.15-2 State diagram for the Access state machine in the Command Sequencer

In case multiple transfers are required, the state machine transitions back to the ST_ACC_RDMULT state in the case of reads and to the ST_ACC_WTMULT state in the case of writes after issuing successive reads/writes. In the case of writes, the state machine transitions back to the ST_ACC_WT state after the last of the sequence of write commands have been issued to the SDRAMs. In the case of reads, the state machine transitions back to the ST_ACC_RD state after the last of the sequence of data phase bursts has commenced.

In addition to the two commands buffered in the Access state machine, two more foreground commands can be buffered within the Command Sequencer. Commands from this buffer are given a higher priority over new commands being posted to the Command Sequencer. If the buffer is empty any new foreground commands being posted to the Command Sequencer get taken directly into the Access state machine.

In addition to the two state machines the Command Sequencer contains logic to perform the following functions:

- a) The DataXfer output signal indicates that the data phase for a request is in progress. The DataXferSource[3:0] signal indicates the port address corresponding to the DataXfer signal. These signals are generated by bit-wise ORing the WtXfer and RdXfer (for DataXfer) and by bit-wise ORing WtDXferSource[3:0] and RdDXferSource[3:0] (for DataXferSource[3:0]), which in turn are generated from decodes of the state register of the Access state machine.
- b) The WaitData signal is asserted when the Command sequencer cannot buffer any more foreground commands. This signal is used to inhibit the DRAM Bank controller from posting any more foreground commands to the Command Sequencer. The WaitCmd signal indicates that the Command Sequencer cannot accept any more background commands.

5.16 Pad Interface (SdramPins)

The Pad interface contains logic for the following functions:

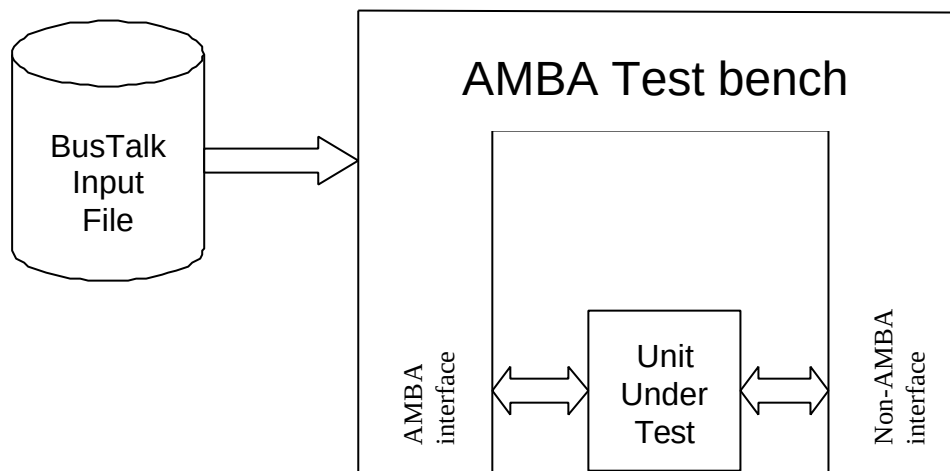
- a) Synchronization of the memory-related signals to the fed-back external clock (CLKIn).
- b) Dynamic shutdown of external clock CLKOut. If the CLKDis input signal is asserted, the ORing function ensures that the external clock CLKOut is held high.

6 SDRAM CONTROLLER AHB TEST DETAILS

6.1 SDRAM Test benches

Functional verification of the SDRAM controller uses two variants of a bus compliance test bench from the AMBA Compliance Test Suite. For further information refer to the ARM Micropack AMBA Compliance Test Suite User Guide.

The test benches are designed to be used with most common simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.



The SDRAM Controller design unit under test (RTL or netlist) connects to the test bench firstly via the AMBA interface and secondly the non-AMBA I/O connects to a module-specific test interface in the test bench. Functional test vectors are developed in 'C' using the "BusTalk" macro commands and the compiled source code is executed to generate a BusTalk Input File to be read by the test bench.

6.1.1 VHDL Bus Compliance Test bench

Figure 6.1-1 illustrates the VHDL test bench *tb_Sdram.vhd* which is located in the *BusTalk/vhdl/tbench* directory.

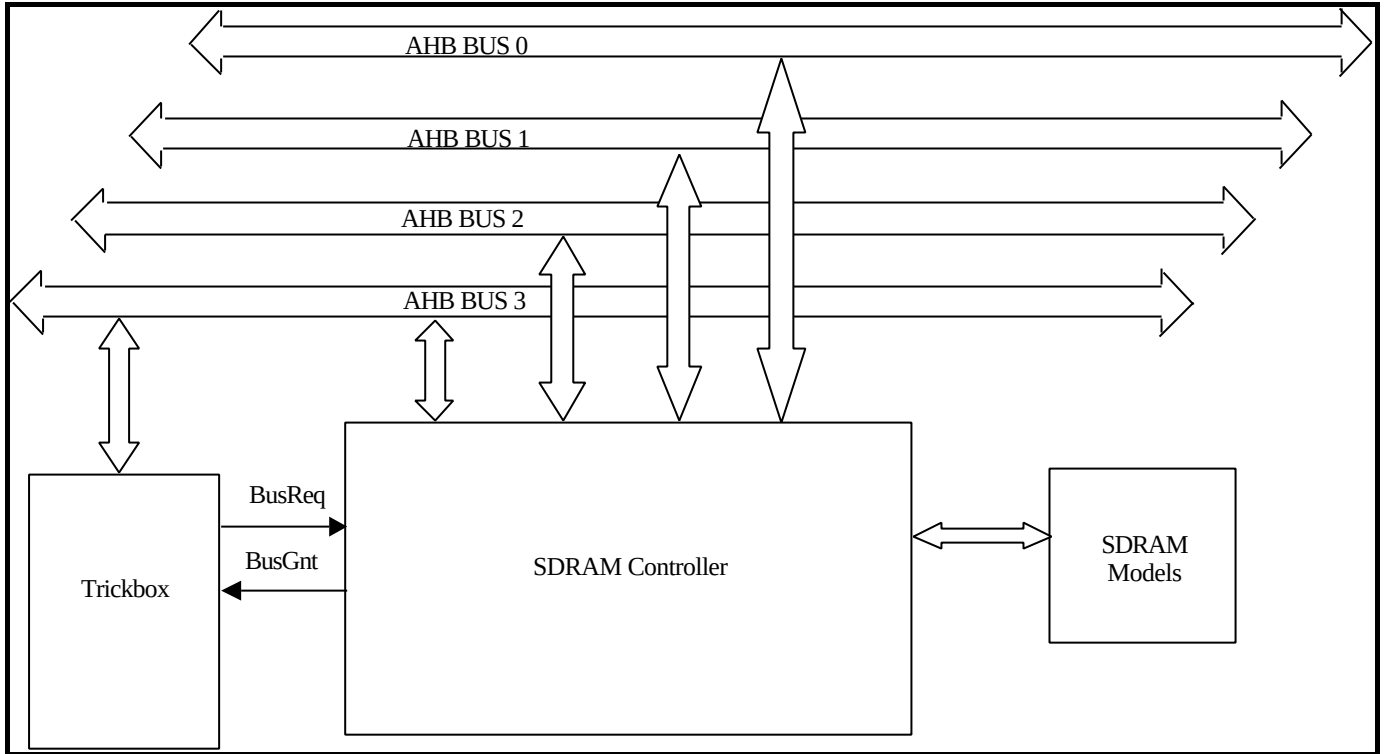


Figure 6.1-1 VHDL test bench

Figure 6.1-2 illustrates the hierarchy for the VHDL test bench.

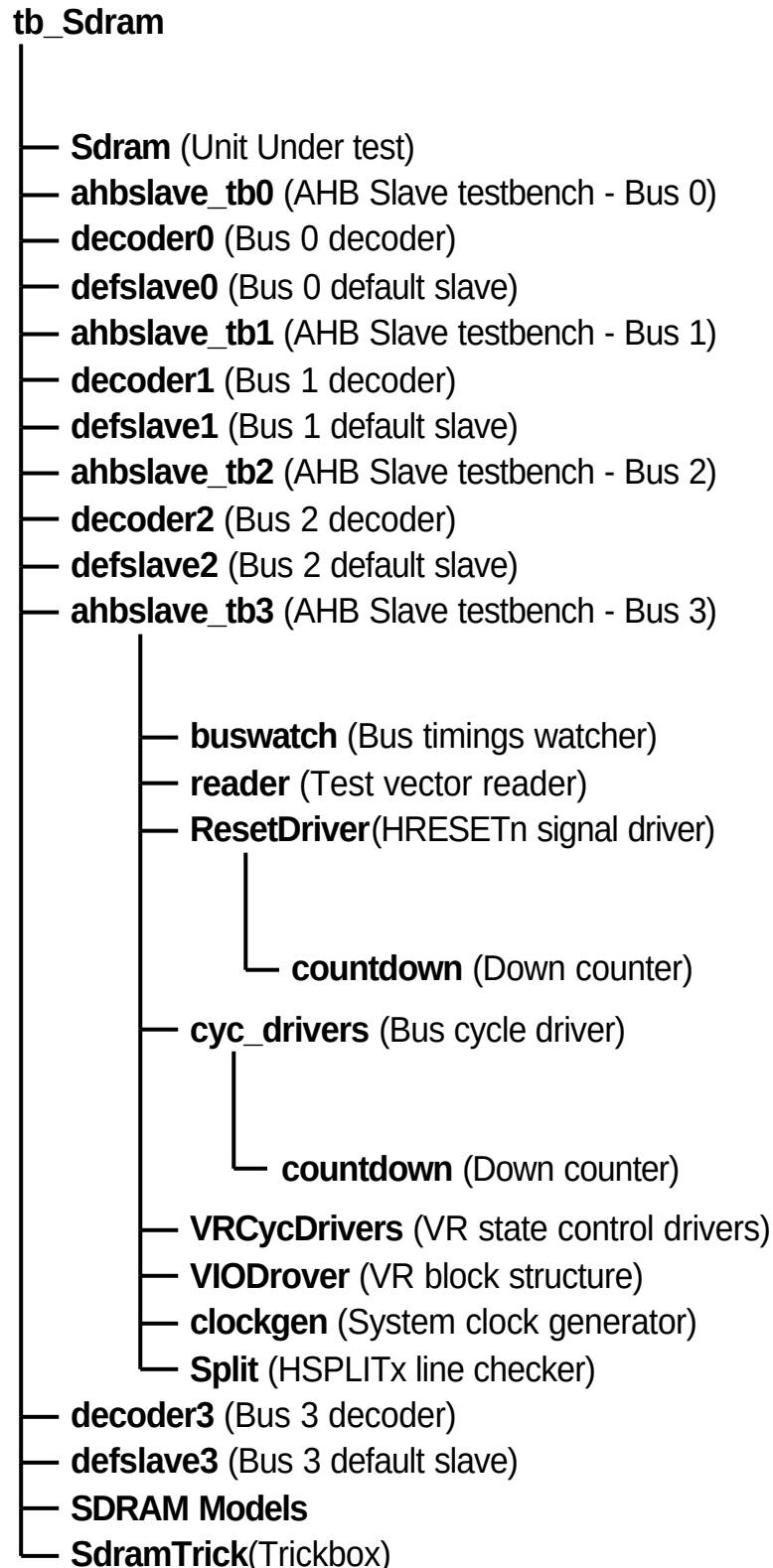


Figure 6.1-2 VHDL Test bench hierarchy

The two test benches, **tb_Sdram.vhd** and **tb_Sdram_256M.vhd** are used for functional testing of the SDRAM Controller with CAS latency values 3 and 2. Test bench, **tb_Sdram_CAS1.vhd** is used for functional simulations of the SDRAM Controller with CAS latency value 1. Each test bench instantiates the SDRAM Controller (Unit Under Test), the 4 behavioral models of the AHB masters/arbiters, the trickbox behavioral model, the decoders, the default slaves and SDRAM behavioral models.

Unit Under Test

The Unit Under Test may be one of the following

- **Sdram.vhd** VHDL RTL
- **Sdram_net.vhd** VHDL netlist

AHB Master/Arbiter Model

The UUT is stimulated primarily by four AHB Master/Arbiter behavioral models. These AHB Master/Arbiter blocks issue commands on the AHB buses under program control.

The AHB master/arbiter model, **ahbslave_tb.vhd** instantiates the buswatcher, reader, reset driver, cycle drivers, VR cycle driver, split checker and clock generator. The reader block reads the test vectors from the .bif file (from the **BusTalk/bustests/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The **cyc_drivers** drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The **VRcycdrivers** selects the appropriate VR registers for a read or write operation. The **bus_watch** module implements the protocol checks for the AHB slave's output signals and will provide warnings and error messages if it finds some mismatch. The **split** module verifies that the HMASTER is asserted on the HSPLITx line on the expected clock. The **clockgen** generates the system clock, HCLK.

In the SDRAM test bench, 4 AHB Master/Arbiter models are used for the 4 AHB buses connected to the 4 Ports of the SDRAM Controller. Each of these models instantiates a different reader. The 4 readers read test vectors from the 4 .bif files, **infile0.bif**, **infile1.bif**, **infile2.bif** and **infile3.bif** (from the **BusTalk/bustests/invec** directory).

Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. Each decoder can generate the select signals for a maximum of 16 slaves.

In the SDRAM test bench, two decoders are used. The trickbox is connected to the AHB bus that is connected to Port 3. Hence an extra HSEL signal is required for the trickbox in the decoder connected to Port 3.

Decoder 0 is used to generate HSEL[0] and HSEL[1] for the uut and the memory addresses for Ports 0, 1 and 2. Decoder 1 is used to generate HSEL[0], HSEL[1] and HSEL[2] for the UUT, the memory and the trickbox respectively. This decoder is used for accesses from Port 3.

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT via the HSEL[0] select line.

Accesses to the address range

0x50000000 to 0x5FFFFFFF

will access the Memory Models via the HSEL[1] select line.

Any access to the address range

0x60000000 to 0x600000FF

will access the trickbox via the HSEL[2] select line.

Accesses to any address outside the above ranges will select the Default Slave.

Default Slave

This module drives the response, when an access is to an unmapped region. Default slave has to drive its HREADY output high, when other slaves are not selected. It has to drive a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

Trickbox

The SDRAM Controller functionality is verified via a trickbox. The trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the SDRAM Controller and sample its non-AHB outputs.

Vector sets

The SDRAM Controller has three sets of vectors for validating functionality. All configurations of read and write buffers being enabled or disabled, address mapping being set or reset and the ExtBusWidth bit being set or reset are tested by applying the 3 sets of vectors in the different configurations.

CAS 3 Tests

The **Sdram0.c**, **Sdram1.c**, **Sdram2.c**, **Sdram3.c** files in the **BusTalk/bustests** directory exercise the functionality of the SDRAM Controller with a CAS latency of 3. These tests use the trickbox to verify proper device operation. Each test file is converted into a BusTalk Input File (*.bif), before it is applied to the Vhdl test bench, tb_Sdram.vhd.

CAS 2 Tests

The **Sdram_CAS2_0.c**, **Sdram_CAS2_1.c**, **Sdram_CAS2_2.c**, **Sdram_CAS2_3.c** files in the **BusTalk/bustests** directory exercise the functionality of the SDRAM Controller with the CAS latency 2. These tests use the trickbox to verify proper device operation. Each of these files is converted into a BusTalk Input File (*.bif), before it is applied to the Vhdl test bench, tb_Sdram.vhd.

CAS 1 Tests

The **Sdram_CAS1_0.c**, **Sdram_CAS1_1.c**, **Sdram_CAS1_2.c**, **Sdram_CAS1_3.c** files in the **BusTalk/bustests** directory exercise the functionality of the SDRAM Controller with the CAS latency 1. These tests use the trickbox to verify proper device operation. Each of these files is converted into a BusTalk Input File (*.bif), before it is applied to the Vhdl test bench, tb_Sdram_CAS1.vhd.

256M Tests

The **Sdram_256M_0.c**, **Sdram_256M_1.c**, **Sdram_256M_2.c**, **Sdram_256M_3.c** files in the **BusTalk/bustests** directory exercise the SDRAM Controller with 256Mbit memory and CAS latency 3. CAS latency 2 tests use the files **Sdram_CAS2_256M_0.c**, **Sdram_CAS2_256M_1.c**, **Sdram_CAS2_256M_2.c** and **Sdram_CAS2_256M_3.c**. These tests use the trickbox to verify proper device operation. Each file set is converted into a BusTalk Input File (*.bif), before they are applied to the Vhdl test bench, tb_Sdram_256M.vhd.

Generics

The SDRAM test bench provides some configurability options using generics. All these generics except SuppressOnReset are configurable through the tb_Sdram.vhd, tb_Sdram_256M.vhd and tb_Sdram_CAS1.vhd files. SuppressOnReset is configurable through the ahbslave_tb.vhd file in the **BusTalk/vhdl/tbench**.

XonSig

XonSig is used in the cyc_drivers.vhd file that resides in **BusTalk/vhdl/bid**.

This generic is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the test bench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in `reset_driver.vhd` and `cyc_drivers.vhd`, which reside in ***BusTalk/vhdl/bid***. It is also used in `bus_watch.vhd` which resides in ***BusTalk/vhdl/buswatcher*** and in `VIODrivers.vhd` which reside in ***BusTalk/vhdl/vr***.

This generic is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the `cyc_drivers.vhd`, which resides in ***BusTalk/vhdl/bid***, the `bus_watch.vhd` which resides in ***BusTalk/vhdl/buswatcher*** and the `VIODriver.vhd` which resides in ***BusTalk/vhdl/vr***.

This generic is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged, but the simulation continues.

Databuswidth

Databuswidth is used in the `cyc_drivers.vhd`, which resides in ***BusTalk/vhdl/bid***.

SuppressOnReset

SuppressOnReset is used in the `bus_watch.vhd`, which resides in ***BusTalk/vhdl/buswatcher***.

This generic is used to suppress all protocol checking on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slaves output signals are not checked for 'X'es when HRESETn is asserted.

Running Simulations

The **README** file in ***BusTalk/vhdl/tbench*** gives instructions for running simulations on the VHDL source code.

Example run time log files are in the ***BusTalk/vhdl/tbench/logs*** directory.

6.1.2 Verilog Bus Compliance Test benches

The Verilog functional test bench and hierarchy are as illustrated in **Figure 6.1-3** and **Figure 6.1-4**.

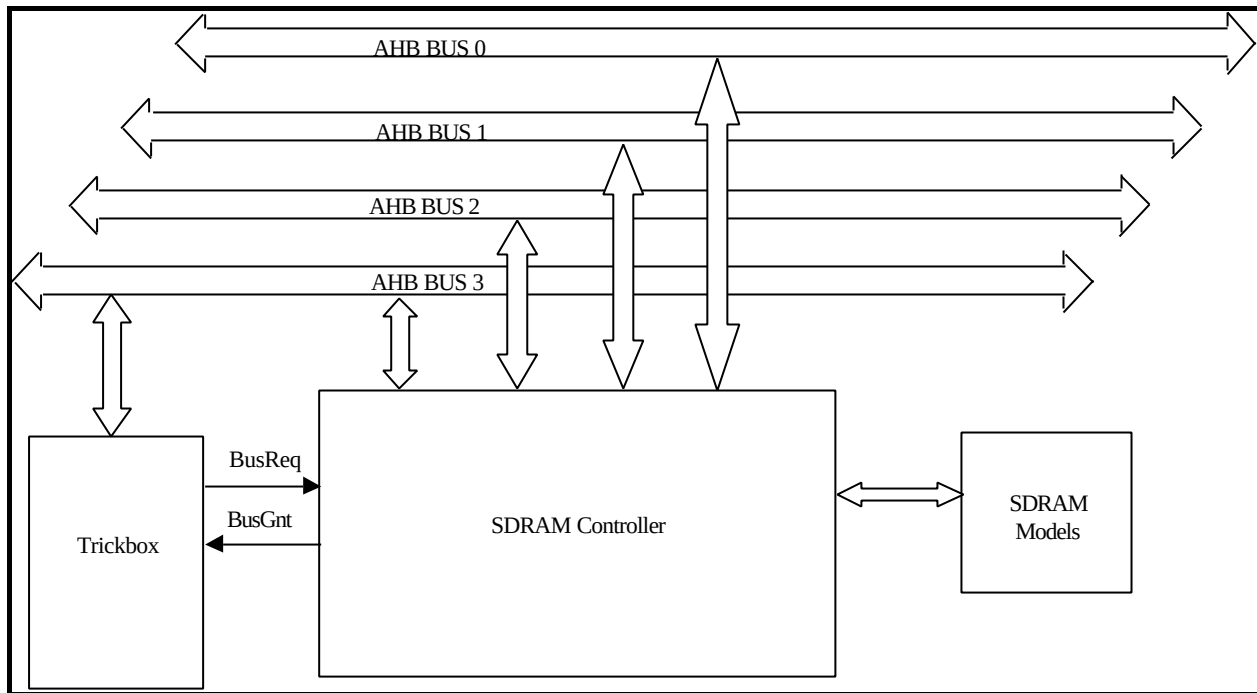


Figure 6.1-3 VERILOG Test bench for running functional test vectors

Figure 6.1-4 illustrates the hierarchy for the functional Verilog test bench.

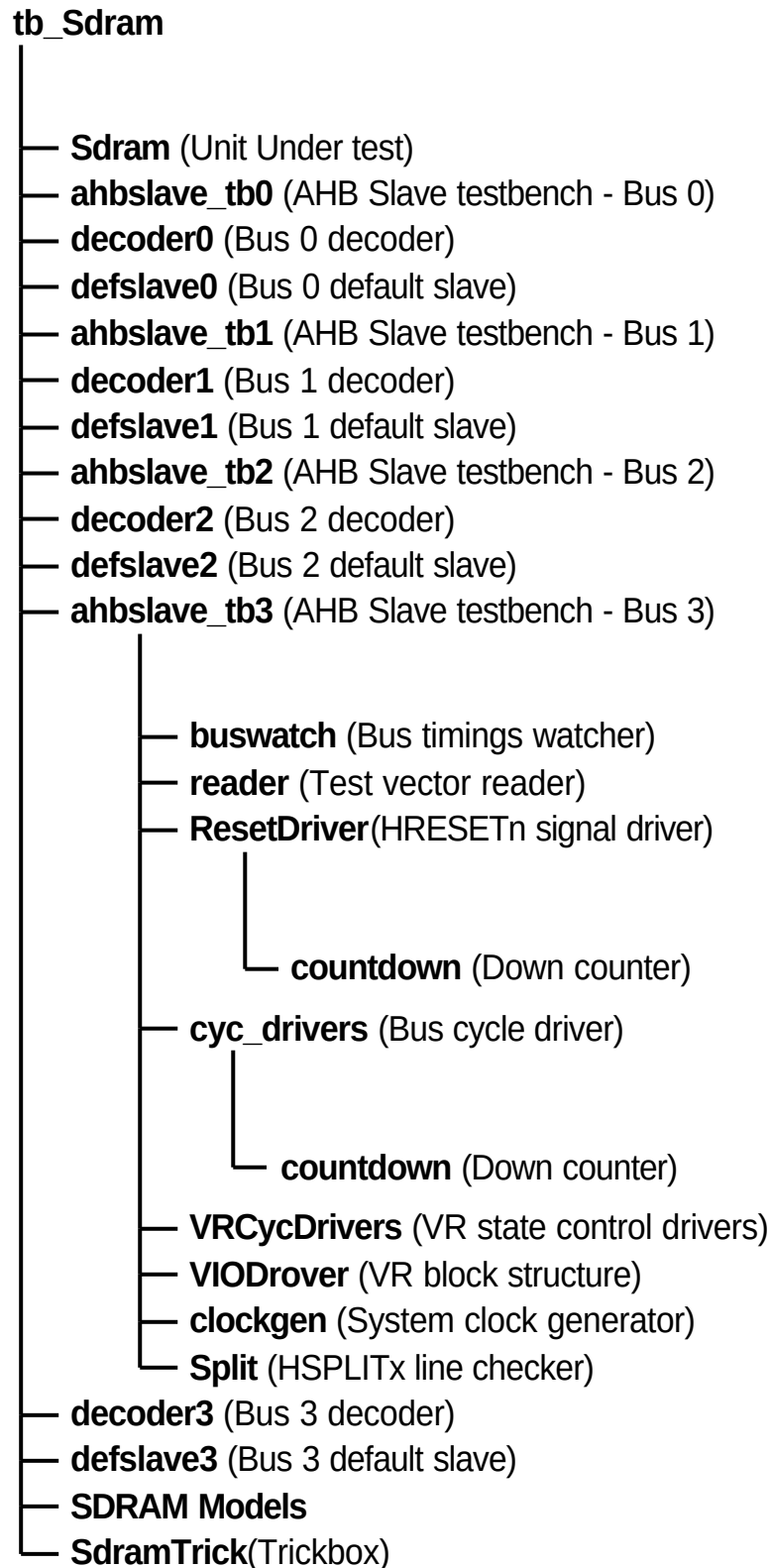


Figure 6.1-4 Verilog Test bench hierarchy for functional vectors

The two test benches, **tb_Sdram.v** and **tb_Sdram_256M.v** are used for functional testing of the SDRAM Controller with CAS latency values 3 and 2. Test bench, **tb_Sdram_CAS1.v** is used for functional simulations of the SDRAM Controller with CAS latency value 1. Each test bench instantiates the SDRAM Controller (Unit Under Test), the 4 behavioral models of the AHB masters/arbiter, the trickbox behavioral model, two decoder models, the default slaves and four SDRAM behavioral models.

The trickbox is connected to the AHB bus connected to Port 3. The trickbox drives all the non-AHB inputs to the SDRAM Controller.

Unit Under Test

The Unit Under Test may be one of the following

- Sdram.v Verilog RTL
- Sdram_net.v Verilog netlist

One out of these may be referenced via the library mapping file "libmap" in the **BusTalk/vlog/modeltech** directory.

AHB Master/Arbiter Model

The UUT is stimulated primarily by four AHB Master/Arbiter behavioral models. These AHB Master/Arbiter blocks issue commands on the AHB buses under program control.

The AHB master/arbiter model, ahbslave_tb.v instantiates the buswatcher, reader, reset driver, cycle drivers, VR cycle driver, split checker and clock generator. The reader block reads the test vectors from the .sim file (from the **BusTalk/bustests/invec** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the HRESETn signal with the correct timings. The cyc_drivers drives the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The VRCycdrivers selects the appropriate VR registers for a read or write operation. The bus_watch module implements the protocol checks for the AHB slave's output signals and will provide warnings and error messages if it finds some mismatch. The split module verifies that the HMASTER is asserted on the HSPLITx line on the expected clock. The clockgen generates the system clock, HCLK.

In the SDRAM test bench, 4 AHB Master/Arbiter models are used for the 4 AHB buses connected to the 4 Ports of the SDRAM Controller. Each of these models instantiates a different reader. The 4 readers read test vectors from the 4 .sim files, infile0.sim, infile1.sim, infile2.sim and infile3.sim (from the **BusTalk/bustests/invec** directory).

Decoder

This module decodes the addresses and generates the HSELx signals for the slaves. Each decoder can generate the select signals for a maximum of 16 slaves.

In the SDRAM test bench, two decoders are used. The trickbox is connected to the AHB bus that is connected to Port 3. Hence an extra HSEL signal is required for the trickbox in the decoder connected to Port 3.

Decoder 0 is used to generate HSEL[0] and HSEL[1] for the uut and the memory addresses for Ports 0, 1 and 2. Decoder 1 is used to generate HSEL[0], HSEL[1] and HSEL[2] for the UUT, the memory and the trickbox respectively. This decoder is used for accesses from Port 3.

Any access to the address range

0x00000000 to 0x0FFFFFFF

will access the UUT via the HSEL[0] select line.

Accesses to the address range

0x50000000 to 0x5FFFFFFF

will access the Memory Models via the HSEL[1] select line.

Any access to the address range

0x60000000 to 0x600000FF

will access the Trickbox via the HSEL[2] select line.

Accesses to any address outside the above ranges will select the Default Slave.

Default Slave

This module drives the response, when an access is to an unmapped region. Default slave has to drive its HREADY output high, when other slaves are not selected. It has to drive a zero wait state OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

Trickbox

The functionality of the SDRAM Controller is verified via a trickbox. The trickbox is a programmable AHB slave, designed to drive stimulus on the non-AHB input terminals of the SDRAM Controller and sample its non-AHB outputs.

Vector Sets

The SDRAM Controller has three sets of vectors for validating functionality. All configurations of read and write buffers being enabled or disabled, address mapping being set or reset and the Extbuswidth bit being set or reset are tested by applying the 3 sets of vectors in the different configurations.

CAS 3 Tests

The **Sdram0.c**, **Sdram1.c**, **Sdram2.c**, **Sdram3.c** files in the **BusTalk/bustests** directory exercise the functionality of the SDRAM Controller with the CAS latency 3. These tests use the trickbox to verify proper device operation. Each of these files is converted into a BusTalk Input File (*.sim), before it is applied to the Verilog test bench, tb_Sdram.v.

CAS 2 Tests

The **Sdram_CAS2_0.c**, **Sdram_CAS2_1.c**, **Sdram_CAS2_2.c**, **Sdram_CAS2_3.c** files in the **BusTalk/bustests** directory exercise the functionality of the SDRAM Controller with the CAS latency 2. These tests use the trickbox to verify proper device operation. Each of these files is converted into a BusTalk Input File (*.sim), before it is applied to the Verilog test bench, tb_Sdram.v.

CAS 1 Tests

The **Sdram_CAS1_0.c**, **Sdram_CAS1_1.c**, **Sdram_CAS1_2.c**, **Sdram_CAS1_3.c** files in the **BusTalk/bustests** directory exercise the functionality of the SDRAM Controller with the CAS latency 1. These tests use the trickbox to verify proper device operation. Each of these files is converted into a BusTalk Input File (*.sim), before it is applied to the Verilog test bench, tb_Sdram_CAS1.v.

256M Tests

The **Sdram_256M_0.c**, **Sdram_256M_1.c**, **Sdram_256M_2.c**, **Sdram_256M_3.c** files in the **BusTalk/bustests** directory exercise the SDRAM Controller with 256Mbit memory and CAS latency 3. CAS latency 2 tests use the files **Sdram_CAS2_256M_0.c**, **Sdram_CAS2_256M_1.c**, **Sdram_CAS2_256M_2.c** and **Sdram_CAS2_256M_3.c**. These tests use the trickbox to verify proper device operation. Each file set is converted into a BusTalk Input Files (*.sim), before they are applied to the Verilog test bench, tb_Sdram_256M.v.

Parameters

The SDRAM Controller test bench provides some configurability options via parameters. All the parameters are configurable through the tb_Sdram.v, tb_Sdram_256M.v or tb_Sdram_CAS1.v files.

XonSig

XonSig is used in the cyc_drivers.v file that resides in **BusTalk/vlog/bid**.

This parameter is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If **XonSig** is set to **0** in the top level of the test bench, these signals will go to '0' when the corresponding line driver is inactive. If set to **1**, the signals go to 'X' when the line driver is idle.

Verbosity

Verbosity is used in the `reset_driver.v` and the `cyc_drivers.v` which reside in ***BusTalk/vlog/bid***. It is also used in the `bus_watch.v` that resides in ***BusTalk/vlog/buswatcher*** and the `VIODrivers.v` which reside in ***BusTalk/vlog/vr***.

This parameter is used to control the verbosity of the transcript generated during simulation. If **Verbosity** is set to **1**, every command issued will have a corresponding message in the transcript file. If **Verbosity** is set to **0**, a message is issued only if an error occurs on a read.

HaltOnMismatch

This is used in the `cyc_drivers.v`, which resides in ***BusTalk/vlog/bid***, the `bus_watch.v` which resides in ***BusTalk/vlog/buswatcher*** and the `VIODriver.vlog` which resides in ***BusTalk/vlog/vr***.

This parameter is used to stop the simulation if an error occurs. If set to **1**, the simulation halts after an error is reported. If set to **0**, the error is flagged but the simulation continues.

SuppressOnReset

SuppressOnReset is used in the `bus_watch.v`, which resides in ***BusTalk/vlog/buswatcher***.

This parameter is used to suppress all protocol checking on the slave's output signals when the HRESETn signal is asserted. If **SuppressOnReset** is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If **SuppressOnReset** is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When **SuppressOnReset** is FALSE the slaves output signals are not checked for 'X'es when HRESETn is asserted.

TestMode

TestMode is used in the `cyc_drivers.v`, which resides in ***BusTalk/vlog/bid***.

TestMode if set, denotes test is being written in a Big endianness mode. Else if cleared, it is in little endianness mode.

Databuswidth

Databuswidth is used in the `cyc_drivers.v`, which resides in ***BusTalk/vlog/bid***.

Running Simulations

The **README** file in the ***BusTalk/vlog/tbench*** directory gives step by step instructions for running simulations using the tests on the Verilog source code.

Example run time logs are available in the ***BusTalk/vlog/tbench/logs*** directory.

6.2 SDRAM Controller Trickbox

The Trick Box is a programmable AHB slave, designed to drive stimulus on the non-AHB input terminals of the SDRAM Controller and sample the non-AHB outputs of the same. The TrickBox also provides a way for Data Transfer Protocol Validation.

Trickbox Signal Description

SDRAM Controller Behavioral TrickBox Signals are listed in **Table 6.2-1** and **Table 6.2-2**.

Name	Type	Source / Destination	Description
HCLK	Input	From Test Bench	AHB clock
HRESETn	Input	From Test Bench	Bus reset signal, Active Low
HADDR[ADDWIDTH:0]	Input	From Test Bench	Subset of AHB address bus
HSEL	Input	From Test Bench	Trickbox select signal from decoder. When set to 1 this signal indicates the trickbox is selected and that a data transfer is required
HSIZE[2:0]	Input	From Test Bench	AHB Data Transfer Size Indicator
HWRITE	Input	From Test Bench	AHB transfer direction signal, indicates a write access when HIGH, read access when LOW
HBURST[2:0]	Input	From Test Bench	AHB Burst type indicator
HREADYIn	Input	From Test Bench	AHB Bus Free Input
HWDATA[Width-1:0]	Input	From Test Bench	AHB Write databus
HRESP[1:0]	Output	To Test Bench	AHB Slave Response
HREADYOut	Output	To Test Bench	AHB Slave Ready Response
HRDATA[Width-1:0]	Output	To Test Bench	Output Databus to AHB

Table 6.2-1 AHB signal descriptions

Name	Type	Source/Destination	Description
CKE[3:0]	Input	From Controller	CKE driven by the SDRAM Controller
nCAS	Input	From Controller	CAS driven by the SDRAM Controller
nRAS	Input	From Controller	RAS driven by the SDRAM Controller
nCS[3:0]	Input	From Controller	Chip Selects driven by the SDRAM Controller
nWE	Input	From Controller	Write Enable driven by the SDRAM Controller
AddrOut[13:0]	Input	From Controller	ADDR driven by the SDRAM Controller
ExtBusReq	Input	From Controller	Bus Request driven by the SDRAM Controller
SREFAck	Input	From Controller	Self Refresh Acknowledge
BIGENDIAN	Output	To Controller	Selects BigEndian byte order on all AHB ports
SREFReq	Output	To Controller	Self Refresh Request
ExtBusGnt	Output	To Controller	Bus Grant output to the SDRAM Controller
NPOR	Output	To Controller	Power On Reset

Table 6.2-2 Non-AHB signal descriptions

6.2.2 Trickbox functional description

The primary function of TrickBox is to provide the non-AHB inputs to the controller and check the non-AHB outputs. The TrickBox contains the following modules.

Behavioral TrickBox

The hierarchy of the Behavioral Trickbox is shown in **Figure 6.2-3**.

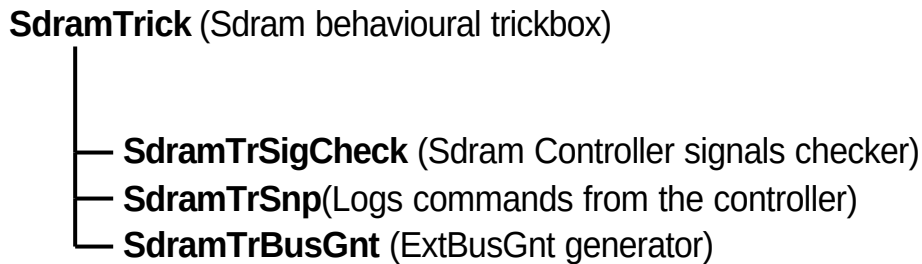


Figure 6.2-3 Hierarchy of the Behavioral Trickbox

SdramTrick

The trickbox interfaces to the external world via the AHB interface. A basic AHB slave interface is supported to provide access to the internal registers. It can respond to single read/write operations only. It does not support the Retry or Split responses.

This module also asserts the Self Refresh Request and Power On Reset to the SDRAM Controller. A Master Control Register has been provided to drive these lines under test code control.

This module, SdramTrick implements the behavioral trickbox for the SDRAM controller.

The SdramTrick module instantiates the following modules:

- SdramTrSnp - Snooper module that tracks commands issued to the Devices.
- SdramTrBusGnt - Controls the Bus Request to Bus Grant delay.
- SdramTrSigCheck - Module to check validity of command sequences issued by the controller.

SdramTrBusGnt

This module generates the ExtBusGnt Signal to the SDRAM Controller.

It implements the following ExtBusGnt generation modes:

- ExtBusGnt high at the start
- Deterministic, programmable delay between ExtBusReq and ExtBusGnt
- Random delay between ExtBusReq and ExtBusGnt

SdramTrSigCheck

This block checks the different conditions and commands issued by the Controller.

It checks the Refresh command width and also the cycles taken between two consecutive Refresh commands. It also checks for the correctness of the Refresh cycle frequency. It also provides the facility to check the issue of Self Refresh Requests.

SdramTrSnp

This module logs commands issued by the controller to the SDRAM devices.

This module "snoops" the SDRAM bus and stores the command encoding and the access address in a FIFO. This FIFO may be read or cleared under program control.

6.2.3 Trickbox Register Description

The Base address of the trickbox is not fixed and depends on the system implementation. However, the offset of any particular register from the Base address is fixed.

PMU_REG: Master Control Register [32] (+0x00)

A 15-bit read/write register which controls the BIGENDIAN, SREFReq and nPOR signals. The register address is the Base address.

Bits	Name	Read/Write	Function
31:15	Reserved	-	-
14:10	BusGntDelay	R/W	Delay between ExtBusGnt Assertion and ExtBusReq in Deterministic mode.
9	BusGntHigh	R/W	Bus Grant will be pulled high.
8	BusGntMode	R/W	Selects Deterministic/Random ExtBusGntDelay. Deterministic = 0.
7:3	Reserved	-	-
2	BIGENDIAN	R/W	Controls BIGENDIAN line.
1	POR	R/W	Controls nPOR line.
0	SREFBit	R/W	Controls SREFReq line.

Table 6.2-4 PMU_REG bit description

The SREFReq line is driven continuously if the SREFBit is set. If SREFBit is reset, SREFReq line is pulled low after sampling a high on SREFAck pin.

If the POR bit is set, the nPOR line will be asserted, at a positive edge of HCLK.

If the BIGENDIAN bit is set, the BIGENDIAN line will be asserted, at a positive edge of HCLK.

SIG_STAT: Signal Status Register [32](+0x100)

Bits	Name	Read/Write	Function
31:27	Reserved	-	-
26:11	RefCycles	R/W	No. of clocks between two Refresh requests
10	RefErrSt	R/W	Refresh frequency Check Status.
9	RefCheckEn	R/W	Refresh frequency Check Enable.
8:5	CKE	R	CKE output from the Controller
4	A10	R	Addr[10] bit output from the Controller
3	SREF	R	This will be asserted as soon as a Self Refresh is issued by the controller.
2	ModeBit	R/W	Default value 0 indicates SDRAM mode.
1	InitSeqSt	R/W	Initial Sequence Error Status. Write will clear it.
0	InitCheckEn	R/W	Enables the Initial Sequence Check

Table 6.2-5 SIG_STAT register bit description

SNOOPER_REG: Snooper Register [32] (+0x200)

This register is used to read the Snooper FIFO contents.

Bits	Name	Read/Write	Function
31:0	FIFOOut	R	Snooper FIFO Output.

Table 6.2-6 SNOOPER_REG register bit description

6.3 Functional Tests

6.3.1 CAS 3 Tests

The following section describes the functional test cases run using the test code in Sdram0.c, Sdram1.c, Sdram2.c and Sdram3.c, one file for each of the 4 AHB ports. For these tests the CAS latency value is set to 3. Only Sdram3.c contains the register tests, the initialization routine check, the buffer enable test, efficiency check, clock enable tests, clock disable tests, self refresh test and refresh frequency tests. All 4 files contain the five quad word tests, the clock disabled accesses, memory tests, the random tests, the HBURST test, the Random HBURST test, the Long HBURST test, the Deterministic HBURST tests, the Degrant test and the Busy degrant test. However, the four sets of test vectors target different addresses. This ensures that no two ports access the same address at any point of time. The addresses are chosen such that the four ports may access different quad words in the same page, but never the same quad word. Idle cycles are introduced in Sdram0.c, Sdram1.c and Sdram2.c tests to wait for the initialization and other non-memory-related tests from the port 3 to be completed. The memory accesses from the ports 0, 1 and 2 start at approximately the same time as the memory accesses from port 3.

These tests use the SDRAM behavioral trickbox. All these tests are repeated with all different configurations of the write and read buffers being enabled or disabled, the address mapping being set or reset and the ExtBusWidth bit being set or reset.

Register Tests

Reset Value Tests

After a reset, each of the following registers is read to check if it contains the specified reset value.

Register Name	Reset Value
CONFIG_REG0	0x01F00000
CONFIG_REG1	0x00000010 (Write Buffers and Read Buffer disabled) 0x00000014 (Write Buffers disabled and Read Buffer enabled) 0x00000018 (Write Buffers enabled and Read Buffer Disabled) 0x0000001C (Write Buffers and Read Buffer enabled)
REFRESHCOUNT	0x00000080
WRITETIMEOUT	0x00000000

Table 6.3-1 Register reset values

Some data is written into all the above registers. The nPOR bit is asserted and then negated after some time. Each of the registers is read to verify that it contains the specified reset value.

Register write-read test

The write time-out are inferred based on the parameters. Similarly read and write buffer enable bits in the configuration register 1 are inferred based on the parameters. To check the inference of these registers and bits various patterns are written to the registers and read back. Moreover these patterns also test that the bits are not wrongly interconnected in the code. The various patterns include 010101..., 101010..., 00110011..., 11001100... etc.

Initialization Routine Check

This test verifies that the correct initialization sequence happens when the I and M bits in the configuration register are appropriately toggled.

The trickbox is programmed for checking the initialization sequence in SDRAM mode by setting the InitSeqSt and InitCheckEn bits of the SIG_STAT register. A NOP is issued by setting the I and M bits in the CONFIG_REG1 register. After a delay, a PALL is issued by setting the I bit and clearing the M bit. A small value is programmed into the REFRESHCOUNT register and enough idle cycles are issued in order to allow at least 8 refresh pulses to pass. A large value is now programmed into the REFRESHCOUNT register and the M bit of the CONFIG_REG1 is set and mode register accesses are issued for each of the 4 DRAM slots to configure the memory in 4 burst sequential type with latency 3 for 32-32 configuration. The memory is configured in 8 burst sequential type with latency 3 for a 32-16 configuration. Finally, the InitSeqSt bit of the SIG_STAT register of the trickbox is read to ensure that the correct initialization sequence has passed without errors.

Clock Disable Test

This will tests the following functionality when the E bit is set:

A single write from port 3 is followed by a single read to verify data.

A Self refresh request is issued and the SREF bit of the trickbox Signal Status register is read to verify that a self refresh is issued.

Efficiency Check

This test verifies that when reads/writes are issued to the same page, the efficiency is improved, as an ACT is not required to open the page before every CAS. Only the first access should result in an ACT-CAS while the subsequent accesses should result in only a CAS for each access.

The Refresh register is programmed with a large value so that a refresh is not issued in the middle of this test. The snooper is enabled. A write is issued to a memory location from Port3. The second write is issued to another memory location in the same page. A read is issued to the first memory location and then to the second. The snooper is disabled and the snooper FIFO is read to verify that an ACT-CAS-CAS-CAS-CAS sequence has been followed.

Refresh Frequency Check

This test verifies that the refreshes are separated by approximately the same number of clocks as the value that is written into the RefreshCount register of the controller.

A value, say 0x18 is written into the Refresh register and also into the RefCycles field of the SIG_STAT register of the trickbox. The RefErrSt and RefCheckEn bits of the SIG_STAT register are read after enough number of idle cycles to verify that they are cleared. These 2 bits being low indicates that the expected number of clocks separates the refreshes.

The same test is repeated for different values written into the Refresh register.

Clock Enable Test

This test checks the clock enable mechanism of the controller.

The trickbox is programmed for continuous grant. After some idle cycles to wait for the busy bits of the CONFIG_REG1 to be set, the CKE bit in the CONFIG_REG0 is set. After waiting for the end of a refresh request, the CKE bits from the trickbox SIG_STAT register are read to verify that they are high even when there are no requests to the device. Again, after waiting for enough idle cycles for the busy bit to be set, the CKE bit is reset in the configuration register, CONFIG_REG0. After the end of a refresh, the CKE bits are read through the trickbox. The CKE bits should be low now.

Self Refresh Check

This function verifies that a self refresh is issued when a SREFReq input is asserted. The self refresh request is asserted by setting the bit 1 of the Master Control Register of the trickbox and resetting it after some idle cycles. The SREF bit of the SIG_STAT register is read to verify that a self refresh is issued. The SREF bit of the SIG_STAT register is read again after some idle cycles to verify that it gets negated after some time.

Self Refresh Test

This test verifies the following functionality:

When a self refresh request is issued at the same time as a normal refresh, the refresh gets higher priority and the self refresh gets delayed by a few clocks.

When a self refresh request is issued one clock before a refresh, it finishes without getting delayed. The refresh gets delayed by a few clocks.

Five Quad word Test

This function issues accesses with HBURST type INCR16 with random starting addresses. Each INCR16 write is followed by an INCR16 read starting from the address of the word next in order and also by a INCR16 read starting from the same word. This sequence of an INCR16 write followed by 2 INCR16 reads is repeated four times, with the write starting address being changed to the next word address for each repetition. The first INCR16 write may be quad word aligned or unaligned. The sequence of INCR16 writes and INCR16 reads is repeated with different starting addresses.

The Five QW accesses are issued with different sets of random addresses from all ports simultaneously to check the functionality of the controller for miss-aligned INCR16 transfers.

Buffer Enable Tests

This function checks the functionality of the controller when the WrBufEn and RdBufEn bits of the configuration register are toggled.

Longburst accesses of different types are issued from all the ports during this test. Data integrity is checked by reading the data after each write is completed.

Clock Disabled Accesses

During this test, the Clock Disable bit E is set in the configuration register 0 of the controller. Longburst accesses of different types are issued from all the ports during this test. Data integrity is checked by reading the data after each write is completed. At the end of this test, the E bit is again reset.

Memory Tests

This memory test specifically targets the AHB interface part of the design. Starting from a single data transfer of various bus sizes, it initiates various combinations of read and write transactions to test the functionality of the AHB interface. The memory is first filled with some valid data before initiating the combination of read and write accesses. The tests include

- Single data transfers to test the generation of data mask for every byte of a word individually.
- Multiple burst at various times with respect to the data transfer from the memory.
- Single and multiple burst transfers targeting the specific write buffer.
- Combination of read match / read mismatch / write match / write mismatch accesses.

Random Tests

Two clusters of locations are filled with some valid data in the memory. A random sequence of transactions is initiated with variation in address, data, bus size, transfer type etc.

HBURST Tests

This test initiates Byte, Half Word and Word data transfers from the AHB port in all the HBURST types. For each HBURST type a random sequence of Byte, Half Word and Word data writes are made. The starting address for each data transfer is aligned depending upon the data size.

The function also inserts Busy states in each of the data transfer types, at a random number of positions in the burst. The number of Busy states inserted at any position in the burst is a random number within 30.

The data integrity is checked by reading back the data once the writes are completed.

The External Bus grant delay is made random during this test.

Random HBURST Tests

This test is similar to the HBURST tests except that the HBURST type is random. That is, an INCR burst can be followed by WRAP4 or INCR16 or INCR8 or WRAP16 etc. In each HBURST type either a byte or a word or a half word transfer occurs. The function also inserts Busy states in each of the data transfer types, at a random number of positions in the burst. The number of Busy states inserted at any position in the burst is a random number within 30. The data are read back once the writes get completed, to check whether the correct data was written into the memory.

Long HBURST Tests

This test is similar to the Random HBURST tests except for the starting address that is used. A random sequence of HBURST is initiated from the AHB port. This will include long and non-long burst accesses, whose data size can be either a byte or a half word or a word. The starting address generated for each of this transfer is such that the long burst accesses will cross a Bank, Page or HBURST boundary. A random number of busy states are also introduced in each of these data writes. The data integrity is checked by reading back the data once the writes get completed.

Deterministic HBURST Tests

This test performs the following access sequences:

- A Quad word write is made to an address. This is followed by a longburst write to the same starting address. Once the writes are completed, the data integrity is checked by reading back the data using the AHB ports.
- Next a Quad word write is made to some address. This is followed by a WRAP8 or INCR8 write whose last Quad word address is same as the starting address of the previous write. The data are read back to check whether the writes occurred in the correct sequence.
- Once again the above sequence is repeated with a Quad word write followed by a WRAP16 or INCR16 write. The data integrity is checked, by reading back the data once the writes are completed.

Degrant Test

This function tests the functionality of the controller when a bus degrant occurs during a burst or a longburst.

In this test, idle cycles are inserted in a burst write of each burst type at any position. The number of idle cycles is decided randomly. After the idle cycles, the burst is rebuilt by using suitable number of transfers with burst type INCR. When the writes are completed the data is read back using the same type of burst.

Busy Degrant Test

This function tests the functionality of the controller when a bus degrant occurs during a busy transfer in a burst or a longburst.

In this test, busy transfers are inserted in a burst write of each burst type at any position in the burst. During a busy, idle cycles are introduced. The number of idle cycles is decided randomly. After the idle cycles, the burst is rebuilt by using suitable number of transfers with burst type INCR. When the writes are completed the data is read back using the same type of burst.

BigEndian byte order Test

This tests the BigEndian mode of the controller. Signal BIGENDIAN is asserted via the trick box and a sequence of bytes is written and read back followed by a word read. Then a sequence of half words is written and read back using half word reads then word reads. The sequence is repeated with BIGENDIAN de-asserted. The BigEndian test sequence is duplicated for each port. Delay loops are used to synchronize the BigEndian transfer sequence of each port with the BIGENDIAN signal state. The delay loops will need adjusting if any changes are made to the test code.

6.3.2 CAS 2 Tests, CAS 1 Tests, CAS 2 256M tests and CAS 3 256M tests

The following sections describe additional functional tests supplied for validation of the SDRAM Controller. Functionally similar tests are run on test benches with various combinations of 16 Mbit, 64 Mbit and 256 Mbit memory models. The tests cover CAS latency 1, 2 and 3 using the relevant test benches.

CAS latency 2 tests with 16 Mbit and 64 Mbit memory models use the test code in Sdram_CAS2_0.c, Sdram_CAS2_1.c, Sdram_CAS2_2.c and Sdram_CAS2_3.c for the 4 ports. CAS latency 1 tests use 16 Mbit memory models (which must support CAS latency 1 operation) using the test code in Sdram_CAS1_0.c, Sdram_CAS1_1.c, Sdram_CAS1_2.c and Sdram_CAS1_3.c for the 4 ports. 256 Mbit memory tests for CAS latency 2 and 3, use the test files Sdram_CAS2_256M_0.c, Sdram_CAS2_256M_1.c, Sdram_CAS2_256M_2.c and Sdram_CAS2_256M_3.c or Sdram_256M_0.c, Sdram_256M_1.c, Sdram_256M_2.c and Sdram_256M_3.c respectively, for the 4 ports. Only the files Sdram_CAS1_3.c, Sdram_CAS2_3.c, Sdram_CAS2_256M_3.c and Sdram_256M_3.c contain the initialization routine and checks, the self refresh tests and the clock disable tests. All 4 files of a set contain the buffer enable tests, clock disabled accesses, five quad word tests, memory tests, random tests, HBURST tests, Random HBURST tests, the Long HBURST test, the Deterministic HBURST tests, the Degrant test and the Busy degrant test. However, the four test files of a set of test vectors target different addresses. This ensures that no two ports access the same address at any point of time. The addresses are chosen such that the four ports may access different quad words in the same page, but never the same quad word. Idle cycles are introduced into the individual port tests to wait for initialization and other non-memory-related tests from port 3 to complete. The memory accesses from the ports 0, 1 and 2 start at the same time as the memory accesses from port 3.

These tests use the SDRAM behavioral trickbox. All these tests are repeated with all the different configurations of the write and read buffers being enabled or disabled, the address mapping being set or reset and the ExtBusWidth bit being set or reset.

Initialization Routine Check

This test verifies that the correct initialization sequence happens when the I and M bits in the configuration register are appropriately toggled.

The trickbox is programmed for checking the initialization sequence in SDRAM mode by setting the InitSeqSt and InitCheckEn bits of the SIG_STAT register. A NOP is issued by setting the I and M bits in the CONFIG_REG1 register. After a delay, a PALL is issued by setting the I bit and clearing the M bit. A small value is programmed into the REFRESHCOUNT register and enough idle cycles are issued in order to allow at least 8 refresh pulses to pass. A large value is now programmed into the REFRESHCOUNT register and the M bit of the CONFIG_REG1 is set and mode register accesses are issued for each of the 4 DRAM slots to configure the memory in 4 burst sequential type with latency used by the test and for 32-32 data bit configuration. The memory is configured in 8 burst sequential type for a 32-16 data bit configuration. Finally, the InitSeqSt bit of the SIG_STAT register of the trickbox is read to ensure that the correct initialization sequence has passed without errors.

Clock Disable Test

This will test the following functionality when the E bit is set:

A single write from port 3 is followed by a single read to verify data.

A Self refresh request is issued and the SREF bit of the trickbox Signal Status register is read to verify that a self refresh is issued.

Self Refresh Check

This function verifies that a self refresh is issued when a SREFReq input is asserted. The self refresh request is asserted by setting the bit 1 of the Master Control Register of the trickbox and resetting it after some idle cycles. The SREF bit of the SIG_STAT register is read to verify that a self refresh is issued. The SREF bit of the SIG_STAT register is read again after some idle cycles to verify that it gets negated after some time.

Self Refresh Test

This test verifies the following functionality:

When a self refresh request is issued at the same time as a normal refresh, the refresh gets higher priority and the self refresh gets delayed by a few clocks.

When a self refresh request is issued one clock before a refresh, it finishes without getting delayed. The refresh gets delayed by a few clocks.

Five Quad word Test

This function issues accesses with HBURST type INCR16 with random starting addresses. Each INCR16 write is followed by an INCR16 read starting from the address of the word next in order and also by a INCR16 read starting from the same word. This sequence of an INCR16 write followed by 2 INCR16 reads is repeated four times, with the write starting address being changed to the next word address for each repetition. The first INCR16 write may be quad word aligned or unaligned. The sequence of INCR16 writes and reads is repeated with different starting addresses.

The Five QW accesses are issued with different sets of random addresses from all ports simultaneously to check the functionality of the controller for miss-aligned INCR16 transfers.

Buffer Enable Tests

This function checks the functionality of the controller when the WrBufEn and RdBufEn bits of the configuration register are toggled.

Longburst accesses of different types are issued from all the ports during this test. Data integrity is checked by reading the data once the writes are completed.

Clock Disabled Accesses

During this test, the Clock Disable bit E is set in the configuration register 0 of the controller. Longburst accesses of different types are issued from all the ports during this test. Data integrity is checked by reading the data once the writes are completed. At the end of this test, the E bit is again reset.

Memory Tests

This memory test specifically targets the AHB interface part of the design. Starting from a single data transfer of various bus sizes, it initiates various combinations of read and write transactions to test the functionality of the AHB interface. The memory is first filled with some valid data before initiating the combination of read and write accesses. The tests include

- Single data transfers to test the generation of data mask for every byte of a word individually.
- Multiple burst at various times with respect to the data transfer from the memory.
- Single and multiple burst transfers targeting the specific write buffer.
- Combination of read match / read mismatch / write match / write mismatch accesses.

Random Tests

Two clusters of locations are filled with some valid data in the memory. A random sequence of transactions is initiated with variation in address, data, bus size, transfer type etc.

HBURST Tests

This test initiates Byte, Half Word and Word data transfers from the AHB port in all the HBURST types. For each HBURST type a random sequence of Byte, Half Word and Word data writes are made. The starting address for each data transfer is aligned depending upon the data size.

The function also inserts Busy states in each of the data transfer types, at a random number of positions in the burst. The number of Busy states inserted at any position in the burst is a random number within 30.

The data integrity is checked by reading back the data once the writes are completed.

The External Bus grant delay is made random during this test.

Random HBURST Tests

This test is similar to the HBURST tests except that the HBURST type is random. That is, an INCR burst can be followed by WRAP4 or INCR16 or INCR8 or WRAP16 etc. In each HBURST type either a byte or a word or a half word transfer occurs. The function also inserts Busy states in each of the data transfer types, at a random number of positions in the burst. The number of Busy states inserted at any position in the burst is a random number within 30. The data are read back, once the writes get completed, to check whether the correct data was written into the memory.

Long HBURST Tests

This test is similar to the Random HBURST tests except for the starting address that are used. A random sequence of HBURST is initiated from the AHB port. This will include long and non-long burst accesses, whose data size can be either a byte or a half word or a word. The starting address generated for each of this transfer is such that the long burst accesses will cross a Bank, Page or HBURST boundary. A random number of busy states are also introduced in each of these data writes. The data integrity is checked by reading back the data once the writes get completed.

Deterministic HBURST Tests

This test performs the following access sequences:

- A Quad word write is made to an address. This is followed by a longburst write to the same starting address. Once the writes are completed, the data integrity is checked by reading back the data using the AHB ports.
- Next a Quad word write is made to some address. This is followed by a WRAP8 or INCR8 write whose last Quad word address is same as the starting address of the previous write. The data are read back to check whether the writes occurred in the correct sequence.
- Once again the above sequence is repeated with a Quad word write followed by a WRAP16 or INCR16 write. The data integrity is checked, by reading back the data once the writes are completed.

Degrant Test

This function tests the functionality of the controller when a bus degrant occurs during a burst or a longburst.

In this test, idle cycles are inserted in a burst write of each burst type at any position. The number of idle cycles is decided randomly. After the idle cycles, the burst is rebuilt by using suitable number of transfers with burst type INCR. When the writes are completed the data is read back using the same type of burst.

Busy Degrant Test

This function tests the functionality of the controller when a bus degrant occurs during a busy transfer in a burst or a longburst.

In this test, busy transfers are inserted in a burst write of each burst type at any position in the burst. During a busy, idle cycles are introduced. The number of idle cycles is decided randomly. After the idle cycles, the burst is rebuilt by using suitable number of transfers with burst type INCR. When the writes are completed the data is read back using the same type of burst.

APPENDIX A ADDRESS MAPPING

A.1 Address Mapping turned ON

The mapping of the on-chip address to the off-chip address should typically satisfy the following two constraints:

- Each target external memory device should appear contiguous in memory.
- The bank address should be changed on the page address.

A.1.1 Address mapping.

Table A.1-1 shows the mapping between the host bus address of the DMA block and the AddrInx[28:2] address lines of the SDRAM Controller, to which the DMA block connects. Note that the AddrInx lines are **internal** to the SDRAM controller top level and correspond to the port addresses as seen by the Engine. This address is interpreted as below :

AddrInx[28:2] : Address input from Port X to the SDRAM control engine

AddrInx[28:27] : Device Address

AddrInx[26] :

- **Higher** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- **Lower** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices
- 0 (zero) if device on slot pointed by AddrInx[28:27] is populated with **2-bank** devices.

AddrInx[25] :

- **Lower** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- **Higher** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices
- Bank address bit if device on slot pointed by AddrInx[28:27] is populated with **2-bank** devices.

AddrInx[24] :

- Row address MSB if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- **Lower** order bank address bit if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices
- 0 (zero) for 2-bank devices

AddrInx[23] :

- Row Address (MSB – 1) if device on slot pointed by AddrInx[28:27] is populated with **256 Mbit** devices
- Row address MSB if device on slot pointed by AddrInx[28:27] is populated with **non-256 Mbit** but **4-bank** devices

- Bank Address bit for 2-bank devices

AddrInx[22:12] : Row Address

AddrInx[11:2] : Column Address

Ext Bus Width	Device		Row/ Column	Device Address		Bank Address		RowAddress														ColumnAddress										Mem cap
	Cap.	Data bus		A 28	A 27	A 26	A 25	A 24	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2		
32	64 Mbit	x32	Row2048 Bank4 Col256	A 28	A 27	A 11	A 10	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	32M Byte	
		x16	Row4096 Bank4 Col256	A 28	A 27	A 11	A 10	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	64M Byte	
		x8	Row4096 Bank4 Col512	A 28	A 27	A 11	A 12	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	128M Byte	
	128 Mbit	x16	Row4096 Bank4 Col512	A 28	A 27	A 11	A 10	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	128M Byte	
		x8	Row4096 Bank4 Col1024	A 28	A 27	A 11	A 12	A 11	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	256M Byte	
		256 Mbit	x16	Row8192 Bank4 Col512	A 28	A 27	A 11	A 12	A 24	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	256M Byte
	16 Mbit	x8	Row8192 Bank4 Col1024	A 28	A 27	A 11	A 12	A 24	A 23	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	512M Byte	
		x16	Row2048 Bank2 Col256	A 28	A 27	0	A 10	0	A 10	A 11	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	16M Byte	
		x8	Row2048 Bank2 Col512	A 28	A 27	0	A 11	0	A 11	A 22	A 21	A 20	A 19	A 18	A 17	A 16	A 15	A 14	A 13	A 12	A 11	A 10	A 9	A 8	A 7	A 6	A 5	A 4	A 3	A 2	32M Byte	

Table A.1-1 Mapping between System Bus Address A[31:0] and AddrInx[28:2] (Shaded regions indicate address replication specifically done to minimize multiplexing)

A.2 Address Mapping turned OFF

When Address Mapping is turned OFF, the address on the Bus is presented directly onto AddrInx[28:2]. This address is interpreted as shown in Section A.1. When Address Mapping is turned OFF, the DMA interface needs to implement the following routing of the Bank address bits :

For accesses targeting 16 Mbit devices (2-bank devices, B bit in Configuration register 0 is cleared), the DMA interface needs to implement the following mapping:

- AddrInx[26] and AddrInx[24] to be driven to 0
- Bank address on AddrInx[25] to be replicated on AddrInx[23]

This mapping is required because 16Mbit (2-bank) devices require the Bank Address to be driven on the A11 device address line. Since AddrInx[23] is driven on A11 when address mapping is turned OFF, it is required that the DMA interface drive the Bank Address on AddrInx[25] and AddrInx[23].

The SDRAM controller uses AddrInx[26:25] for bank match-mismatch calculations irrespective of the device type (2-bank/4-bank). Hence, for 2-bank devices, it is required that AddrInx[26] and A[24] be driven to 0.

For accesses targeting non-256Mbit, but 4-bank devices (B = 0 and F = 0), the DMA interface needs to implement the following mapping:

- Bank address on AddrInx[26] to be replicated on AddrInx[24]

For accesses targeting 256 Mbit devices, no such mapping is required.

Note that these mappings are applicable only when Address Mapping is turned OFF. When Address Mapping is turned ON, the mapping shown in Table A.1-1 applies.